

Grado en Ingeniería Electrónica Industrial y Automática
2023-2024

Trabajo Fin de Grado

Desarrollo de interfaz de tele-operación multimodal de robots industriales ABB vía EGM

Jorge Benito Sanabrias

Tutor

Edwin Daniel Oña Simbaña

Leganés, Septiembre 2024



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

La teleoperación es una ciencia aplicada muy extendida a día de hoy, estando presente en multitud de entornos (espacial, medicina, industrial,...) y siendo aplicada a diario en diversos contextos. Surge, como otras ciencias, de la necesidad del ser humano por adentrarse en entornos inhóspitos con el objetivo de dar soluciones a un problema establecido. Sin embargo para poder alcanzar estos objetivos implica superar barreras que, en algunos de los casos, se extienden más allá de las capacidades humanas. Es por medio de la inventiva que el ser humano idea nuevas formas de superarlas.

En este trabajo de fin de grado se expone la investigación realizada sobre este marco de aplicación y el desarrollo de un proyecto acorde, describiendo sus etapas de diseño, simulación, e implementación. Dicho proyecto se reconoce como un sistema de teleoperación multimodal que basa la comunicación de sus módulos principales en los protocolos EGM. La aplicación del sistema se enfoca en el control remoto a tiempo real de brazos robóticos industriales ABB, a través de múltiples controladores y junto a una GUI que proporciona apoyo visual. La finalidad de este proyecto es comprobar la viabilidad de la teleoperación en un robot por medio de las conexiones EGM y su adaptabilidad ante múltiples enfoques de control, posibilitando una mejor integración ante multitud de situaciones y usuarios.

Además de incluir toda la información necesaria para el entendimiento del sistema, su funcionamiento, ventajas, limitaciones y puesta en marcha de aplicación.

Palabras clave: Teleoperación; Robot Industrial; Control; Movimiento guiado; ABB; Multimodal

DEDICATORIA

Este trabajo esta dedicado a todos aquellos que me han acompañado durante este viaje.

A mi padre y a mi madre que me han impulsado a perseguir mis objetivos y luchar por ellos aunque no viese el cómo.

A mi hermana que me apoya en mis peores momentos y con la que he compartido tanto aunque jamás lo reconozca.

A aquellos individuos ejemplares que tuve el lujo de conocer en estos últimos años de universidad y a los que ahora puedo llamar amigos

A los que me llevan viendo caerme y estresarme desde el instituto pero siempre han estado ahí por si alguna vez no hubiese podido levantarme.

A los que vinieron después y con los que he compartido tanto y aun así tan poco pero aun así forman parte de una baraja muy, muy especial; todos y cada uno de ellos a su curiosa manera.

A esos 5 individuos tan singulares que decidieron escucharme un poco más, con los que puedo compartir algo más que una simple tarde alrededor de una mesa.

Y por último, a mi, por haber llegado hasta aquí. Y por continuar hacia delante.

Sic Parvis Magna (La grandeza nace de los pequeños comienzos) -Sir Francis Drake

ÍNDICE GENERAL

1. INTRODUCCIÓN.	2
1.1. Justificación.	2
1.2. Motivación	3
1.3. Objetivos	3
1.4. Marco regulador	4
1.5. Entorno socio-económico	5
1.6. Estructura del documento	5
2. MARCO TEÓRICO.	6
2.1. Teleoperación.	6
2.2. Antecedentes	9
2.3. Aplicaciones	13
2.4. Arquitecturas de Control	23
2.5. HRI	25
2.6. Controladores Modulares	27
3. METODOLOGÍA	32
3.1. Herramientas Hardware	34
3.1.1. Mando xbox	34
3.1.2. Spacenavigator	36
3.1.3. MindRove armband	36
3.2. Herramientas Software	38
3.2.1. RobotStudio	38
3.2.2. Python.	43
4. DESARROLLO	46
4.1. Implementación de interfaz gráfica GUI	46
4.2. Comunicación entre módulos	54
4.3. Estación en RobotStudio	56
5. RESULTADOS	68
5.1. Ejecución	68

5.2. Producto del proyecto	72
6. CONCLUSIONES	75
7. GESTIÓN DEL PROYECTO	77
7.1. Presupuesto	77
7.2. Diagrama GANTT	77
BIBLIOGRAFÍA	78

ÍNDICE DE FIGURAS

2.1	Elementos de un sistema teleoperado	7
2.2	Estación robótica de soldadura, montada en suelo y montada en pared . .	8
2.3	Raymond C. Goerz con su primer modelo teleoperado M1	10
2.4	Demostración de operación del Telekino acoplado a una barca	11
2.5	Handyman de Mosher, primer manipulador electro-hidráulico	11
2.6	Modelo KR FAMULUS, primer robot industrial con motor	12
2.7	Línea de montaje robotizada en la industria de la automoción	14
2.8	Robot ROBTET para el mantenimiento de líneas de alta tensión	15
2.9	Diagrama de dispositivo submarino teleoperado: Jason	15
2.10	Matriz de modelos de comunicaciones ROVs	16
2.11	Vehículo remotamente operado CURV III y Personal de la Marina posan- do con la bomba-h recuperada por el CURV I	16
2.12	Canadarm 2, sucesor del primer sistema maestro-esclavo de la firma SPAR	17
2.13	Lunokhod_1 operado en la luna 11 meses y Estación de control del Lu- nokhod	18
2.14	Puesto de teleoperación militar y Robot (esclavo) adaptado de la milicia Estadounidense	18
2.15	Tipos de vehículos militares no tripulados	19
2.16	Robot de salvamento para descombrar perteneciente al cuerpo de bombe- ros del ayuntamiento de Madrid	20
2.17	PackBot y Warrior	20
2.18	Uso del sistema tele-quirúrgico Da Vinci en una operación de Urología . .	22
2.19	Modelo prótesis biónica pie-tobillo	23
2.20	Diferentes clases de arquitecturas de control telerobótico	24
2.21	Esquema de clasificación de pHRI	26
2.22	Paradigmas de HRI cognitiva	26
2.23	Morfología de los joysticks	27
2.24	Brazo robótico teleoperado por medio de un controlador tipo exoesquele- to (UPC)	28

2.25	Representación de un sistema teleoperado por VX	29
3.1	Esquema del sistema multimodal de telecontrol via EGM	32
3.2	Estructuración de comunicaciones	32
3.3	Operación de un brazo robótico con asistencia de una GUI	33
3.4	Diagrama de controlador estilo xbox	35
3.5	Modelo Mouse 3D Spacenavigator	36
3.6	Visualización de los movimientos captados por el sensor del Spacenavigator	36
3.7	Brazalete de electrodos Mindrove armband	37
3.8	Logo RobotStudio	38
3.9	Entorno virtual de RobotStudio	38
3.10	Brazo industrial modelo IRB	41
3.11	Planos de diseño del IRB-1200	41
3.12	Representación de alcance del IRB-1200	42
3.13	Logo Python	44
4.1	Resultado de consola al leer el controlador Xboxr	48
4.2	Resultado de consola al leer el controlador Spacenavigator	51
4.3	Visualización de las GUIs de los controladores	53
4.4	Esquema de interacción de los módulos dentro del controlador robótico .	56
4.5	Graficado de los movimientos en las articulaciones del robot durante la sesión	61
5.1	Configuración del entorno robótico 1	68
5.2	Configuración del entorno robótico 2	69
5.3	Configuración del entorno robótico 3	69
5.4	Configuración del entorno robótico 4.1	70
5.5	Configuración del entorno robótico 4.2	70
5.6	Configuración del entorno robótico 5	71
5.7	Configuración del entorno robótico 6	71
5.8	Estado del controlador robótico tras la configuración	72
5.9	Operación del brazo IRB ₁ 200coneldispositivoXbox	73
5.10	Operación del brazo IRB ₁ 200coneldispositivospacenavigator	73

- 7.1 Diagrama para el manual de control xbox
- 7.2 Diagrama para el manual de control spacenavigator

ÍNDICE DE TABLAS

3.1	Actuadores del Mando	35
3.2	Tabla de características Armband	37
3.3	Características Generales del IRB 1200	43
3.4	Especificaciones de Movimiento del IRB 1200	43
7.1	Presupuesto del proyecto	77
7.2	Diagrama GANTT	77
7.3	Asignación de acciones para operación con xbox	
7.4	Asignación de acciones para operación con spacenavigator	

LISTA DE ABREVIATURAS

- AR Augmented Reality
- EGM External Guided Motion
- EMG Electromiográfica
- FRMC Force-Reflecting Manual Controller
- GDL Grado de Libertad
- HID Human Interface Device
- HRI Human Robot Interaction
- I/O Input y Output
- MR Mixed Reality
- pHRI phisical Human Robot Interaction
- ROV Remotely Operated Vehicle
- RWS Robot Web Services
- TCP Transmision Control Protocol
- TcP Tool Center Point
- UAV Unmanned Air Vehicle
- UDP User Datagram Protocol
- UdpUc User Datagram Protocol Unicast Communication
- UGV Unmanned Ground Vehicle
- VR Virtual Reality
- VRE Valor de Rango Extendido

1. INTRODUCCIÓN

1.1. Justificación

Para hablar de la teleoperación antes debemos mencionar brevemente a la robótica. Este es un campo de la ingeniería que ha tenido un origen histórico, y aunque solo ha visto un desarrollo significativo en las últimas décadas, los pasos dados en este camino han sido agigantados y aún siguen en aumento. La robótica se ha convertido en uno de los marcos con más posibilidades de la ciencia y hacia los que la humanidad dirige su avance. Siendo así interpretada como una de las llaves para el futuro, teniendo en cuenta su versatilidad en formas y aplicaciones.

Una de sus facetas más presentes hoy en día es la teleoperación. Dentro de ella se puede considerar a un gran grupo de actividades. Muchas que incluso ya se realizaban antes de la contemporaneidad en la que se sitúa el ser humano. El uso de herramientas simples, como las pinzas de un herrero, o la vara de un remecedor; hasta métodos de empleo más complejos, como el uso de dispositivos no tripulados de investigación submarina, brazos robóticos y cirugía robotizada. Todos ellos entran dentro de este campo tecnológico debido a que cumplen con la misma finalidad, ayudar al ser humano a llegar a donde no puede y manipular su entorno en contextos peligrosos para su fisonomía.

De todos modos el origen más sólido que tiene la teleoperación data en la Industria nuclear, surgido de la necesidad de manipular materiales peligrosos sin encontrarse directamente expuestos a ellos. Permitiendo la seguridad del operario y a su vez de toda la planta.

No fue hasta más adelante, al inicio de la década de los 60 que empezaron a surgir más cargos que necesitaban de esta metodología. O más bien dicho, se planteó que implementar la teleoperación en dichas labores suponía un menor riesgo y una mayor eficiencia para su realización.

Su uso inicial podría considerarse hoy en día como deficiente. Las conexiones eran inestables, los tiempos de reacción lentos, y la complejidad mecánica de los elementos manipuladores en muchas ocasiones limitante. Pero su potencial en los campos de la exploración, la investigación y el estudio; impulsó su desarrollo para expandirse en estos contextos y mejorando su eficiencia para las tareas asignadas.

Y la reciente accesibilidad de los medios para crear proyectos propios también ha resultado muy importante. Abriendo un gran abanico de posibilidades al permitir hallar nuevas formas de utilizar los elementos de los que ya disponemos, compartiéndolo y mejorándolo con otras personas.

Por ello la teleoperación en la robótica ha resultado crucial para la investigación y el progreso. El hecho de llevar al ser humano a manipular entornos más allá de su alcan-

ce le ha permitido alcanzar objetivos que hace décadas parecían lejanos, y hace siglos imposibles.

1.2. Motivación

Con frecuencia, el ser humano se enfrenta a circunstancias en las que el entorno presenta desafíos que resultan demasiado peligrosos o inalcanzables, lo que limita significativamente su capacidad para intervenir de manera efectiva. Adentrarse en entornos desconocidos siempre supone un reto, ya sea por la exposición a nuevos peligros, o por lo abrumador que resulte la tarea; sin embargo el impulso humano a superar estas barreras es lo que nos ha llevado al desarrollo, creando así tecnologías con las que de ampliar nuestras capacidades.

Las aplicaciones que se le pueden dar a la teleoperación suponen una solución clave para ello, permitiendo al ser humano expandirse más allá de sus limitaciones en multitud de entornos y configuraciones, permitiéndonos, desde poder sustituir la función de extremidades, hasta alcanzar distancias planetarias, pasando por la manipulación robótica. De este modo se abre un abanico de posibilidades de interacción con el mundo, más allá de lo que las capacidades físicas del ser humano le conceden.

En el entorno robótico, los elementos que lo componen ya son de por sí complejos de operar, por lo que es necesario dotar a los operarios de las herramientas necesarias para su control. La funcionalidad que estas desempeñan deben de ir más allá de la simple manipulación mecánica y proporcionar suficiente información y ayuda para generar una interfaz de control intuitiva y eficiente, creando así un entorno de interacción multimodal. Esta ofrece una amplia gama de usos al operador para poder aplicar el uso de los componentes e interfaces que le resulten más intuitivas en el caso que le conciernen.

Atendiendo a estas necesidades, a lo largo de este trabajo de fin de grado se explora tanto el contexto de la teleoperación como su investigación, proponiendo la implementación de un sistema telerobótico diseñado con el fin controlar un brazo industrial mediante protocolos EGM, empleando una interfaz de comunicación multimodal. Este enfoque permite la integración de múltiples controladores haciendo al sistema más accesible y adaptable ante diferentes entornos y/o aplicaciones.

1.3. Objetivos

Teleoperar en tiempo real un brazo robótico ABB a través de una comunicación EGM (External Guided Motion) que le comunique con los controladores a través de una interfaz de python.

El modelo robótico en el que se desarrolla este proyecto es un CRB_15000 en el entorno virtual de ABB “RobotStudio” y su soporte de RAPID. RobotStudio es un software

distribuido por la propia corporación ABB que permite la operación y gestión de sus propios modelos robóticos y soporte. Con la correcta configuración se puede establecer desde el programa una conexión EGM para que la comunicación en tiempo real con otros entornos sea posible.

Como controladores principalmente serán usados un mando de Xbox y un ratón 3D SpaceNavigator de Logitech. El mando de Xbox presenta una alta compatibilidad con entornos dentro de Windows evitando la necesidad de mayores adaptaciones, sin embargo se creará un entorno de Python para leer y procesar su señal. De igual modo el ratón 3D se leerá por los mismos medios, puesto que el dispositivo necesita la instalación de unos controladores específicos en el equipo, pero el entorno programado ya se encarga de que la lectura sea posible.

Por último Python ha sido el lenguaje de programación seleccionado que se usará a lo largo del proyecto por su ser uno de los principales entornos usados en la creación de aplicaciones y la gran cantidad de librerías y soporte de la que dispone gracias a la comunidad del código abierto.

Es cierto que el uso de programas e investigaciones de código abierto facilita la accesibilidad, el encontrar información relacionada con el tema de proyecto deseado y abre una gran cantidad de posibilidades a la hora de plantear un objetivo al que llegar; sin embargo, esta misma es la razón por la que impone una pronunciada curva de aprendizaje, tanto para que haya compatibilidad con el proyecto, como para comprender en su totalidad y familiarizarse con el entorno de desarrollo. Y pese a que estos puedan ser elementos limitantes, en el caso de este proyecto, se ha conseguido sobrellevarlos de forma satisfactoria para así llevarlo a cabo.

1.4. Marco regulador

En caso de que los métodos y resultados discutidos en este trabajo de fin de grado, que tratan la aplicación de programas de teleoperación vía EGM para el control de brazos robóticos ABB, sean llevados a la implementación en un entorno real deberá ajustarse a aquellas normativas bajo las que opera. El marco regulador aplicable lo constituyen La Directiva de Máquinas 2006/42/CE, las directivas ISO 10218-1:2011, ISO 10218-2:2011 e ISO/TS 15066:2016 y la Ley 36/2015 de Seguridad Nacional, que imponen directrices reguladoras relacionadas con los requisitos de seguridad para los robots industriales y su control, además de la seguridad del trabajador junto al dispositivo robótico y la protección de datos ante los sistemas de comunicación

De este modo, se asegurará que su aplicación es segura para el entorno industrial y el usuario sin intervenir de manera dañina o peligrosa en el correcto funcionamiento de todos sus sistemas.

1.5. Entorno socio-económico

Al tratar de un marco de implementación suficientemente extendido en aplicaciones contemporáneas el impacto socio-económico que podría tener la aplicación de los procedimientos tratados en este proyecto son reducidos. Sin embargo, no son nulos, pues las investigaciones y aplicaciones de los protocolos de comunicación empleados, pueden resultar sólidos puntos de partida para la mejora de la eficiencia industrial y el avance en la intromisión a accesos remotos; así como la modularidad del sistema puede contribuir al desarrollo en el acceso a la tecnología y la democratización de la robótica.

1.6. Estructura del documento

El documento está dividido en 7 capítulos. El capítulo 1 resulta la introducción al documento del trabajo de fin de grado, tratando la justificación, motivación y objetivos del proyecto desarrollado; además de un vistazo al marco regulador y el impacto socio-económico del mismo. El capítulo 2 aborda el marco teórico del proyecto, con la investigación de los antecedentes en el campo tecnológico tratado. El capítulo 3 expone tanto la metodología seguida a lo largo del proyecto como los detalles de los elementos empleados en su realización. El capítulo 4 se adentra en el desarrollo del proyecto, explicando detalladamente el funcionamiento de cada una de sus partes y como se relacionan entre sí. El capítulo 5 narra los resultados obtenidos al finalizar el proyecto al igual que descripciones del estado último del mismo. En el capítulo 6 se abordan las conclusiones a las que se ha llegado durante la realización del proyecto. Y finalmente en el capítulo 7 se presenta una tabla de presupuestos cuantificando los gastos invertidos para poder llevar a cabo el proyecto y las horas empleadas en el desarrollo y documentación del mismo. También se incluye anexos al final del código

2. MARCO TEÓRICO

2.1. Teleoperación

El concepto de teleoperación se origina de la contextualización de su prefijo “tele” ($\tau\eta\lambda\epsilon$), que significa “distancia”, dentro del campo de la industria y la robótica. En estas aplicaciones se le refiere como a una distancia que implica la existencia de una barrera entre un sujeto y el entorno, la cual puede ser entendida como cualquier factor que impida al usuario alcanzar o manipular por sí mismo un entorno definido; como que el medio implique algún tipo de riesgo, que su escala tenga una diferencia excesiva haciendo imposible su control, que existan impedimentos físicos o que se interpongan las propias limitaciones de la fisiología humana. Pese a ello, el ser humano ha tratado de superar estos obstáculos con la tecnología, con el control remoto de máquinas, robots o sistemas especializados como se irá viendo a lo largo de la investigación de este proyecto.

Independientemente de los modelos y la distancia presente entre operador y esclavo, por lo general podemos diferenciar los sistemas de teleoperación en dos etapas principales:

- Local: en ella se comprende el elemento humano y los componentes que permiten la conexión entre el sistema y el usuario. Por ejemplo controladores, display, pantalla y sistemas de I/O entrarían dentro de esta categoría .
- Remota: esta otra abarcaría los elementos fundamentales en el otro extremo del esquema de la teleoperación. Como el propio entorno, los actuadores, el esclavo, sensores de apoyo y elementos de control

Al entrar en más profundidad dentro de las etapas que componen el sistema teleoperado nos encontramos con diversos términos atribuidos a los elementos que lo componen que merece la pena definir de antemano:

En el entorno local en primera instancia se halla el “Operador”, sujeto encargado del sistema que, de manera remota, se ocupa de supervisar o controlar el dispositivo teleoperado por medio de la interfaz que conecta el sistema. Este “Interfaz” resulta el medio compuesto por distintos dispositivos del sistema de teleoperación que permiten la comunicación mediante los canales físicos entre Operador y Esclavo. Ofrece al Operador las herramientas y datos imprescindibles para poder comandar correctamente el sistema remoto y a su vez recibir información del mismo. Pero para poder tener acceso a la interfaz, el usuario lo hace a través de un “Controlador” que se trata de un componente de la interfaz que tiene la exclusiva función de introducir al sistema las órdenes del Operador sobre el Esclavo

Entre el entorno local y el remoto se pueden encontrar otros módulos del sistema, tales como los “Canales de comunicación”, llamados así a los medios por los que se transmiten los datos allá donde sea necesario dentro del sistema. Estas conexiones pueden ser llevadas por medio de cables o conexiones inalámbricas; y estos varían dependiendo de la distancia entre Operador y Esclavo y las necesidades para llevar a cabo dicha comunicación. Y luego está el módulo del “Control” responsable de traducir los comandos enviados desde la interfaz para que lleguen al dispositivo teleoperado en un formato que comprenda. Tanto los Canales de comunicación como el Control componen el conjunto de medios principales de intercambio de información que modulan y envían las señales o conjunto de ellas entre el elemento Esclavo del sistema y el Operador sirviendo como puente entre sendos entornos.

Por último, ya en el entorno remoto se identifica el dispositivo teleoperado o “Esclavo”, definido como sistema, robot, máquina o similar que trabaja en base a unas órdenes dadas por un sujeto humano a distancia remota. Este dispositivo extiende la capacidad de observación y/o manipulación del Operador en un entorno apartado de este o junto a él, dependiendo de la configuración que presente el sistema. Además el elemento Esclavo también es capaz de percibir su entorno por medio de “Sensores”, elementos presentes a lo largo de cada segmento del sistema que no solo recogen los datos del entorno remoto sino también del funcionamiento del resto de componentes [1], [2].

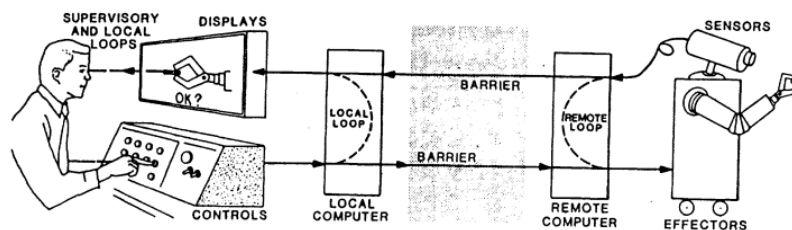


Fig. 2.1. Elementos de sist. teleoperado [3]

Para apoyar el funcionamiento de la teleoperación en muchas ocasiones se integran aplicaciones de otras áreas dependiendo de las funcionalidades necesarias; de la robótica para la realización de comandos y para aportar cierto grado de autonomía al sistema; biónica con la que se consiguen imitar estructuras, cualidades o procesos de organismos biológicos, incluidos el humano para desempeñar diferentes tareas; medicina con la que reconocer parámetros vitales, ya sea para el uso de estas tecnologías en la rama sanitaria o para integrar el control de los procesos a un nivel más fisiológico e intuitivo; entre muchas otras. Todo ello tiene como objetivo pulir la ejecución del sistema teleoperado para cumplir sus funciones en el campo específico para el que ha sido diseñado.

Una de las aplicaciones más notorias dentro de la teleoperación como podemos ver, es la telerobótica, pues se enfatiza en el control remoto de un robot a tiempo real. Esta expresión de la teleoperación es la más extendida de la industria casi convirtiéndose en la segunda definición del término a nivel práctico.

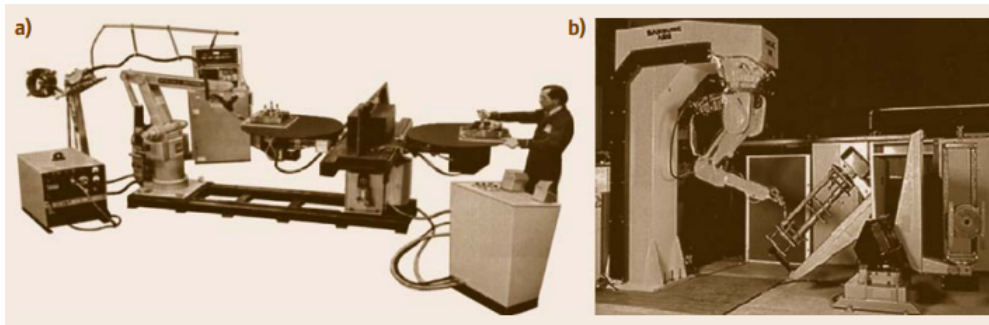


Fig. 2.2. Estación robótica de soldadura, montada en suelo (a), montada en pared(b) [4]

Respecto a las comunicaciones que se originan durante los eventos de teleoperación, el modo en el que estas se dan varían enormemente dependiendo de las características que se requieren para su desempeño. Por ello los componentes de comunicación y los sensores que permiten la detección de los parámetros en cada contexto son imprescindibles.

En sistemas con una gran distancia de por medio entre entorno y operador se recurre a comunicaciones inalámbricas, que permiten salvar estas distancias además de llegar allá donde los cables no pueden. Las redes permiten el diseño de nuevas arquitecturas multilaterales, con las que se consigue conectar a varios usuarios a un mismo sistema, o a un usuario con varios sistemas simultáneamente, logrando establecer un entorno colaborativo teleoperado. Sin embargo el uso de redes inalámbricas supone cierto grado de precariedad, pues son menos fiables y rápidas que su contraparte. Esto puede desestabilizar los bucles de control que se usan para los reajustes de medidas tales como la fuerza, movimiento, posición y velocidad del elemento esclavo. Mientras tanto las conexiones directas por medio de cables o circuitos resultan más sólidas y fiables, sumado a que los errores por fallo de conexión son más sencillos de localizar.

Por otra parte, el control utilizado en los procesos de teleoperación abarca un espectro de arquitecturas. A este espectro, sus extremos los componen el control directo y el control supervisor

- Control directo o manual: el operador controla el movimiento del componente esclavo directamente, sin ninguna clase de ayuda o medios automatizados, haciendo que el elemento esclavo replique equivalentemente la información introducida al sistema por parte del controlador.
- Control supervisado: los métodos de comando y retroalimentación suceden a alto nivel, puesto que para funcionar el nivel requiere de cierto grado de autonomía.

Pero el espectro de control no se excluye a estas dos modalidades; si no que dispone de niveles intermedios que requieren de características de ambos en mayor o menor medida.

Muchos sistemas requieren de alguna clase de control directo. Como en las pruebas realizadas en este proyecto, los joysticks y displays son elementos clave de la interfaz del usuario para el control del robot/elemento esclavo.

El joystick es un instrumento que en el contexto de la teleoperación puede considerarse un robot de categoría local. La diferenciación entre robot local y remoto reside en su papel como elementos maestro y esclavo del sistema teleoperado respectivamente. Para conseguir un control directo el robot esclavo es programado para seguir los movimientos del robot maestro que es a su vez manipulado por el operador. En muchos entornos el robot maestro resulta ser, en algún grado, una réplica cinemática del robot esclavo proporcionando al usuario un control más intuitivo

Existen sistemas retroalimentados maestro-esclavo capaces de enviar al operador los datos recibidos desde los sensores situados en el entorno remoto, para darle información sobre diferentes factores que se “sienten” desde la etapa remota. De forma que el robot maestro no solo determina los movimientos, sino también muestra información sobre estos eventos adicionales; como la representación de la fuerza inculcada en el entorno remoto por un brazo robótico, transcrita al entorno local. Este comportamiento recibe el nombre de Interfaz Bidireccional o Sistema Háptico. Estas características, aunque se aplican más hoy en día para integrar al usuario en entornos virtuales que en entornos remotos, también son sumamente útiles en los diferentes campos de aplicación de la teleoperación.

La telepresencia es considerada como el fin último de los sistemas maestro-esclavo y la telerobótica. Según la teoría, esto permitiría al usuario no solo manipular el entorno remoto, si no percibirlo como si estuviese directamente allí; ofreciéndole los estímulos y sentidos de retroalimentación necesarios. Cuando se consigue este efecto, idealmente, provoca que la cognición del usuario deje en segundo plano el medio de control, viéndose envuelto por el entorno remoto. Llegados a este caso, se podría clasificar al sistema teleoperado como “transparente”

Pero invertir recursos del sistema en este aspecto de la teleoperación también plantea problemas de estabilidad y control. Por ello mismo es uno de los principales focos de atención para la investigación [3].

2.2. Antecedentes

Históricamente el origen de la teleoperación propiamente dicha se remonta a su investigación en la Industria nuclear. En 1947 Raymond C. Goerz junto a un equipo de investigación en Chicago comenzaron el desarrollo el primer teleoperador maestro-esclavo diseñado específicamente para manejar los materiales radiactivos en las plantas nucleares, que les permitiría hacerlo tras la seguridad de los muros protectores sin necesidad de exponerse a la radiación. Sin embargo no fue hasta 1948 que el primer modelo de este tipo estuvo operativo, el M1 se convertiría en el punto de partida de todos los sistemas de teleoperación maestro-esclavo hasta la fecha [1], [3], [5].

El M1 se componía de un sistema de mecanismos de pantógrafo que permite al operador manipular los materiales peligrosos desde fuera de la “célula caliente” donde se llevaba a cabo esta tarea [1]. Unas pinzas instaladas en el entorno remoto reproducen fiel-

mente los movimientos realizados a mano por el operador desde el extremo local [3], [4]. Al tratarse de un mecanismo de pantógrafo, ambos extremos eran prácticamente idénticos y los movimientos se transmitían de uno a otro eje a eje a una escala equivalente.



Fig. 2.3. Raymond C. Goerz con su primer modelo teleoperado M1 [4]

Pronto los actuadores eléctricos suplantarían a los enlaces mecánicos de cintas, palancas y cables. Junto a ello la televisación del entorno remoto, abriendo la posibilidad de situar al operador a una distancia arbitraria [1]. El modelo E1, también diseñado por Goerz, sería el siguiente en esta línea [4], [6].

A pesar de tener un punto de partida tan definido, merece la mención a un precursor de la teleoperación mucho antes que Goerz. La primera solicitud de patente registrada de una operación a distancia fue el Telekino, de la mano de Torres de Quevedo, que presentó su invención en 1903 en la Academia de Ciencias de París. En esta presentación realizó una demostración experimental en la que el dispositivo interpretaba y ejecutaba órdenes a distancia, transmitidas mediante ondas hertzianas. Esta hazaña le consiguió el privilegio de invención en España (Un sistema denominado "Telekine" para gobernar a distancia un movimiento mecánico, patentes 31918 de 10/6/1903 y 33041 de 9/12/1903), Gran Bretaña y Estados Unidos. No fue hasta más adelante, en 1905 que implementó su diseño a un coche eléctrico y una pequeña barca, convirtiéndolos en los primeros aparatos de radiodirección del mundo. Finalmente este invento fue reconocido por la IEEE como un hito "Milestone" para la historia de la ingeniería a nivel mundial en 2006 [7], [8].

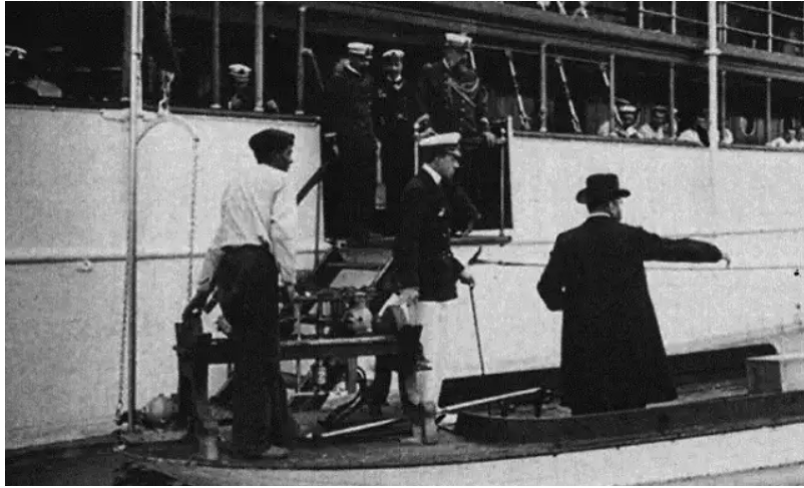


Fig. 2.4. Demostración de operación del Telekino acoplado a una barca [8]

Reanudando el hilo histórico, a mediados de la década de los 50, las tecnologías en la teleoperación empezaron a rondar el concepto de la “telepresencia”, con el objetivo de que su funcionamiento se integrase de manera más natural en la industria. Comenzaron a enfocarse en que los sistemas móviles fueran capaces de, como mínimo, ejercer fuerzas en los seis GDL (Grados de Libertad) de manera simultánea, al mismo tiempo que se investigaba el uso de pantallas y controladores montados en la cabeza de los operadores para crear así una telepresencia visual [1]. En este punto destacó el Handyman, desarrollado por R. Mosher y General Electric. El diseño contaba con dos brazos electrohidráulicos, superando con creces las expectativas del momento, pues cada uno de los brazos del Handyman ofrecía diez GDL.

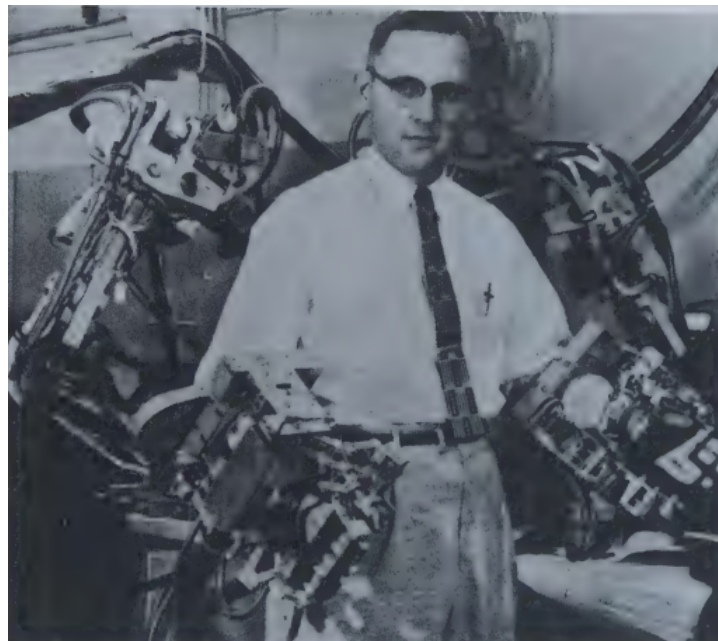


Fig. 2.5. Handyman de Mosher, primer manipulador electrohidráulico [1]

Para finales de los 50, empezó a surgir un interés sobre el uso de las nuevas tecnologías en comunión con el cuerpo humano. Se buscaba mejorar el estilo de vida de los pacientes con necesidad de prótesis integrándoles a estas los servomecanismos del mismo modo que se utilizaron con los sistemas maestro-esclavo casi una década antes. Numerosos desarrollos se llevaron a cabo en este tiempo, pero uno de los primeros en tener éxito, y posiblemente uno de los más importantes es el experimento de Aaron Kobrinskii en Moscú en 1960; su modelo constaba de una prótesis del segmento inferior del brazo que era comandada por señales mioeléctricas de los músculos todavía conservados de la intervención. Este hito resultó ser un precedente que muchos otros siguieron para ahondar más en esta especialidad [1], [3].

A partir de los 60 la teleoperación empezó a conectar un gran abanico de campos, aunque solo fuese en el papel, puesto que suponía lograr un mayor desempeño en muchas tareas demasiado complicadas o peligrosas para el ser humano, como la exploración submarina y espacial. Su desarrollo en carrera espacial reveló un gran problema, y este fue a causa de los retrasos en las comunicaciones. Este impedimento obligó a utilizar un método de control de “mover y esperar”, sometiendo al sistema a realizar movimientos en bucle abierto, y luego esperar confirmación antes de proceder con el siguiente [1]. Desde entonces el potencial de sus aplicaciones ha llevado a esta ciencia a pasar por diferentes etapas de investigación y desarrollo; desde experimentos iniciales donde se buscaba entender los efectos de los retrasos en las comunicaciones, hasta la implementación métodos de control más eficientes como el control supervisor; u otros basados en la pasividad y el análisis de Lyapunov, que pese a tratarse de aplicaciones teóricas, resultan potenciales nodos de desarrollo [5].

Ya desde el 70 en adelante, la teleoperación comienza a coger velocidad. En este punto empieza a converger simbióticamente con otras áreas de la tecnología, siendo la más importante su relación con la robótica industrial, con la que iría de la mano desde entonces. Sin embargo sus avances conjuntos no fueron muy fructíferos en sus primeras etapas, aunque es a causa de ello que las prácticas de la teleoperación empiezan a descubrir aplicaciones viables en muchas otras áreas [1], [3], [9].

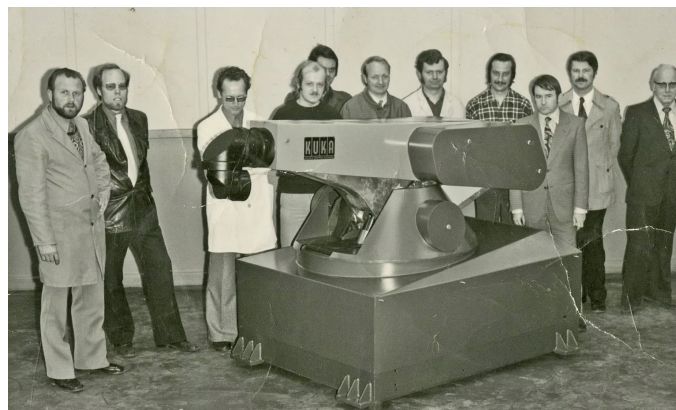


Fig. 2.6. Modelo KR FAMULUS, primer robot industrial con motor [10]

Los avances en la háptica comenzaron en los años 80 con el M2, también de Goerz, un sistema telerobótico usado como prototipo de demostración para diferentes aplicaciones como la militar, espacial o nuclear; y el primero en su tipo en proporcionar retroalimentación de la fuerza en el extremo remoto [3]. En esta década se comenzaron a investigar métodos más sofisticados para asegurar una mayor estabilidad dentro de los sistemas de teleoperación, además de tratar de mejorar su capacidad de transparencia. Especialmente se trabajó sobre aplicaciones que van desde la operación de vehículos submarinos y la robótica espacial, hasta la telecirugía [5].

Finalmente desde entonces hasta la fecha el concepto de la teleoperación no ha variado demasiado, aunque su evolución ha acompañado al resto de tecnologías a un ritmo sorprendente, y sigue operando en los mismos campos solo que cada vez a un nivel más alto, haciendo que el desempeño de más funciones dependan de ello. Las principales cuestiones de estudio se han vuelto las comunicaciones y la transparencia de los sistemas, en lugar de los alcances mecánicos; tratando de superar las nuevas barreras que se interponen en nuestro camino.

2.3. Aplicaciones

El factor de que la teleoperación incluya a un operador humano la hace una práctica potencial para manejar y adentrar se en entornos desconocidos o no estructurados. Como ha sido mencionado anteriormente, resulta de gran utilidad al ser incluida en aplicaciones como la robótica espacial, el tratamiento de entornos peligrosos, rescate, sistemas médicos y rehabilitación. Aquí se entrará en mayor detalle sobre los campos de aplicación más significativos.

Aplicación Industrial

La industria es uno de los muchos campos de desarrollo que se beneficia de la introducción de la teleoperación, pero es comúnmente confundida con la robótica y pese a que puedan estar relacionadas en ciertas ocasiones, es necesario saber diferenciarlas.

Los robots industriales por lo general se identifican por realizar unas series de tareas de forma repetitiva, con un entorno, precisión y velocidad predefinidos, completando así una asignación ofrecida por un supervisor humano. Son programados de este modo para llegar a las cada vez más altas demandas de forma eficiente. Sin embargo, esta definición dista de lo que supone la teleoperación, donde la repetición no está presente, ni la asignación de la misma exacta tarea.

En la telerobótica, incluso si un proceso se encuentra automatizado, este se desarrolla a partir de nuevas combinaciones de comandos o variantes de posibles existentes, objetos en diferentes localizaciones, y cualquier otra clase de evento que lo hace necesitado de la interacción humana, de manera más o menos directa, para llevarlo a cabo [1].



Fig. 2.7. Línea de montaje robotizada en la industria de la automoción [4]

Además, en los últimos años se ha podido observar la aparición de robots colaborativos en las líneas de producción desempeñando una serie de tareas mano a mano con los trabajadores. Los robots son controlados en primera instancia por un supervisor, pero también reciben órdenes de sus “compañeros” utilizándolos como herramientas de control directo siempre que sea necesario para alguna labor concreta o para asegurar un entorno laboral seguro para los trabajadores creando una relación trabajador-robot en el espacio de la planta [11].

Pero uno de los campos más comunes relacionado con la industria donde se emplea la teleoperación es en las tareas de mantenimiento, inspección y revisión en localizaciones que supongan un peligro, o sean de difícil acceso y manejar desechos peligrosos; siendo el principal ejemplo de estos las centrales energéticas. Como se ha mencionado anteriormente, la necesidad de control del entorno nocivo de una planta nuclear sin poner en riesgo a los operarios al exponerlos frente al material radioactivo de fue la precursora de la teleoperación tal y como la conocemos. Hoy en día, en la industria nuclear, diferentes formatos de teleoperación se siguen empleando para múltiples tareas, incluyendo el mantenimiento del reactor, la desmantelación de la planta e intervenciones de emergencia [1], [3], [4].

Otra clase de aplicación importante reside en el mantenimiento de las redes eléctricas donde las tareas a menudo implican grandes alturas, clima adverso y riesgo de electrocución. Antes un trabajador humano o un equipo necesitaban desplazarse físicamente hasta la zona afectada, exponiéndose a un enorme riesgo para poder desempeñar su labor; pero ahora es posible emplear sistemas robóticos teleoperados que se encargan de las reparaciones, la sustitución de aislantes cerámicos, la subida y bajada de puentes eléctricos, las revisiones, etc. Un ejemplo de ello es el ROBTET, diseñado para la manipulación de líneas de alta tensión y reemplazo de aislantes [1], [2], [4].

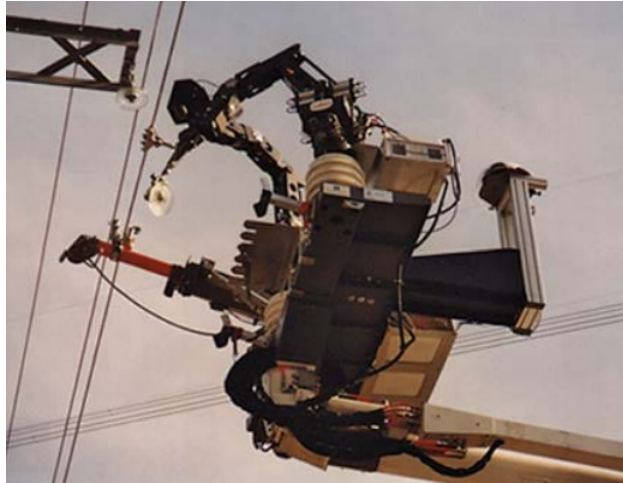


Fig. 2.8. Robot ROBTET para el mantenimiento de líneas de alta tensión [4]

Aplicación Submarina

La exploración submarina es una rama que ha resultado esencial para la industria del petróleo y gas en alta mar, además de para la investigación sobre la biodiversidad subacuática. Las herramientas que emplean esta especialidad son los ROVs (Remotely Operated Vehicles), nombre que se le asigna a los vehículos submarinos teleoperados [1].

En la mayoría de casos los ROVs también cuentan con característicos manipuladores hidráulicos, diseñados para soportar altas fuerzas y las presiones consecuentes por la profundidad a la que operan. Les permiten ser de gran utilidad para la inspección, mantenimiento y construcción de instalaciones submarinas [4]. Aunque en consecuencia son relativamente torpes. Pero su fiabilidad y capacidad de desempeño en estos entornos los han situado como mejor alternativa a la mano de obra humana para tareas con estos atributos [1].

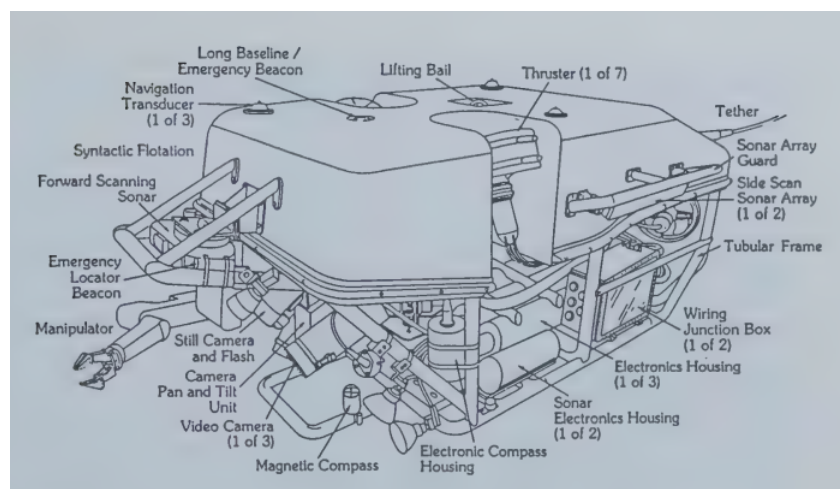


Fig. 2.9. Diagrama de dispositivo submarino teleoperado: Jason [1]

Los sistemas de comunicaciones de un ROV pueden variar enormemente dependiendo de la tarea para la que haya sido asignado. Por lo general pueden tratarse transmisión alámbrica o por sonar, pero cada una de ellas cuenta con sus desventajas. Las comunicaciones inalámbricas dificultan en gran medida a la cinemática del ROV perjudicando mucho su ya limitado movimiento; por otro lado el uso del envío de datos por acústica dispone de un ancho de banda muy limitado provocando retrasos importantes. Por ello es necesario estudiar qué método resulta menos perjudicial para cada situación [1], [4].

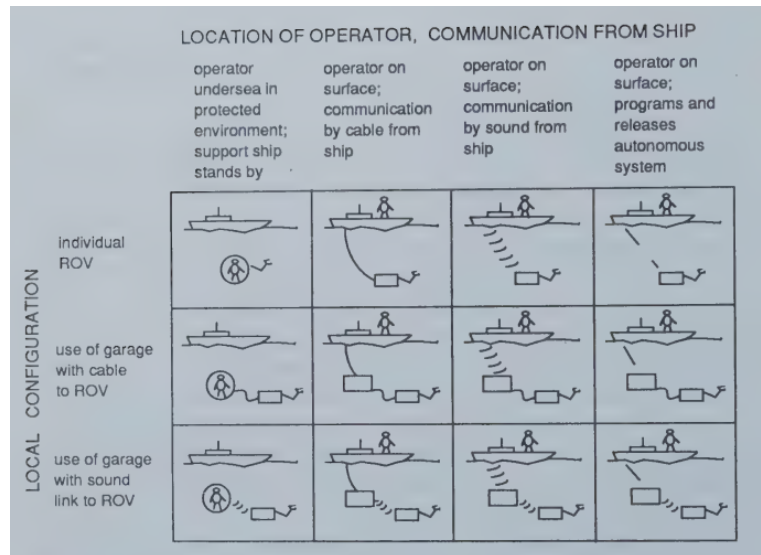


Fig. 2.10. Matriz de modelos de comunicaciones ROVs [1]

Otro modo de empleo de la teleoperación en el entorno subacuático son la monitorización de buzos humanos, la minería submarina, la búsqueda y estudio de biodiversidad, y la recuperación de naufragios o hundimientos. Por ejemplo, en 1966, el vehículo CURV de la Marina de los EE.UU. se utilizó con éxito para recuperar del fondo del océano una bomba nuclear que había caído accidentalmente desde un avión frente a las costas de Palomares, España [1], [12].



(a)



(b)

Fig. 2.11. Izquierda(a): Vehículo remotamente operado CURV III; Derecha (b): Personal de la Marina posando con la bomba-h recuperada por el CURV I [12]

Aplicación Espacial

En el auge de la exploración espacial, la manipulación remota del entorno se ha revelado como una de las técnicas más convenientes y prometedoras de este sector, puesto que la implicación de personas en estas campañas resulta enormemente ineficaz por factores como el tiempo, el coste y la seguridad. Introduciendo modelos como los Rover, manipuladores en forma de brazo robótico, incluso satélites.

En el espacio las misiones de exploración e investigación se desenvuelven en un periodo de tiempo demasiado extenso como para que se pueda sustentar por agentes humanos, por lo que el uso de dispositivos no tripulados resulta ser la opción más fiable a largo plazo, además de requerir de menos costos económicos. Además la teleoperación ayuda a salvaguardar la seguridad de los cosmonautas haciendo que no se expongan a tareas en entornos extremadamente hostiles.

Pero no todo lo que es capaz de ofrecer este enfoque es mejor. Las barreras de la comunicación en los sistemas a distancias tan lejanas son el impedimento más importante que sufre la implementación en este área, siendo posiblemente uno de los obstáculos más lentos y problemáticos de resolver dentro de la ciencia espacial.

Tenemos ejemplos de sistemas maestro-esclavo en el espacio, como es el RMS (Remote Manipulator System) desarrollado por SPAR uno de los primeros incursores en la robótica espacial. Se trata de un brazo robótico instalado en las lanzaderas espaciales, con un control directo y seis GDL. El operador lo controla por medio de dos joysticks de triple-eje, para permitir su movimiento en los ejes lineales y los ejes de rotación respectivamente, mientras que observa la manipulación del equipo desde el interior de la lanzadera a través de una ventana. El conjunto es algo rudimentario, no cuenta con sistema de televisualización ni de automatización de ningún tipo y supone un movimiento extremadamente lento; pero este es a su vez su punto fuerte, al ser considerado un actuador significativamente ligero, los motores de movimiento lento le permiten mover hasta cientos de kilos con facilidad en la microgravedad orbital [1].



Fig. 2.12. Canadarm 2, sucesor del primer sistema maestro-esclavo de la firma SPAR [4]

Por el otro lado, el primer vehículo tipo “rover” teleoperado en la luna fue Lunokhod_1, desarrollado por los rusos a principios de los 70. Este realizó un trayecto de 10 kilómetros en un intervalo de 11 días, el problema fueron los grandes retardos en las telecomunicaciones, y no se debía principalmente a las largas distancias, si no a que, por la inestabilidad de las señales, se tuvo que emplear el método de “mover y esperar” [4], [6]. Más adelante el Sojourner, diseñado por la NASA, corregiría y mejoraría muchos de los fallos de su antecesor adaptándolo a la exploración sobre la superficie marciana, pero el inconveniente de las comunicaciones persistirá, siendo más agravado por la separación interplanetaria [9].

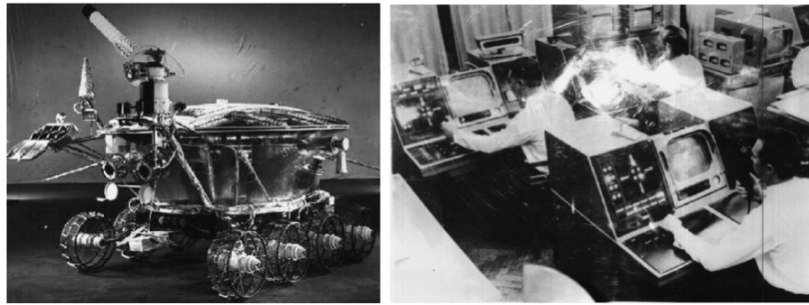


Fig. 2.13. Izquierda: Lunokhod_1 operado en la luna 11 meses, Derecha: Estación de control del Lunokhod [13]

Aplicación Militar

En el área de la militarización se puede ver cada vez más el uso de vehículos y herramientas teleoperadas en entornos tanto bélicos como de reconocimiento.

Los modelos más comúnmente utilizados en este contexto son los vehículos no tripulados para operaciones de reconocimiento y manipulación en zonas de conflicto sin tener que exponer a los activos al peligro. Un ejemplo perfecto de esto son los robots para el manejo y deshabilitación de dispositivos peligrosos, como el tEODor, que son usados por los militares y los cuerpos de policía para desarmar diversas clases de explosivos. En su mayoría están provistos de manipuladores y cámaras de video-transmisión que cuentan con una gran precisión a causa de la delicada naturaleza su objetivo [9].

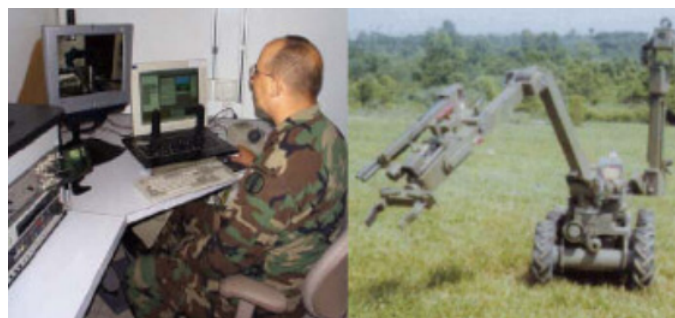


Fig. 2.14. Izquierda: Puesto de teleoperación militar, Derecha: Robot (esclavo) adaptado de la milicia Estadounidense [9]

Otra sección de agentes no tripulados están enfocadas al reconocimiento y desplazamiento tanto de las vías terrestres como aéreas; estos son los UAV (Unmanned Air Vehicle) y los UGV (Unmanned Ground Vehicle). Son secciones, en este caso, carentes de armamento que muestran su mayor potencial en sistemas de detección de movimiento y la visualización de objetos. Teniendo como objetivo principal el estudio y vigilancia del entorno a tiempo real, sus funciones se ven respaldadas por la tecnología de seguimiento de GPS, que les permite operar con un grado de autonomía en base a su posicionamiento y un operador en base que le encomiende los parámetros de la misión [1], [6], [9].



(a)



(b)

Fig. 2.15. Arriba(a): UAV Air Force Predator; Abajo(b): Vehículo tipo UGV: SARGE [6]

Aplicación de Salvamento

Es común considerar que aquellas situaciones en las que una persona se ve atrapada o en peligro, será igual de arriesgado que, otra persona, aun con el equipo necesario, acuda a su rescate. En estas situaciones los factores de riesgo son variados, desconocimiento del entorno de la catástrofe, inaccesibilidad a la zona afectada y complicaciones para mover escombros, entre otras. Visto así, los robots diseñados especialmente para las tareas de salvamento suponen una solución pragmática a la vez que efectiva para minimizar los daños en las operaciones de rescate y mejorar las tasas de éxito [14].

Una de las principales ventajas de utilizar estos robots, es la facilidad con la que son capaces de acceder a lugares que, de otra forma, nosotros no podríamos, y así recopilar información relevante para estudiar de forma más segura como proceder con la operación. Estas intervenciones a su vez apartan la manipulación humana directa en las zonas afectadas, minimizando aún más la adición de víctimas al desastre [14].



Fig. 2.16. Robot de salvamento para descombrar perteneciente al cuerpo de bomberos del ayuntamiento de Madrid [14]

Para los diferentes escenarios catastróficos se consideran diferentes tipos de robots según se adecuen a las necesidades de la crisis, pero por lo general los modelos usados están adaptados para atravesar terrenos escarpados y cuentan con un sistema tele-visualización que informa al operador y al resto del equipo de rescate la situación en el interior de la zona.

Estos modelos resultan especialmente útiles en situaciones terrestres o subterráneas como en el derrumbamiento de la mina Crandall Canyon en 2007 que introdujeron un pequeño modelo por uno de los conductos de oxígeno para conocer la situación de los mineros . En otras situaciones como la tragedia del huracán Katrina en 2005, se decantó por utilizar UAVs, pues era la forma más eficientes de buscar supervivientes o personas en situación de socorro en medio de todo el desastre. Y en la fuga de radiación de Fukushima en 2011 se utilizaron los modelos PackBot y Warrior, robots que además de contar con la estructura básica, añadían en sus diseños un actuador manipulador para mover obstáculos y navegar por el interior de la central [14].



Fig. 2.17. PackBot (Izquierda) y Warrior (derecha) [14]

Además un punto fuerte de esta clase de aplicaciones es que cada intervención es vital para la evolución de esta aplicación, pues recopila un gran número de datos que resultan extremadamente útiles ante futuras crisis.

Aplicación Médica

La teleoperación ha encontrado un enorme potencial en las ramas médicas, hasta tal punto que podemos observar su aplicación en estos entornos día a día y como numerosas investigaciones se centran en el uso de estas tecnologías. Particularmente, en este campo podemos destacar el telediagnóstico, la telecirugía y la asistencia para discapacitados.

El telediagnóstico, siendo uno de las aplicaciones con menor implicación técnica, es el término se le atribuye a la presencia remota de un especialista que analice y diagnostique al paciente con el uso de comunicaciones audiovisuales y la única intervención física del cuerpo de enfermería y auxiliares.

Por lo contrario, en la telecirugía, la implementación de sistemas teleoperados es sumamente compleja y debe de llevarse a cabo de forma precisa, al ser el propio cuerpo humano en entorno remoto en el que se desarrolla. Durante las cirugías, en la mayoría de casos es necesario conseguir una observación detallada y orgánica del interior del cuerpo del paciente en zonas donde, con solo las capacidades humanas, resulta demasiado complicado tratar.

El exponente más básico es el endoscopio; artilugio compuesto por un tubo móvil capaz de maniobrar por el interior de las estructuras del cuerpo que, dependiendo de la necesidad, puede desempeñar un gran número de labores. Existen endoscopios de observación, que cuentan con una cámara conectada por fibra óptica para facilitar la visualización en zonas comprometidas durante la intervención; en ocasiones van acompañados de otros que presentan pequeños actuadores en forma de gancho accionados por cables para ayudar con labores de manipulación; y aquellos que su función es retirar también entran dentro de esta categoría [1].

En las últimas décadas se han invertido muchos recursos en el diseño y fabricación de robots de asistencia quirúrgica, múltiples modelos diseñados y empleados en múltiples áreas, desde biopsias hasta la cirugía ocular. Todo esto ha sido gracias a los estudios en teleoperación en paralelo con otras ciencias. Su presencia en la telecirugía ha evolucionado hasta uno de los máximos exponentes actuales en este campo, la cirugía mínimamente invasiva, donde a día de hoy, sistemas como el daVinci permiten realizar procedimientos quirúrgicos complejos de forma remota con la característica de minimizar los riesgos que puedan dejar secuelas al paciente y reducir considerablemente el tiempo de rehabilitación [3].



Fig. 2.18. Uso del sistema tele-quirúrgico Da Vinci en una operación de Urología [15]

Por último, estas tecnologías también han resultado sumamente importantes para el desarrollo de tecnologías de apoyo para discapacidades. Una de estas facetas es su relación con exoesqueletos humanos. Controlados por un operador humano, paciente o enfermero rehabilitador. Toda toma de decisiones última son dejadas al usuario humano, mientras que el exoesqueleto se encarga de procesar esta información y ejecutar sus funcionalidades con la menor latencia posible. La diferencia principal es que tanto el sistema local como el remoto se encuentran en el mismo entorno, combinados físicamente, en este caso el usuario interactúa directamente con el robot como herramienta. Expande las capacidades físicas o de manipulación del usuario, reforzando el uso de extremidades atrofiadas o sustituyendo las que ya no están, incluso añadiendo funcionalidades a la fisiología humana natural. El uso más común de los exoesqueletos en el ámbito de la rehabilitación, donde los pacientes ya no tienen fuerzas para desempeñar ciertos movimientos por su cuenta debido a la edad o a la atrofia; consiguiendo grandes resultados [1], [3].

La otra cara de este tipo de aplicaciones son las prótesis biónicas. Elementos mecánicos integrados en el cuerpo que tratan reemplazar la presencia y/o el uso de un elemento motriz ausente en el paciente. La teleoperación se centra en aquellas con funcionalidad motriz que sustituyan cualquier parte del cuerpo, principalmente controlado por el propio paciente. Y pese que al principio eran más comunes las prótesis de acción mecánica u operadas mediante comandos predefinidos enviados a servoactuadores, que limitaban en gran medida la libertad de movimiento; hoy en día, y cada vez más, se trabaja en el uso y la transparencia de la interfaz humano-máquina por medio de implantes de electrodos que permiten al sujeto operar el dispositivo por medio de sus propios impulsos mioeléctricos

El objetivo de las prótesis es mejorar la calidad de vida de las personas, y aunque aquellas en necesidad de usar estas tecnologías necesitan de duras sesiones, físicas y mentales, de rehabilitación, los resultados indican que, tras adaptarse, pueden volver a ejercer una vida autónoma [16].

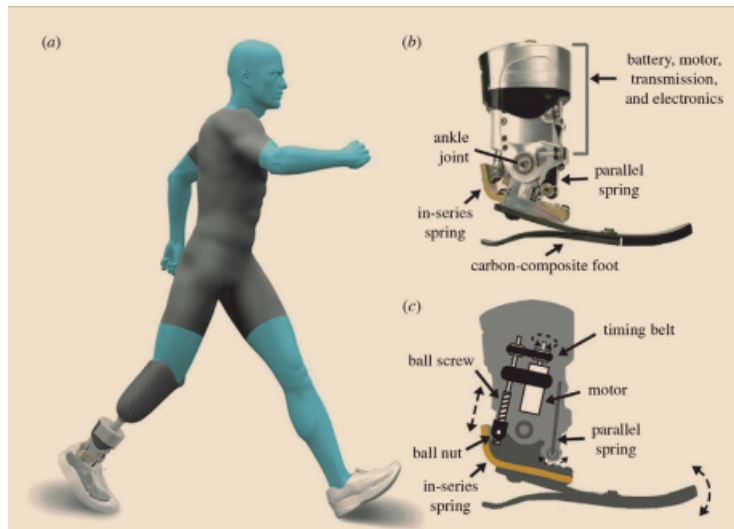


Fig. 2.19. Modelo prótesis biónica pie-tobillo [16]

Otras aplicaciones

Los sectores mencionados, pese a ser los principales beneficiarios de la teleoperación y donde su influencia es más evidente, no son los únicos ámbitos donde se usan este tipo de tecnologías. Muchas otras áreas como la construcción, estudio de fauna, la docencia y la minería, entre otras [1], [3]; se ven significativamente favorecidas por el uso de sistemas teleoperados. Ya sea por abaratar costes, realizar tareas más eficientemente, mantener al ser humano alejado de entornos peligrosos garantizando su seguridad o acercándolo al entorno remoto de manera artificial; las aplicaciones de la teleoperación pueden extrapolarse a una gran variedad de situaciones

El progreso de la teleoperación no solo va de la mano del resto de tecnologías, si no que también se expande a nuevos dominios y mejora su desempeño en aquellos en los que ya está integrada. Y su evolución se ve particularmente acentuada en los campos de la telerobótica y la industria, desvelando cada vez un paradigma diferente que amplía sus posibilidades de aplicación y desarrollo.

2.4. Arquitecturas de Control

Los sistemas de teleoperación requieren de una comunicación de control específica dependiendo de la tarea para la que son diseñados. Llamamos “arquitectura de control” a la combinación y grado de uso de la información proporcionada por el sistema y los comandos recibidos con los que hacen posible su funcionamiento. Dependiendo de su estilo y tipo de comunicación podemos organizarlos en un espectro del que definir cuatro tipos de arquitecturas principales:

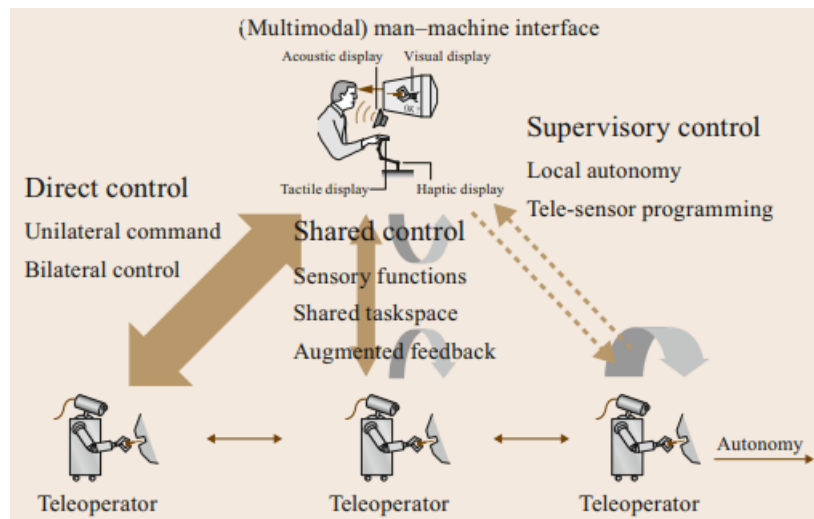


Fig. 2.20. Diferentes clases de arquitecturas de control telerobótico [3]

Control Directo

Como se ha mencionado anteriormente el control directo es uno de los márgenes que contienen las diversas arquitecturas de control en la teleoperación. Implica la ausencia de cualquier tipo de autonomía o retroalimentación por parte del sistema que realice más acciones que las comandadas expresamente por el operador a través de la interfaz del controlador o robot maestro [3].

Control Bilateral

El control bilateral ahonda en el uso de la háptica para el desempeño de funciones. Se utiliza una retroalimentación a tiempo real que proporciona al operador un conocimiento sensorial del entorno remoto. Tanto el operador como el propio sistema se comunican entre ellos y responden a los factores del entorno para realizar correctamente su cometido. Este modo de operación resulta de suma utilidad en situaciones donde la precisión es una característica crucial [3], [5].

Control Compartido

La arquitectura de control compartido tiene similitudes con las características presentes en el control directo sumadas a cualidades propias de un control autónomo. El ejemplo más claro es el control directo de un robot esclavo, pero debido a que la tarea que debe desempeñar necesita seguir comandos adicionales no impuestos por el supervisor, corregir los patrones de movimiento regular las articulaciones por factores externos al sistema de comunicaciones [3].

Control Supervisado

Por otro lado, el control supervisado compone el otro margen del espectro de control. Este estatuto implementa la realización de tareas en bucle cerrado al utilizar comandos a alto nivel y no involucra el envío de órdenes de movimiento o acción directas de ningún tipo para comandar cada acción individual del sistema. En esta relación el operador asigna una tarea al sistema; el elemento esclavo tiene la capacidad de tomar decisiones, adaptarse en base a los datos del entorno y llevarla a cabo por medio de sus propias habilidades, devolviendo al usuario información sobre el curso de los resultados. Sin embargo la monitorización del proceso por parte del operador sigue siendo crucial para que todos los procesos se desempeñen correctamente. Es común utilizar estas arquitecturas en situaciones donde la aplicación de órdenes directas no es viable a causa de los posibles retardos o son demasiado complejas y prolongadas que requieren de un gran nivel de autonomía [1], [3], [4], [11].

2.5. HRI

Los sistemas robóticos necesitan interpretar, tomar decisiones y responder a su entorno, por lo general el físico; todo esto al mismo tiempo que cooperan con los diferentes factores que representen otros agentes con los que realice la labor asignada, incluidos los humanos. Dentro de la telerobótica, uno de los conceptos de estudio e investigación más importantes, tal vez más que el entorno, las aplicaciones en las que se pueda desarrollar o su clase de control, es la HRI (Human-Robot Interaction). La composición y características del puente interactivo entre humano y robot.

La interacción humano-robot se puede dividir en dos subtipos de percepción reconocibles, no como rangos separados, sino como características que pueden estar presentes dentro de un mismo sistema en mayor o menor proporción e identifican diferentes enfoques del mismo: HRI físico (pHRI) y HRI cognitivo.

- La variante física de este concepto se centra en la colaboración con el usuario y escenarios que impliquen contacto físico. Por lo general se observa en el control supervisado, siendo el estado de interacción presente en mayor proporción en campos como la teleoperación industrial, la colaborativa o la de rehabilitación. Los elementos robóticos deben de seguir unas pautas, estas pueden estar supervisadas, pero por lo general son automatizadas, y esto supone un riesgo para la presencia humana dentro del entorno de operación si no son gestionados correctamente. Los robots destinados a esta clase de contextos son diseñados con una serie de protocolos que les permiten desarrollar respuestas a tiempo real basadas en los estímulos del entorno externo, principalmente provenientes de los humanos. Con esta información, planean rutas que respeten las preferencias humanas, generan planes de colaboración y eventos de co-acción basándose en su habilidad para sentir las interacciones físicas y planificar tareas [3].

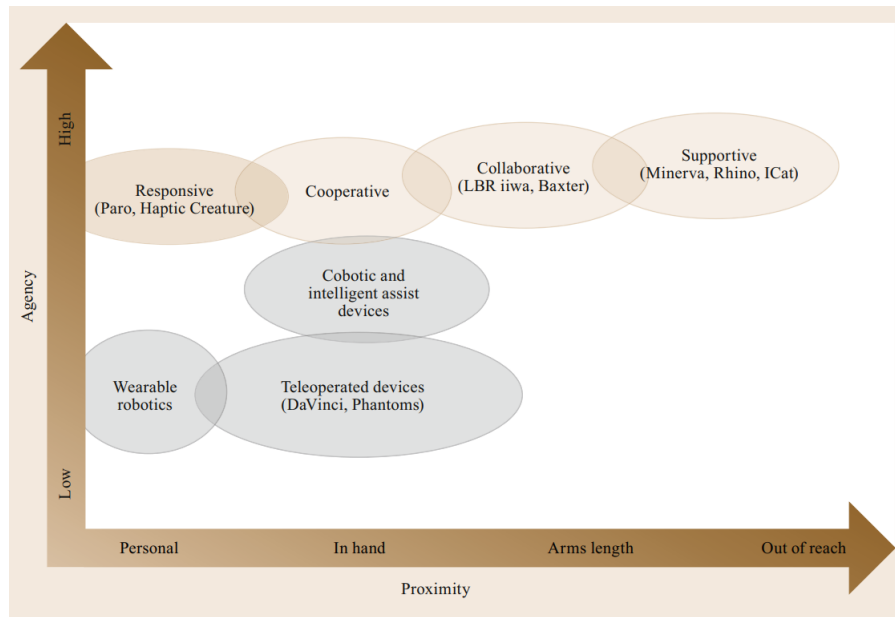


Fig. 2.21. Esquema de clasificación de pHRI [3].

- La variante cognitiva está más estrechamente relacionada con la interpretación que ambos extremos del sistema elaboran respecto a su entorno. Trata de mejorar esta interpretación con modelos capaces de “comprender” y “razonar” acorde al contexto en el que se desarrollan, de este modo consiguen adaptar al usuario para volverlo un elemento efector del sistema, con el objetivo de integrarlo al mismo tiempo que se mejora la transparencia. Para poder alcanzar este grado de percepción, los robots requieren de sensores y modelos virtuales que representan de manera fidedigna las características físicas y cognitivas de su entorno. Se encuentra presente en una mayor proporción en escenarios como robots sociales o de servicios, donde la toma de decisiones es crucial. Pero los paradigmas de interacción cognitiva apuntan a representar características detalladas tan concretas como la relación entre control y acción que representan las interfaces en la teleoperación, o tan amplias como la colaboración humano-robot en la colaboración entre pares [3].

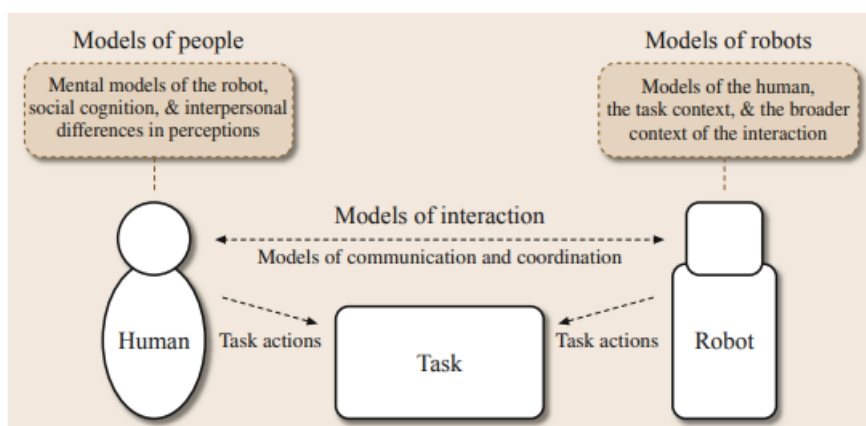


Fig. 2.22. Paradigmas de HRI cognitiva [3].

La interfaz es un elemento más del abanico que supone la HIR y; sin embargo; compone una de las características más importantes dentro de los contextos de la teleoperación y la robótica. Como elemento de interacción humano-máquina, es el medio por el cual el usuario se relaciona con el sistema remoto por medio de múltiples canales, facilitando su control y, en ocasiones, mejorando su percepción del entorno remoto. Dependiendo del estilo de interacción las interfaces son clasificables en tres categorías diferentes: directas, multimodal y de control coordinado; dependiendo de la naturaleza del intercambio de datos entre efector y actuador [13].

Por el lado de las interfaces directas, estas son las más habituales, donde el usuario emplea dispositivos de control como joysticks para operar el sistema que, por lo general, replica los movimientos efectuados generando un significativo grado de telepresencia, sobre todo en situaciones de baja latencia, dándole la oportunidad al operador a responder en tiempo real. Las interfaces multimodales, independientemente del tipo de control respecto al que estén contruidos, se caracterizan por ofrecer al operador múltiples representaciones del entorno remoto para una percepción completa y más precisa del mismo, gracias diversos sensores y actuadores con retroalimentación visual que integran la visualización de estos datos. Por último las interfaces de control coordinado son el máximo exponente del desarrollo háptico, integrando la capacidad sensorial del usuario en el sistema de control, invitando a la toma de decisiones en base a la respuesta recibida por parte del sistema [13].

2.6. Controladores Modulares

El elemento primordial que sirve de puerta al vínculo de interacción humano-máquina son los controladores. Mediante ellos el operador es capaz de tomar parte de este sistema, comandando las acciones del robot. Existen diferentes clases de controladores a tener en cuenta:

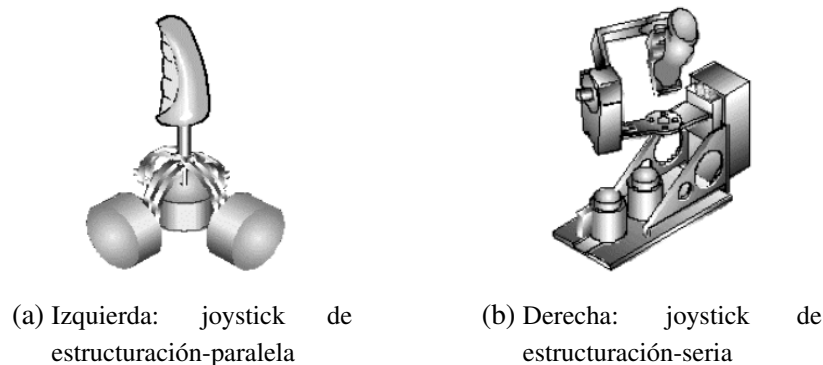


Fig. 2.23. Morfología de los joysticks [17]

- Joysticks y palancas de control: También llamados FRMC (force-reflecting manual controller), son uno de los dispositivos de control directo más utilizados a día de

hoy. Su funcionalidad destaca sobre la de otros tipos de controladores gracias a ser extremadamente intuitivos, permitiendo al usuario averiguar con facilidad su funcionalidad y pueden contar con múltiples GDL dependiendo del diseño. Dentro de ellos podemos diferenciar dos tipos de FRMC: aquellos con un diseño de estructuración-serial y otros con un diseño de estructuración-paralela, siendo su única diferencia el método de disposición de sus sensores y servomotores [17].

- Mandos a distancia/teclados: Estos dispositivos, a pesar de gozar de una numerosa cantidad de formatos, su funcionalidad siempre se ve reducida al mismo principio, la señal compuesta por el uso de sus pulsadores o la combinación de ellos es lo que envía la orden. Son empleados en aplicaciones donde esos comandos básicos bastan para controlar al elemento esclavo [1].

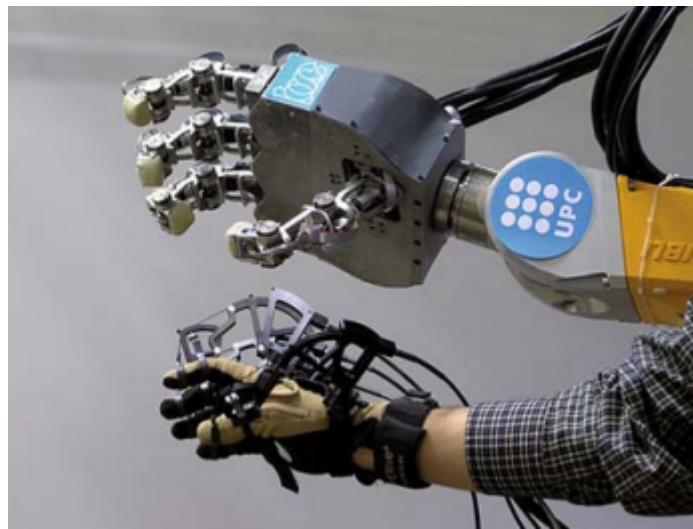


Fig. 2.24. Brazo robótico teleoperado por medio de un controlador tipo exoesqueleto (UPC) [4].

- Exoesqueletos: Bien como pueden ser del mismo modo elementos teleoperados de un sistema, en ocasiones empleamos este tipo de estructuras para controlar el sistema remoto, sirviendo como representante de nuestros movimientos anatómicos en el entorno local. El operador lleva el exoesqueleto en una zona del cuerpo concreta cuyo movimiento quieran replicar para la operación transfiriendo de forma precisa esta información. Son dispositivos que cuentan con una gran robustez, adquiriendo muy poco ruido durante la adquisición de señales, pese a que resultan aparatosos y en ocasiones limitan los movimientos del operador [4].
- Guantes sensorizados: Se trata de guantes equipados con sensores capaces de identificar la posición articular de los dedos y la palma. Por lo general este efecto es logrado a través de galgas instaladas en el guante volviéndolo un control muy natural; sin embargo es su misma estructura y composición lo que los vuelven una alternativa delicada, tanto estructuralmente, como en la naturaleza de sus señales [4].

- **Lectores de EMG:** Relacionados con los anteriores, se encuentran los lectores de señales electromiográficas. Su modelo puede variar dependiendo de la zona del cuerpo para la que hayan sido diseñados, pero por lo general comparten la misma base de concepto. Se trata de una serie de electrodos conectados entre sí a un elemento procesador que extraen las señales generadas por la actividad muscular de la zona. Resulta uno de los componentes clave para el control de prótesis, al poder establecer una interfaz y canal de comunicación directa entre el cuerpo del paciente y el dispositivo [18].

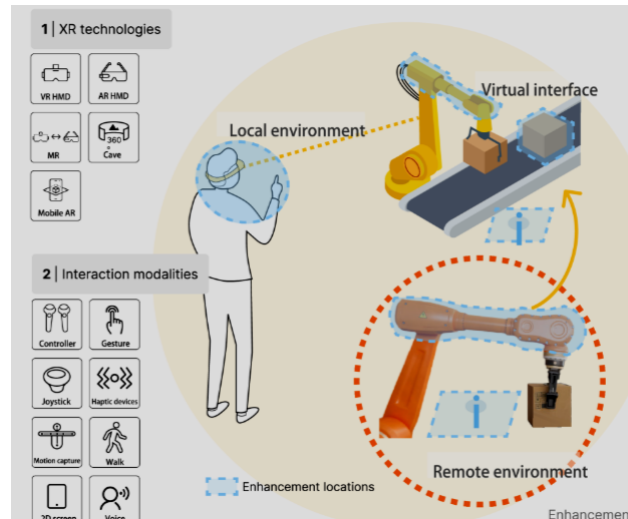


Fig. 2.25. Representación de un sistema teleoperado por VX [19].

- **Controladores XR:** En esta categoría se incluyen todos aquellos dispositivos que expanden o sustituyen la percepción del entorno real por medios artificiales generando una simulación de interacción para el ser humano, tales como Realidad Virtual (VR), Realidad Aumentada (AR) y Realidad Mixta (MR). Los controladores que toman parte en llevar a cabo estos métodos de inmersión cognitiva están enfocados exclusivamente en la transparencia del sistema mediante el uso de conjuntos como gafas especializadas, mandos, sensores de seguimiento y guantes entre otros. Permitiendo que la colaboración conjunta de estos elementos envuelva al usuario en un entorno digital con el que interactuar [19].
- **Sistemas de visión:** Otra de las opciones que integra a la fisonomía humana en el puesto efector del sistema. En este caso el operador no necesita interactuar con ningún elemento físico para desempeñar la acción de control, sino que emplea visión por computador para reconocer las poses, o gestos que haga. Pero resulta arriesgado utilizar controladores basados en este concepto, pues si el programa de reconocimiento no está bien implementado se pueden perder muchos datos en el proceso [4].
- **Sistemas por voz:** Relacionado con el método anterior, los sistemas por voz emplean el reconocimiento por computador para extraer patrones sonoros e interpre-

tarlos, reconociendo así las palabras pronunciadas por el usuario, de este modo se transforman en comandos útiles para operar el sistema. Cuentan con la ventaja de que pueden ser utilizados sin la necesidad de realizar ninguna acción motriz en caso de no ser posible este tipo de acciones [1], [20]

- Controladores automáticos o autonomía parcial: Son los responsables de que la aplicación del control supervisado sea posible, formando a su vez parte de la estructura de los elementos robóticos y componiendo un bucle cerrado de control en las que el operador puede revisar o introducir nuevas órdenes al proceso de operación. Se reconocen subtipos como los controladores de autonomía parcial que automatizan acciones básicas reduciendo la carga de trabajo del operador o los de control predictivo. Todos ellos son controladores basados en software que utilizan GUIs (Graphical User Interface) para permitir a los operadores supervisar y ajustar las tareas llevadas a cabo por el elemento teleoperado sin necesidad de una intervención física directa ya sea para todo el proceso de la función o para eventos puntuales [4].

Muchos de estos dispositivos no solo tienen la función exclusiva de enviar señales para controlar el dispositivo actuador, sino que también, dependiendo de la arquitectura de control empleada, pueden devolver estímulos representantes del entorno remoto, por medio de zumbadores, electrodos, señales visuales, retroalimentación de fuerzas, etc. Todos aquellos que tengan esta característica reciben la clasificación de aparatos “hápticos” [4], [5].

Los controladores pueden ser utilizados individualmente, o configurados bajo cualquier combinación para crear una entrada a la interfaz adecuada a los requisitos que se buscan cumplir. De todos modos las aplicaciones en las que se puede llevar a cabo la implementación de un sistema teleoperado son numerosas, y del mismo modo la forma de abordar los retos y barreras que ofrecen solo están limitados por la inventiva humana. Por ello se debe de considerar que la cantidad de diseños posibles para modelos de controlador es inabarcable. Cada tarea específica llevada a cabo dentro de diversos entornos necesitarán de enfoques distintos, lo que llevará a la personalización de los sistemas y dispositivos de control para lograr la integración de usuario de la manera deseada.

La modularidad en los controles de un sistema teleoperado ofrece al usuario una flexibilidad y adaptabilidad fundamentales cuando se trata con entornos de operación complejos y que requieren de múltiples planes de acción, pudiendo cambiar entre dispositivos dependiendo de las necesidades de la tarea. Tal vez se necesite ser más preciso, tener un mayor rango de movimiento o adquirir señales específicas del entorno local entre otros requisitos; y no se disponga de controladores que cumplan con todas las características al mismo tiempo. Por lo que la opción más eficiente es cambiar de controlador a uno más apropiado para la situación [21].

Además, este tipo de estrategia de interfaz permite que el sistema sea accesible para una gran variedad de usuarios, ya sea por preferencias personales para desempeñar mejor sus funciones, o por limitaciones físicas que impiden usar eficientemente el controlador

inicial. Por ejemplo para una misma tarea en la que se pueden emplear diferentes tipos de controladores, el seleccionar uno con el que el usuario se encuentre más familiarizado, ya sea por su trasfondo o experiencia con dispositivos similares, puede resultar ser un factor determinante en que las tareas se lleven a cabo de forma eficiente. Y existen una enorme gama de controladores alternativos que cubren toda clase de discapacidades. Esto hace que la utilización de las mismas tecnologías puedan ser alcanzadas por todos por igual, sin importar la condición.

Al extender el concepto de modularidad al resto de elementos de la estructura teleoperativa se obtienen ventajas como abrir la posibilidad de integrar nuevas tecnologías al sistema conforme sea necesario sin requerir un rediseño completo de los otros elementos del régimen. Por ejemplo, si se introducen mejoras significativas en el software del sistema o surge una versión mejorada del componente esclavo, la modularidad permite reemplazar fácilmente los elementos antiguos por las nuevas versiones. Incluso resulta beneficioso a la hora de compartir funcionalidades individuales del sistema con otros proyectos, aprovechando la reusabilidad de estos elementos.

La modularidad aplicada a entornos teleoperados tienen la capacidad de mejorar la adaptabilidad y su flexibilidad frente a nuevos obstáculos. El hecho de abrir las puertas a futuras innovaciones al igual que a permitir una mayor accesibilidad para un mayor número de usuarios permite que estos entornos se vuelvan más eficaces y eficientes

[21]

3. METODOLOGÍA

Este apartado del proyecto describe los procedimientos y métodos empleados para el desarrollo práctico del proyecto de teleoperación llevado a cabo, además de entrar en detalle de los elementos, tanto de hardware como de software propuestos que componen la aplicación de teleoperación un brazo robótico. El objetivo al que se trata de llegar a través de estos pasos es el de conseguir teleoperar un brazo robótico industrial ABB mediante diferentes tipos de controladores y el uso de comunicaciones EGM que transfieran los datos adquiridos a tiempo real.

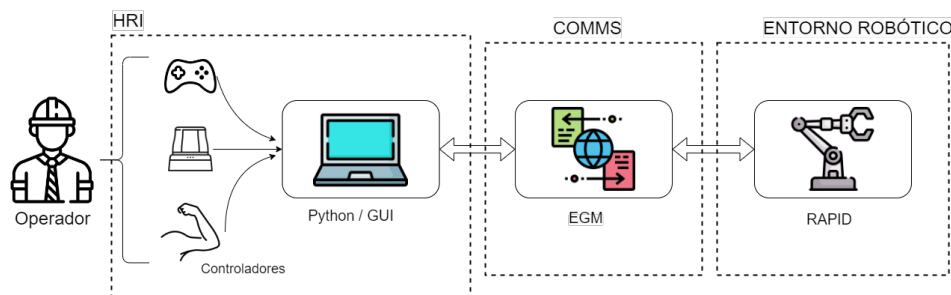


Fig. 3.1. Esquema del sistema multimodal de telecontrol via EGM

Para llegar al objetivo propuesto de teleoperación de un robot industrial se plantea una arquitectura de control directa respaldada por una interfaz multimodal. En esta se emplean aplicaciones de lectura para cada uno de los controladores seleccionados que extraen y clasifican las señales generadas al ejercer acción con sus elementos de interacción para posteriormente enviarlas a los siguientes programas de la jerarquía. Estos datos llegan a la aplicación de display que forma parte de la interfaz multimodal a su vez que a la aplicación de conexión EGM para poder inicializar las comunicaciones entre los scripts de Python y el entorno robótico virtual de RobotStudio que permitan a la configuración de la estación interpretar los datos de control como datos de movimiento logrando así el funcionamiento del sistema telerobótico.

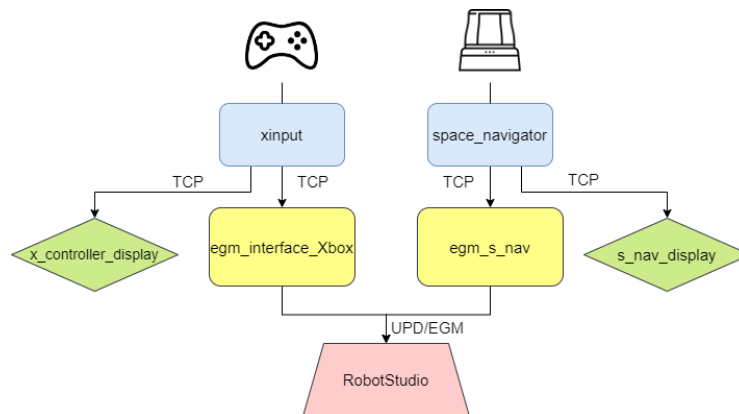


Fig. 3.2. Estructuración de comunicaciones

La adquisición de datos extraídos de los controladores es llevada a cabo a través de sus respectivas aplicaciones de lectura y procesamiento, diseñadas por medio del lenguaje de programación Python. En ellas se implementa el uso de un conjunto de librerías que ayudan a procesar los datos introducidos por sus controladores correspondientes, permitiendo ver en la consola en todo momento el estado del controlador y los diferentes eventos que toman lugar en ellos.

Sin embargo este conjunto de datos puede resultar insuficiente para una interfaz entendible e intuitiva, debido a que lo que se busca con este tipo de GUIs es una rápida comprensión visual, consiguiendo una interfaz multimodal, se aborda el uso de una aplicación de display que permita a cualquier operador reconocer que está ocurriendo con su controlador y como lo interpreta el programa. La aplicación de GUI establece una conexión de protocolo TCP (Transmission Control Protocol) con el programa de lectura de controlador al que representa adquiriendo la función de cliente, de esta forma se consigue que el flujo de envío de datos sea constante y se realice de forma ordenada, un factor crucial a la hora de controlar directamente dispositivos que requiera de una estructuración de datos clara y se puedan ver perjudicados por posibles confusiones de interpretación. Por ello mediante la librería de “pygame”, son diseñadas las interfaces de cada controlador gracias a que se trata de una librería especialmente enfocada a la implementación gráfica.



Fig. 3.3. Operación de un brazo robótico con asistencia de una GUI [22]

Del mismo modo se establece una relación servidor-cliente entre las aplicaciones de lectura y la interfaz EGM creando una conexión TCP que envíe los datos de los controladores de modo similar mientras las aplicaciones están en uso. Es mediante el uso de dos librerías críticas en el desempeño de este proyecto, disponibles en Python, que se consigue transformar y enviar a tiempo real los datos o métodos especificados en esta etapa del proyecto. Ambas, `abb_robot_client` y `abb_motion_program_exec` se encargan de proporcionar a la siguiente etapa las características correspondientes que se especifiquen en su código. A través de sus comandos se establecen los datos necesarios de corrección, los modos de operación deseados y los métodos de movimiento basados en las entradas de los

controladores, pudiendo operar el brazo tanto por medio de un planteamiento cartesiano, como de un planteamiento polar.

La última etapa del proyecto se desarrolla en el entorno virtual de RobotStudio, donde la simulación del brazo robótico y su operación es realizada. Para lograrlo es importante asegurar los correctos parámetros y que el controlador robótico cuente con los protocolos adecuados. Pese a que todos los controladores de ABB tengan acceso a la configuración EGM, para este tipo de implementación es empleado un controlador robótico IRC5 debido a la configuración de “Multitask” que ofrece, permitiendo al entorno virtual operar a varios niveles simultáneamente, asegurando que los sectores del programa puedan trabajar en paralelo para garantizar su correcto funcionamiento y eficiencia. El IRC5 también ha sido seleccionado por ser el único controlador robótico que dispone de compatibilidad con las librerías de comunicación entre Python y RobotStudio.

Para la comunicación entre los programas externos y el entorno virtual a través de EGM se habilita la escucha de un puerto por protocolo UDP que permite que se intercambien los datos de manera rápida y eficiente. Además el diseño del entorno virtual es realizado de manera que sea compatible con cualquier modelo de brazo robótico ABB siempre que cuente con 6 GDL y sea capaz de funcionar con el controlador robótico IRC5. Una vez configurado todo el entorno virtual se implementan algunas funciones de desarrollo. La aplicación de RobotStudio cuenta con tres módulos, dos de ellos se encargan de registrar archivos en la memoria dinámica datos de análisis como los errores y la posición del robot durante la operación, mientras que el tercero es el encargado de recibir los datos que llegan desde la interfaz EGM de Python, enviados en cartesianas o en articular, para después ejecutarlos a tiempo real; del mismo modo al finalizar la operación o al dispararse un error registrado, se envía el archivo con dicha información al programa de Python para ser mostrado, ofreciendo cierto grado de bidireccionalidad.

3.1. Herramientas Hardware

3.1.1. Mando xbox

Uno de los controladores propuestos para la teleoperación del brazo robótico debido a que se trata de un dispositivo de fácil acceso y con una curva de aprendizaje de uso prácticamente nula. El hecho de combinar palancas, gatillos y botones lo convierten en un elemento con multitud de configuraciones.

Muchos controladores de la misma familia disponen de una configuración de activadores muy similar, tanto a nivel de software como de hardware; además de poseer un relativo bajo coste y fácil acceso. Por lo que resultan un buen punto de partida al crear una aplicación de teleoperación que pueda funcionar en cualquier computador.

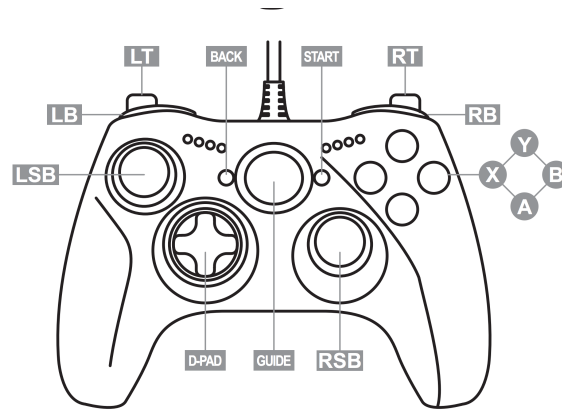


Fig. 3.4. Diagrama de controlador estilo xbox [23]

Los joysticks duales y gatillos con valores graduales son excelentes para aportar los GDL a la interfaz de control casando con las que dispone el robot colaborativo. Sumado a esto si fuese necesaria alguna otra funcionalidad, aplicar comandos fijos, u operar una herramienta que el robot tenga instalada los botones del controlador pueden ser configurados para esta tarea.

El modelo empleado durante el proyecto es el GC-100XF, pero todos los mandos de soporte oficial de xbox tienen características que ofrecer.

Elemento	Identificador	Tipo de dato
PAD_UP	button_1	bool
PAD_DOWN	button_2	bool
PAD_LEFT	button_3	bool
PAD_RIGHT	button_4	bool
START	button_5	bool
BACK	button_6	bool
LSB	button_7	bool
RSB	button_8	bool
LB	button_9	bool
RB	button_10	bool
MODE	button_11	command
D/X	button_12	command
A	button_13	bool
B	button_14	bool
X	button_15	bool
Y	button_16	bool
LT	left_trigger	analog 8-bit
RT	right_trigger	analog 8-bit
LJoystick_X	l_thumb_x	VRE int16
LJoystick_Y	l_thumb_y	VRE int16
RJoystick_X	r_thumb_x	VRE int16
RJoystick_Y	r_thumb_y	VRE int16

TABLA 3.1. ACTUADORES DEL MANDO

3.1.2. Spacemavigator

Un controlador de tipo ratón 3D desarrollado por Logitech. Es una herramienta que usualmente va de la mano con facilitar la navegación en programas de diseño. Pero su funcionamiento hizo que se plantease como controlador para este proyecto. Cuenta con un sensor, o conjunto de ellos, que ofrece datos suficientes para componer 6 GDL con los que complementar a los movimientos necesarios para operar al brazo robótico con precisión.



Fig. 3.5. Modelo Mouse 3D Space-navigator [24]

El dispositivo es capaz de moverse en los tres ejes espaciales “x”, “y”, “z” y a su vez también es capaz de recorrer los tres ejes de rotación principales “pitch”, “roll” y “yaw”; sumado a esto cuenta con 2 botones adicionales. El disponer de esta capacidad lo convierte en un controlador interesante para este tipo de aplicaciones.

Cuenta con un instalador propio que lo adecua para el uso en numerosos entornos y aplicaciones, pero en el caso de este proyecto se implementará una aplicación de lectura que permita leer el input de este dispositivo para obtener con precisión los datos necesarios por lo que esta característica no será necesaria [24].



Fig. 3.6. Visualización de los movimientos captados por el sensor del Spacemavigator [24]

3.1.3. MindRove armband

Dispositivo de adquisición de señales EMG (electromiográficas) creado por la firma MindRove Ltd. (Budapest, Hungary). Es generalmente empleado en el campo de la investigación biomédica y tiene un historial en aplicaciones de control de dispositivos mediante la actividad muscular.

Se trata de un sistema de electrodos compacto e inalámbrico de 8 canales. Mediante su conjunto de electrodos secos monopolares es capaz de registrar señales EMG superficiales pertenecientes a los músculos que se activan con el movimiento dactilar y los gestos realizados con las manos. La característica de disponer de electrodos secos implica que no es necesario ningún medio fluido entre los electrodos y el brazo para conseguir una impedancia correcta que asegure la correcta lectura de las señales EMG, optimizando el tiempo de preparación para su uso.

El brazalete de electrodos cuenta con soporte de librerías de código abierto SKD con el que se posibilita crear aplicaciones de interfaz gráfica para los usuarios . Su capacidad para leer gestos extrayendo las señales eléctricas de los músculos del antebrazo del sujeto, y contar con un acelerómetro y giroscopio que reconocen cuando y como mueve el brazo el sujeto; lo consiguieron hacer un candidato idóneo para expandir el alcance de este proyecto.



Fig. 3.7. Brazalete de electrodos Mindrove armband [25]

Las principales características con las que cuenta el brazalete son:

TABLA 3.2. TABLA DE CARACTERÍSTICAS ARMBAND

EMG		IMU	
Colocación recomendada	Antebrazo	Giroscopio y Acelerómetro	3-axis
Electrodos EMG	8+2 (bias, ref)channel	Rango del Giroscopio	+/-500 dps
Ratio de Muestreo	500 Hz	Rango del Acelerómetro	+/-2g
Resolución	24 bit	Ratio de Ruestreo de IMU	50

Fuente: [25]

3.2. Herramientas Software

3.2.1. RobotStudio

Este software de configuración de entornos robóticos desarrollado por ABB es una herramienta de programación, simulación y optimización para robots industriales, que permite operar en diferentes entornos y configuraciones que pueden ser definidos por el usuario.



Fig. 3.8. Logo RobotStudio

Cuenta con herramientas como la simulación 3D que permite una visualización mayor del entorno aplicable al robot y posibilita la recreación de un área fidedigna a las condiciones en las que se vaya a desenvolver gracias a su compatibilidad con modelos CAT. Permitiendo a los usuarios ver a tiempo real el comportamiento del robot respecto a la configuración atribuida [26].

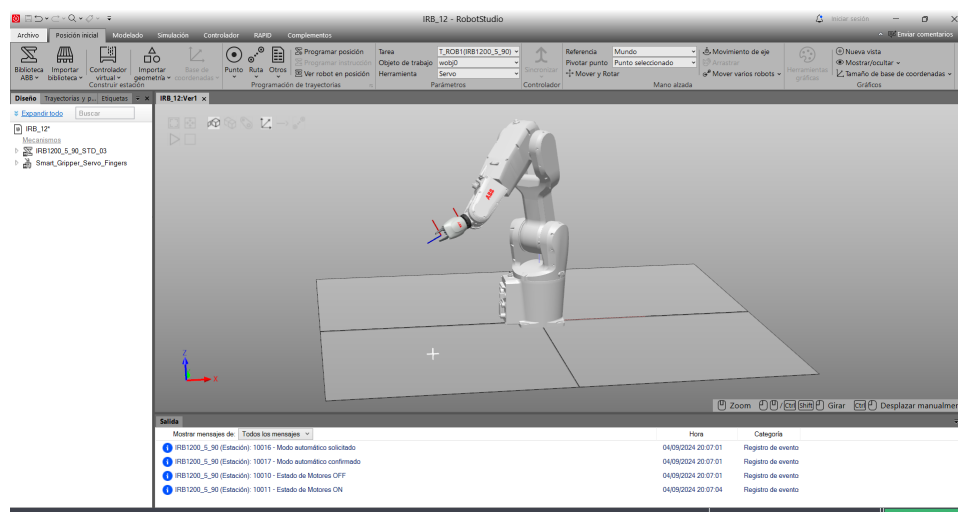


Fig. 3.9. Entorno virtual de RobotStudio

También cuenta con otras características como la de controladores virtuales, para emular la comunicación entre robot maestro y robot esclavo; la optimización de procesos, que ayuda a maximizar la eficiencia operativa del robot.

Que se trate de una aplicación de programación fuera de línea hace que los usuarios puedan programar y depurar el código en un entorno completamente virtual, esto elimina la necesidad de uso constante del hardware para comprobar que funcione correctamente y una vez generado el código y superado todas las pruebas, el programa se puede transferir directamente al robot físico [26].

EGM

Para la transmisión de datos que habilitan el telecontrol del brazo robótico se emplea un sistema de comunicación servidor-cliente basado en EGM. Es una característica de comunicación empleada por ABB (ABB Ltd., Zürich, Switzerland) en su software de gestión de estaciones robóticas RobotStudio. En base a ella se han programado las funcionalidades de envío de datos al brazo para la teleoperación, y la llegada de los datos de posición con los que generar una gráfica donde queda constancia de todos los movimientos realizados de cada articulación.

El EGM permite al elemento manipulador ser movido en base a los datos recibidos por un dispositivo externo, y su operación puede ser dividida en tres modos [27].

- EGM Position Stream: Proporciona la capacidad de enviar los datos de posición actuales y planeados de la unidad mecánica periódicamente. Solo se encuentra disponible para una comunicación UdpUc (User Datagram Protocol Unicast Communication). El canal de comunicación UDP puede ser ejecutado con alta prioridad en entorno de red del controlador robótico, lo cual asegura un intercambio estable de datos a 250Hz.
- EGM Position Guidance: Usado para leer y escribir sobre el sistema de movimiento a una alta tasa de muestreo. Las referencias para el movimiento pueden ser especificadas para emplear el movimiento de las articulaciones o para asignar una pose. Su alto muestreo y su baja latencia lo aventajan respecto a otros modos de movilización externa.
- EGM Path correction: Ofrece al usuario la posibilidad de corregir el camino de un robot programado mediante su sistema de coordenadas. Pero los datos obtenidos del trayecto deben de ser medidos por un sensor externo.

El EGM puede gestionar la transmisión de datos de diversas maneras; sin embargo la más relevante es su comunicación de datos vía UdpUc, la cual permite enviar los datos de retroalimentación del elemento mecánico a una frecuencia de 250 Hz, según aseguran los fabricantes. Y ya que no requiere de otros intermediarios para asegurar el intercambio de información, resulta la mejor opción a la hora de controlar al robot a través de una aplicación que se ejecuta externamente en un ordenador [27].

Sin embargo para el uso de las siguientes características de las comunicaciones EGM

se deben de tener en cuenta algunas limitaciones a tener en cuenta durante su aplicación, de ente todas, las más importantes son:

EGM Position Stream:

- RGM Position Stream solo está disponible con comunicación UdpUc.
- Tool data y Load data no pueden ser cambiados dinámicamente mientras haya un Position Stream activo.
- EGM Position Stream no es compatible con EGM Path Correction.
- NO está permitido activar o desactivar unidades mecánicas si EGM Position Stream está activo.

EGM Position Guidance

- Tiene que empezar y acabar en un fine point.
- El primer movimiento tras un reinicio del sistema controlador ABB no puede ser parte del EGM.
- Pose mode admite 6-axis robot, 4-axis Palletizer robots, YuMi robots y SCARA robots.
- Si el robot acaba cerca de una singularidad, i.e. Cuando dos ejes del robot se encuentran casi paralelos, el movimiento del robot será detenido junto con un mensaje de error. En esa situación la única forma de sacarlo de ahí es forzar al robot fuera de la singularidad.

EGM Path Correction

- Solo tiene soporte con 6-axis robots.
- El dispositivo externo debe de estar montado en el robot.
- Las correcciones sólo pueden ser aplicadas en el sistema de coordenadas del camino.
- Solo se pueden realizar correcciones de posición en “y” y “z”. No es posible realizar correcciones de orientación, ni correcciones en “x” (que es la dirección/tangente del camino).

Considerando las propias limitaciones de este paradigma de comunicación, el modo de empleo para conseguir la comunicación a tiempo real buscada es mediante la implementación de los modos EGM Position Stream y Position Guidance [27].

Robot Industrial IRB_1200_5_90

El programa está enfocado a su uso con cualquier brazo robótico industrial de 6 ejes o con 6 GDL que sean compatibles con el controlador IRC5. Sin embargo merece la pena entrar en detalle del modelo que se ha seleccionado para las pruebas de desarrollo durante la realización del proyecto.

El modelo IRB_1200_5_90 es un robot industrial traído por la firma ABB. Se trata de un brazo robótico caracterizado por diseño compacto, versatilidad y gran flexibilidad de montaje, haciéndolo ideal para integrarlo en múltiples entornos de aplicación.



Fig. 3.10. Brazo industrial modelo IRB [28]

Dentro de la familia IRB supone el siguiente paso de sus modelos predecesores 140 y 120. Mejorando el espacio de trabajo empleado, permitiendo la instalación del robot en áreas más reducidas y limitadas, a su vez que las fuerzas de carga, incrementando la situación del baremo superior para desempeñar sus funciones en diferentes contextos [29].

IRB 1200-5/0.9

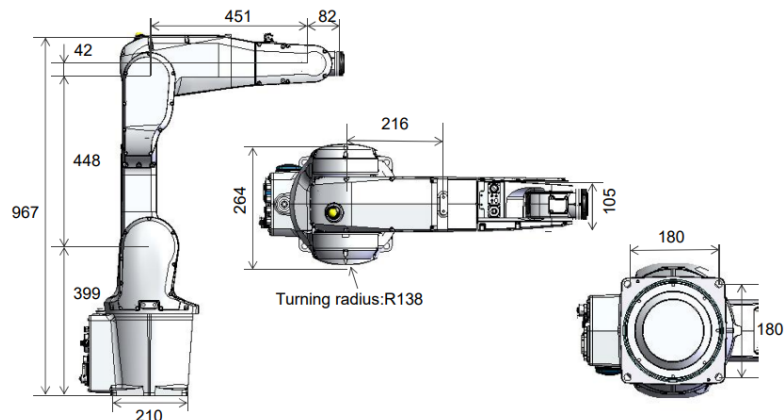


Fig. 3.11. Planos de diseño del IRB-1200 [29]

Se caracteriza por su estructura que lo dota de 6 GDL, permitiéndole alcanzar un amplio rango de movimiento. Define su zona de trabajo por la combinación de los rangos que disponen cada una de sus articulaciones y segmentos, delimitando el espacio por donde el brazo puede moverse y operar sin obstáculos. Al no contar con un offset en el eje 2, dispone de un rango de movimiento superior al de otros robots de su misma categoría, permitiendo situar el brazo muy cerca del área de trabajo por necesidades logísticas, haciendo capaz de desempeñar ciclos un 10 % más rápido que sus análogos [29].

El modelo más básico del 1200 está equipado con una protección IP40, sin embargo ABB ofrece opciones adicionales que mejoran notablemente sus capacidades, adecuándolo a entornos más exigentes con una protección IP64. Estas protecciones y características lo hacen una opción viable para su uso en diversos mercados, como el ensamblaje, la manipulación de alimentos, y el manejo de materiales. El diseño compacto y sellado junto a su características de protección le permiten operar sin problemas en escenarios hostiles o con materiales contaminantes que puedan afectar a su integridad. Además de contar con la propiedad de “Cleanroom robot” lo que significa que durante su operación ofrece un entorno salubre y no contaminante cumpliendo con la certificación ISO 14644-1 [29].

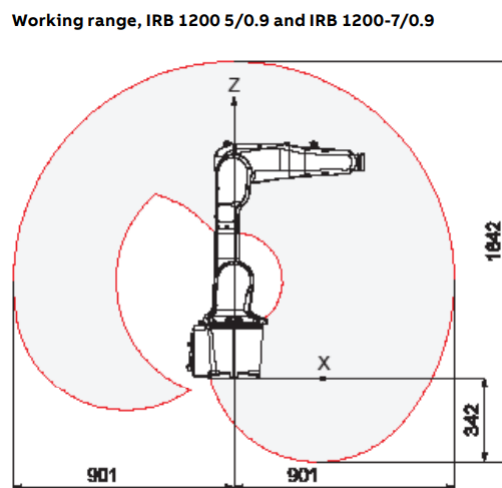


Fig. 3.12. Representación de alcance del IRB-1200 [29]

TABLA 3.3. CARACTERÍSTICAS GENERALES DEL IRB 1200

	IRB 1200-5/.09
Carga útil	5 kg
Alcance	901 mm
Precisión	.02 mm
Huella	210mm * 210mm
Peso	52 kg
Posición de montaje	Suelo, pared, techo
Protección IP	IP 40 (standad), IP 67 (Foundry Plus 2)
Clean Room	Disponible
SaveMove2	Disponible
Food Grade Lubrication	Disponible

Fuente: [28]

TABLA 3.4. ESPECIFICACIONES DE MOVIMIENTO DEL IRB 1200

Eje de movimiento	Rango de trabajo	Velocidad eje max.
Eje 1 rotación	+170° a -170°	288°/s
Eje 2 brazo	+130° a -100°	240°/s
Eje 3 brazo	+70° a -200°	297°/s
Eje 4 muñeca	+270° a -270°	400°/s
Eje 5 flexión	+130° a -130°	405°/s
Eje 6 giro	+400° a -400° (Max.rev. +-242)	600°/s

Fuente: [29]

Al tratarse de un producto que viene directamente de la mano de ABB esto supone que cuenta con un gran soporte de configuración por parte de uno de los principales modelos de software de la compañía: RobotStudio. Permitiendo a cualquiera con acceso al software, a configurar y simular el brazo para una infinidad de tareas y procesos.

3.2.2. Python

Es un lenguaje de programación de alto nivel utilizado en múltiples áreas de desarrollo. Principalmente empleado en áreas como:

- Ciencias de datos
- Desarrollo de software
- Automatización



Fig. 3.13. Logo Python [30]

Es una opción popular en el desarrollo de aplicaciones gracias a su simplicidad y legibilidad que permite a los desarrolladores escribir código fácil de entender y que habilita la colaboración entre proyectos de forma sencilla.

Se trata de un lenguaje capaz de operar entorno a múltiples paradigmas al funcionar mediante scripts y es capaz de emplear diversas estructuras de datos como programación orientada a objetos, programación estructurada y programación estructural. Las posibilidades de enfoque que se abren respecto a las funcionalidades a las que se puede llegar con este lenguaje de programación lo sitúan como uno de los más flexibles entre los suyos. Además el intérprete de Python dispone de soporte en múltiples sistemas operativos, una amplia comunidad de desarrolladores y una multitud de librerías siempre en crecimiento; estas características permiten compartir información con otras personas en proyectos similares impulsando a la cooperación y así expandir el alcance de los objetivos en estos proyectos [30].

Cabe la mención de las diferentes clases de librerías utilizadas para este proyecto:

pygame

Esta librería multimedia está principalmente enfocada en su uso para la creación de videojuegos en 2D. Pero su conjunto de módulos también permiten el diseño de otras aplicaciones multimedia basadas en el lenguaje de Python. Capaz de ejecutarse casi desde cualquier plataforma, es seleccionado por sus funcionalidades de dibujo con las que se representa el display [31], [32].

```
1 | python3 -m pip install -U pygame==2.6.0
```

socket

La librería socket permite configurar una interfaz de conexión a red de bajo nivel, empleada para mantener en conexión las aplicaciones dentro del entorno de Python. Los parámetros de configuración que se han empleado para llevar a cabo las comunicaciones son los siguientes: `AF_INET`: este parámetro utilizado especifica las direcciones que se utilizarán durante la conexión, en concreto empleando el protocolo IPv4, el cual se debe

tener en cuenta a la hora de especificar las direcciones de conexión para hacerlo correctamente. `SOCK_STREAM`: se encarga de establecer la conexión con protocolo TCP que cuenta con características que garantizan el intercambio de datos sin errores, enviándolos como un flujo de bytes concreto que puede ser establecido como bidireccional. Pertinente para la integridad y orden de los datos [33].

abb_robot_client.egm

Este paquete de Python ofrece la creación de clientes para robots ABB. Su modo de empleo puede ser a través de RWS (Robot Web Services) o EGM, siendo este último el método seleccionado. Su compatibilidad solo se encuentra disponible con los controladores robóticos IRC5, y a menudo es empleado junto a la librería `[abb_motion_program_exec]` permitiendo un mejor diseño de interfaz de programas de movimiento [34].

```
1 | pip install abb-robot-client[aio]
```

abb_motion_program_exec

Empleada para el envío de ordenes al entorno robótico virtual, la librería permite transferir y ejecutar unas secuencias de comandos en controladores robóticos IRC5 como "MoveJ", "MoveAbsJ", "MoveL", "MoveCz "WaitTime". Cuenta con soporte para la transmisión de ordenes EGM para objetivos de posición, articulaciones y correcciones de trayecto [35].

```
1 | pip install abb-motion-program-exec
```


4. DESARROLLO

En esta sección se describe la implementación del sistema de teleoperación multimodal para robots ABB, compuesta por tres módulos principales: interfaz gráfica, estación con robot, y método de comunicación.

4.1. Implementación de interfaz gráfica GUI

En primer lugar se describe el bloque de la interfaz gráfica desarrollada en código Python. Dicha interfaz gestiona e interpreta la activación de botones de los sensores de control: Space Navigator y Xbox Controller.

Lector Xbox

El código de lectura del controlador xbox ha sido desarrollado a partir de [36] y [37]. Es un programa de lectura compatible con dispositivos que dispongan de dos palancas direccionales analógicas, cada uno con un botón digital, dos triggers analógicos, una almohadilla direccional digital con cuatro direcciones y ocho botones digitales.

La interfaz de lectura del mando debe su función principalmente a la clase [**XInputJoystick**], pues en ella se llevan a cabo los procesos de reconocimiento de estados, ajustes, limitaciones y demás características que ofrece el programa. en su interior uno de los métodos más notorios es [**get-state**] el cual ofrece el estado actual del mando a tiempo real por medio del protocolo XInput de Windows definido previamente para su uso.

```
1     def get_state(self):
2         "Obtiene el estado del controlador representado por este
          objeto"
3         state = XINPUT_STATE()
4         res = xinput.XInputGetState(self.device_number, ctypes.byref(
          state))
5         if res == ERROR_SUCCESS:
6             return state
7         if res != ERROR_DEVICE_NOT_CONNECTED:
8             raise RuntimeError(
9                 "Unknown error %d attempting to get state of device
              %d" % (res, self.device_number))
10        # else return None (no hay dispositivo conectado)
```

Estos datos obtenidos al momento son empleados por los diversos procesos de gestión, con los que se clasifica y reconoce la información que llega por parte del input del controlador. Es a través de [**dispatch-events**] que se genera el bucle principal de eventos y que

se recurre a los métodos por los que se decodifica la información obtenida separándola en los tipos correspondientes para su representación, así como **[dispatch-button-events]** para los eventos de los botones y **[dispatch-axis-events]** para todo aquello que no son botones (palancas y gatillos).

```

1  def dispatch_events(self):
2      "Bucle principal de eventos de joystick"
3      state = self.get_state()
4      if not state:
5          raise RuntimeError(
6              "Joystick %d is not connected" % self.device_number)
7      if state.packet_number != self._last_state.packet_number:
8          # si el estado cambia lo maneja
9          self.update_packet_count(state)
10         self.handle_changed_state(state)
11         self._last_state = state
12
13     def handle_changed_state(self, state):
14         "Maneja varios eventos resultantes de cambios de estados"
15         self.dispatch_event('on_state_changed', state)
16         self.dispatch_axis_events(state)
17         self.dispatch_button_events(state)

```

Por lo general los controladores con gatillos o palancas analógicas ofrecen sus valores de bit, con los cuales puede resultar abrumador trabajar, por ello se decide normalizar sus datos a un rango de trabajo más operable: $[0, 1]$ ó $[-0.5, 0.5]$ dependiendo de si tienen signo o no. Esto se lleva a cabo por medio de una sencilla operación de normalización, garantizando que los valores se ajusten a un rango estándar independientemente del tamaño de su alcance y haciendo más sencilla su operación con ellos.

$$valorN = valor / (2^b - 1) \quad (4.1)$$

Siendo *valorN* el valor normalizado, y *b* el numero de bits, se consigue transformar cualquier rango de valores analógicos a una escala unitaria.

```

1  def translate_using_data_size(self, value, data_size):
2      # normaliza los datos analógicos en un rango de [0, 1] para
3      # los rangos estrictamente positivos
4      # y [-0.5,0.5] para los datos con simbolo
5      data_bits = 8 * data_size
6      return float(value) / (2 ** data_bits - 1)

```

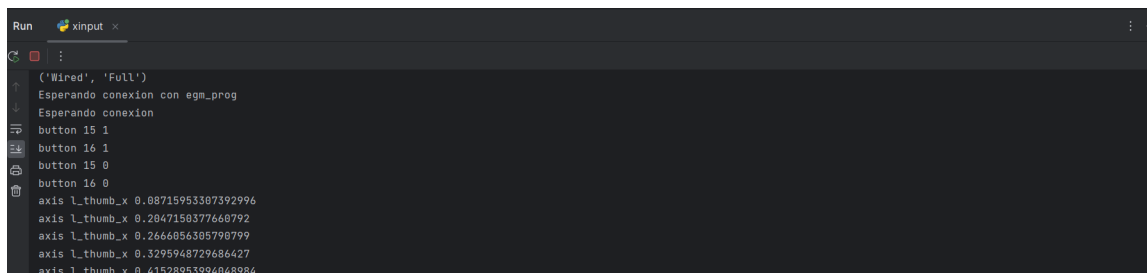
También se ha decidido implementar un sistema de zonas muertas, para gestionar cualquier clase de "drifting" que pueda presentar el controlador del usuario. Que, junto a la normalización de valores constituyen un conjunto de operaciones que contribuyen a la precisión de control.

```

1     def dispatch_axis_events(self, state):
2         # axis fields se refieren a todo menos los botones
3         axis_fields = dict(XINPUT_GAMEPAD._fields_)
4         axis_fields.pop('buttons')
5         for axis, type in list(axis_fields.items()):
6             old_val = getattr(self._last_state.gamepad, axis)
7             new_val = getattr(state.gamepad, axis)
8             data_size = ctypes.sizeof(type)
9             old_val = self.translate(old_val, data_size)
10            new_val = self.translate(new_val, data_size)
11
12            # establece zonas muertas: minimas
13            # ags, 2014-07-01
14            if ((old_val != new_val and (new_val >
15                0.08000000000000000 or new_val <
16                -0.08000000000000000) and abs(old_val - new_val) >
17                0.000000000500000000) or
18                (axis == 'right_trigger' or axis == 'left_trigger')
19                and new_val == 0 and abs(old_val - new_val) >
20                0.000000000500000000):
21                self.dispatch_event('on_axis', axis, new_val)

```

Por ultimo el método de **[sample-first-joystick]** es el encargado de mostrar por medio de la consola el estado del controlador, al llamar a **[dispatch-events]** y ofrecerle una interfaz sencilla por medio de la consola del compilador que, aunque es poco visual, resulta extremadamente útil a la hora de mantener un registro exacto de todo lo que ha detectado del controlador durante una sesión.



```

Run xinput
('Wired', 'Full')
Esperando conexion con egm_prog
Esperando conexion
button 15 1
button 16 1
button 15 0
button 16 0
axis l_thumb_x 0.08715953307392996
axis l_thumb_x 0.2047150377660792
axis l_thumb_x 0.2666056305790799
axis l_thumb_x 0.3295948729686427
axis l_thumb_x 0.41520953994048984

```

Fig. 4.1. Resultado de consola al leer el controlador Xbox

Lector Spacenavigator

El código de lectura del controlador spacenavigator ha sido desarrollado a partir del [38]. Esta interfaz de lectura dispone de compatibilidad con multitud de productos de la gama spacenavigator, aunque los paquetes de envío de datos se encuentran configurados para tratar exclusivamente con aquellos parámetros correspondientes al modelo básico, al ser esta la herramienta entorno a la que se ha trabajado.

La estructuración de datos dentro del programa se establece a través de dos tuplas

'AxisSpec' y 'ButtonSpec' que se encargan de guardar, con un orden predefinido, la serie de datos correspondientes con el estado actual del eje y los botones respectivamente. Aun así la funcionalidad de la aplicación se estructura entorno a la clase [**DeviceSpec**] donde se encuentra diseñada toda la lógica de gestión del dispositivo y la actualización de datos.

```
1 class DeviceSpec(object):
2     """Contiene las especificaciones de un solo dispositivo 3
3     Dconnexion"""
4
5     def __init__(
6         self, name, hid_id, led_id, mappings, button_mapping,
7         axis_scale=350.0
8     ):
9         self.name = name
10        self.hid_id = hid_id
11        self.led_id = led_id
12        self.mappings = mappings
13        self.button_mapping = button_mapping
14        self.axis_scale = axis_scale
15
16        self.led_usage = hid.get_full_usage_id(led_id[0], led_id[1])
17        # se inicializa en un vector de reposo "0" para cada estado
18        self.dict_state = {
19            "t": -1,
20            "x": 0,
21            "y": 0,
22            "z": 0,
23            "roll": 0,
24            "pitch": 0,
25            "yaw": 0,
26            "buttons": ButtonState([0] * len(self.button_mapping)),
27        }
28        self.tuple_state = SpaceNavigator(**self.dict_state)
29
30        # start in disconnected state
31        self.device = None
32        self.callback = None
33        self.button_callback = None
34
35        def describe_connection(self):
36            ....
37            ....
38
39        @property
40        def connected(self):
41            ....
42
43        @property
44        def state(self):
45            ....
```

```

44         ....
45
46     def open(self):
47         ....
48         ....
49
50     def set_led(self, state):
51         ....
52         ....
53
54     def close(self):
55         ....
56         ....
57
58     def read(self):
59         ...
60         ....
61
62     def process(self, data):
63         ....
64         ....

```

El dispositivo, al conectarse a la aplicación de lectura, es abierto por medio del método externo a la clase **[open]** y lo convierte en el dispositivo activo actual, haciendo que todos los procesos se centren en la información que este ofrezca; sin embargo debe de ser reconocido como un elemento compatible para su lectura. Si no se encuentra dentro de su lista de dispositivos permitidos, nada de esto será posible. En caso de que se encuentre un dispositivo compatible se generará un objeto con la identidad del controlador y vinculado a él por el que se llevarán a cabo todas las iteraciones.

El spacenavigator presenta 6 sensores de posicionamiento susceptibles a la detección de cualquier movimiento de su eje. Dichos cambios de estado se gestionan a través de un manejador de datos en bruto (raw data handler) obtenido gracias a la librería 'pywinusb.hid'. Esto establece una función que se llama automáticamente cada vez que se detecta movimiento en el controlador.

```

1 | dev.set_raw_data_handler(lambda x: new_device.process(x))

```

Los datos del nuevo estado son entonces recibidos por la función **[process]** del objeto. En ella se compara esta nueva información del estado del controlador con el anterior, comprobando que realmente ha habido un cambio, y de ser así actualiza el estado del objeto, estableciendo los valores para cada eje [x, y, z, roll, pitch, yaw] en un rango de [-1.5, 1.5], asegurándose de que la nueva tupla de estados solo se configure cuando los 6 GDL han sido leídos correctamente. De este modo se obtiene un refresco constante de la situación del dispositivo en caso de actividad y sin necesidad de un bucle continuo de comprobación.

```

1  def process(self, data):
2      button_changed = False
3
4      for name, (chan, b1, b2, flip) in self.mappings.items():
5          if data[0] == chan:
6              self.dict_state[name] = (
7                  flip * to_int16(data[b1], data[b2]) / float(
8                      self.axis_scale)
9              )
10
11     for button_index, (chan, byte, bit) in enumerate(self.
12         button_mapping):
13         if data[0] == chan:
14             button_changed = True
15             # update the button vector
16             mask = 1 << bit
17             self.dict_state["buttons"][button_index] = (
18                 1 if (data[byte] & mask) != 0 else 0
19             )
20
21     self.dict_state["t"] = high_acc_clock()
22
23     # debe de recibir ambas partes del estado de los 6GDL antes
24     # de devolver el diccionario de estado
25     if len(self.dict_state) == 8:
26         self.tuple_state = SpaceNavigator(**self.dict_state)
27
28     # llama cualquier llamada relacionada
29     if self.callback:
30         self.callback(self.tuple_state)
31
32     # solo llama a la llamada de los botones si el estado de los
33     # botones ha cambiado
34     if self.button_callback and button_changed:
35         self.button_callback(self.tuple_state, self.tuple_state.
36             buttons)

```

Finalmente es el procedimiento **[print-state]** el encargado de copiar a la consola el cambio de estado cada vez que el el programa detecta uno.

```

Run space_navigator
Devices found:
SpaceNavigator
SpaceNavigator found
SpaceNavigator connected to 3Dconnexion SpaceNavigator version: 1028 [serial: ]
Esperando conexion
Esperando conexion
x +0.00 y +0.00 z +0.00 roll +0.00 pitch +0.00 yaw +0.00 t +692806.17
buttons=[0, 1]
x +0.00 y +0.00 z +0.00 roll +0.00 pitch +0.00 yaw +0.00 t +692806.52
buttons=[0, 0]
x +0.00 y +0.00 z +0.00 roll +0.00 pitch +0.00 yaw +0.00 t +692808.42
x +0.00 y +0.00 z +0.00 roll -0.01 pitch +0.00 yaw +0.00 t +692808.42

```

Fig. 4.2. Resultado de consola al leer el controlador Spacenavigator

Conexión servidor-cliente TCP

A ambos lectores se les acoplan módulos adicionales para la comunicación TCP por medio de sockets, en los que tomarán el rol de servidores que conectarán a sus respectivos clientes: tanto con la aplicación de display como con la de conexión EGM; para mantener un flujo constante de entrega de información en el que se envía una biblioteca de estados con los datos

En este ejemplo se muestra la configuración del servidor del sistema de lectura del controlador xbox, pero la estructura es prácticamente idéntica para el spacenavigator:

```
1  def run_xinput_server(address='localhost', port=5000):
2      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
3      )
4      server_socket.bind((address, port))
5      server_socket.listen(1)
6      print("Esperando conexion con egm_prog")
7      connection, client_address = server_socket.accept()
8      print("Conexion de", client_address)
9
10     joystick = XInputJoystick(0)
11
12     try:
13         while True:
14             joystick.dispatch_events() # para leer a tiempo real
15             los eventos internos del joystick
16             state = joystick.get_state()
17             if state:
18                 data = {
19                     'buttons': state.gamepad.buttons,
20                     'left_trigger': state.gamepad.left_trigger /
21                         255.0,
22                     'right_trigger': state.gamepad.right_trigger /
23                         255.0,
24                     'l_thumb_x': state.gamepad.l_thumb_x / 32767.0,
25                     'l_thumb_y': state.gamepad.l_thumb_y / 32767.0,
26                     'r_thumb_x': state.gamepad.r_thumb_x / 32767.0,
27                     'r_thumb_y': state.gamepad.r_thumb_y / 32767.0,
28                     'cartesian_mode': joystick.cartesian_mode
29                 }
30                 message = json.dumps(data) + '\n'
31                 connection.sendall(message.encode('utf-8'))
32                 # # connection.sendall(json.dumps(data).encode('utf
33                 -8'))
34                 # connection.sendall(data.encode('utf-8'))
35                 time.sleep(0.01) # limita frec de envio a 100Hz
36     finally:
37         connection.close()
38         server_socket.close()
```

Display de los controladores

Para mostrar de forma clara que está ocurriendo en todo momento con los controladores sin necesidad de recurrir a mirar constantemente a los valores en la consola y poder salir del compilador del programa, se aplica la intervención de un display. De este modo se le ofrece al usuario una alternativa más visual sobre su controlador, y como reconocen las comunicaciones los inputs ofrecidos durante la teleoperación. Se les asigna la función de clientes de la comunicación TCP, y funcionan en paralelo al resto de funcionalidades sin intervenir o interrumpir en ninguna de ellas.

El dibujado de su GUI se realiza a través de los comandos disponibles en la librería pygame, con los que se trata de replicar la estructura de elementos que presenta el controlador al que está ligado, facilitando así la correlación visual del operador.

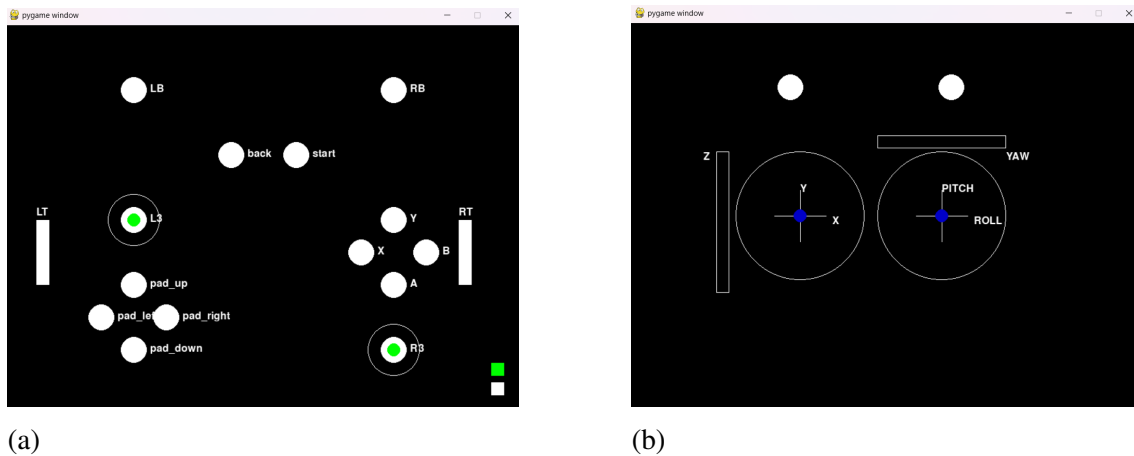


Fig. 4.3. Izquierda(a): Visualización del GUI del controlador Xbox; Derecha(b): Visualización del GUI del controlador Spacemanager [12]

Cada modo de dibujado y ajuste de parámetros acorde a los datos recibidos varía respecto al controlador empleado; sin embargo la base de comunicación y aplicación de los valores se realiza de forma común para todos los controladores, empleando el mismo procedimiento de cliente y lógica de dibujo. Siendo el extremo receptor de la relación servidor-cliente con el lector, la aplicación de display debe de ser capaz de decodificar los datos enviados por el lector del mismo modo en el que han sido codificados y tener una frecuencia de muestreo acorde al servidor para evitar así la pérdida de paquetes, una vez recibidos se guardan dinámicamente en una variable que se usará para recurrir a la subbiblioteca creada y así obtener los datos de cambio de estado en tiempo real dentro del cliente.

```
1 if __name__ == "__main__":
2     pygame.init()
3     screen = pygame.display.set_mode((800, 600))
4     font = pygame.font.Font(None, 24)
5     # run_display_client()
```



```

6     client_socket = run_display_client()
7
8     buffer = ""
9     try:
10         while True:
11             data = client_socket.recv(4096).decode('utf-8') # antes
                recv() era 1024
12             if not data:
13                 break
14             buffer += data
15             while '\n' in buffer:
16                 message, buffer = buffer.split('\n', 1)
17                 state = json.loads(message)
18                 update_display(screen, state)
19             # state = json.loads(data.decode('utf-8'))
20             # update_display(screen, state)
21
22             for event in pygame.event.get():
23                 if event.type == pygame.QUIT:
24                     pygame.quit()
25                     sys.exit()
26
27             # pygame.time.delay(10)
28     finally:
29         client_socket.close()

```

4.2. Comunicación entre módulos

El módulo que cumple la función de puente entre la aplicación de lectura y el entorno robótico es la pieza clave para hacer funcionar la conexión EGM del sistema. La relación cliente-servidor que tiene con el lector del controlador es muy similar a la presente en el display, codificando y enviando los datos de la misma manera, leyendo en bucle los datos recibidos para interpretarlos y modificar acordemente los parámetros que se enviarán al servidor para activar la unidad mecánica.

```

1     while True:
2         t2 = time.perf_counter()
3         res, feedback = egm.receive_from_robot(timeout=0.05)
4         if res:
5             data = xinput_socket.recv(4096).decode('utf-8')
6             buffer += data
7             if '\n' in buffer:
8                 message, buffer = buffer.split('\n', 1)
9                 state = json.loads(message)

```

Para establecer una transmisión de datos correcta hacia el entorno robótico se deben de especificar unos parámetros de configuración requeridos, generando correctamente el

archivo que el controlador robótico debe de leer, evitando así confusiones o asignaciones erróneas en los datos. Entre estos parámetros se encuentran aquellos que definen el tipo de operación a realizar durante la sesión, si esta va a ser articular o cartesiana.

Dependiendo de por cual se quiera optar hay ciertos parámetros específicos que se deben estipular junto al resto. Del mismo modo, la forma en la que se establece que los cambios de estado varíen las ordenes de operación también es modificada acorde para ofrecer un control coherente en base a la situación. Variando únicamente la posición de las articulaciones en el protocolo de movimiento articular, e indicando, por un lado el desplazamiento en los ejes [x, y ,z], y por el otro la orientación mediante cuaterniones [q1, q2, q3, q4] en el protocolo de movimiento cartesiano

```
1      mm = abb.egm_minmax(-1e-3, 1e-3)
2
3      egm_config = abb.EGMJointTargetConfig(
4          mm, mm, mm, mm, mm, mm, 1000, 1000
5      )
6      # /////
7      ...
8      En los casos de requerir de un control ARTICULAR o CARTESIANO
9      se deberan especificar en esta parte los par metros de
        iniciacion
10     ...
11     # /////
12
13     client = abb.MotionProgramExecClient(base_url="http
        ://127.0.0.1:80")
14     lognum = client.execute_motion_program(mp, wait=False)
15
16     xinput_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
        )
17     xinput_socket.connect(('localhost', 5001))
18     buffer = ""
19
20     t1 = time.perf_counter()
21
22     egm = EGM()
```

De este modo se están transmitiendo ordenes de movimiento controladas mediante los datos de lectura de estado, originados en los lectores de controlador, directamente al entorno robótico en tiempo real, actualizando los paquetes de datos constantemente y siendo gestionados gracias a los procedimientos correspondientes dentro de RobotStudio. Por último al finalizar la sesión de comunicación con el entorno de RobotStudio este cliente reclama un archivo log que se ha estado realizando durante la operación del brazo robótico en el servidor, registrando los movimientos de sus articulaciones durante el desarrollo de la sesión de forma gráfica

4.3. Estación en RobotStudio

El entorno robótico virtual basa su funcionamiento en 4 programas diferentes escritos en formato RAPID, lenguaje interno del software de RobotStudio. Estos programas disponen de una funcionalidad específica cada uno que ayudan a que se lleven a cabo todos los procesos necesarios para el funcionamiento del control remoto en tiempo real. Este segmento del código se ha realizado a partir de [39], y pueden ser vistos en su totalidad en el anexo [A.5]

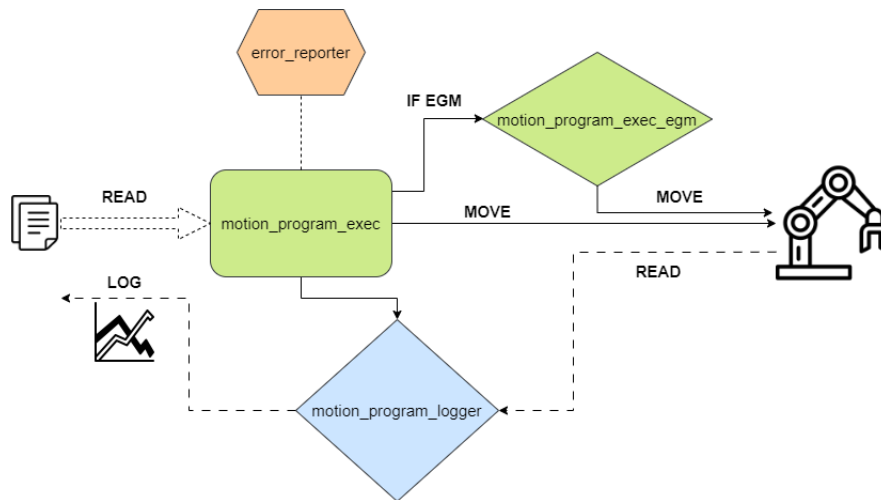


Fig. 4.4. Esquema de interacción de los módulos dentro del controlador robótico

error_reporter

Módulo encargado de la gestión y reporte de errores durante la ejecución de la aplicación para lograr un diagnóstico completo de la situación. Este método es implementado debido a que la conexión EGM ignora múltiples protocolos de seguridad, volviendo a la zona robótica un entorno de trabajo hostil si no se gestiona cuidadosamente, por lo que un sistema de monitorización de errores resulta de gran utilidad.

En un principio se decide almacenar el valor correspondiente al error que se detecta en una variable, que es utilizada para poder manejar el error en cuestión.

```
1  VAR intnum motion_program_err_interrupt;
```

El proceso principal es diseñado con la intención de conectar el disparo de errores que ocurren en el sistema durante su ejecución con el método de trampa indefinidamente para conseguir enviarle los datos en cualquier momento que sea detectado.

El uso de trampas 'TRAMP' permiten al programa manejar la situación dentro de ellas sin detenerse por completo, operando como excepciones dentro de la aplicación, lo que resulta extremadamente útil a la hora de lidiar con errores puntuales. **[TRAP mo-**

tion_program_trap_err] Se encarga de capturar los errores cuando se disparan y guardarlos en un registro. Para ello se obtienen los datos del error enviados por el proceso principal del módulo del que luego se extraen sus datos, como su dominio, número, título, cadenas de texto y otros importantes para un posible diagnóstico. Finalmente en este método se emplea una función de escritura de errores 'ErrWrite' con el que se registra el archivo de log que contiene un mensaje de fallo junto a los detalles del error.

```

1  TRAP motion_program_trap_err
2      VAR trapdata err_data;
3      VAR errdomain err_domain;
4      VAR num err_number;
5      VAR errtype err_type;
6      VAR string titlestr;
7      VAR string string1;
8      VAR string string2;
9      VAR num seqno;
10     VAR string err_filename;
11     VAR iodev err_file;
12
13     GetTrapData err_data;
14     ReadErrData err_data, err_domain, err_number, err_type \
        Title:=titlestr \Str1:=string1 \Str2:=string2;
15
16     ErrWrite \W, "Motion Program Failed", "Motion Program Failed
        at command number " + NumToStr(motion_program_state{1}.
        current_cmd_num,0)
17     \RL2:= "with error code " + NumToStr(err_domain*10000+
        err_number,0)
18     \RL3:= "error title '" + titlestr + "'"
19     \RL4:= "error string '" + string1 + "'";
20
21     seqno := motion_program_seqno_started;
22     IF seqno > 0 THEN
23         err_filename := "motion_program_err---seqno-" + NumToStr
            (seqno,0) + ".json";
24         Open "RAMDISK:"\File:=err_filename,err_file\Write;
25         Write err_file, "{"seqno": " + NumToStr(seqno,0)\
            NoNewLine;
26         Write err_file, ","error_domain": " + NumToStr(
            err_domain,0)\NoNewLine;
27         Write err_file, ","error_number": " + NumToStr(
            err_number,0)\NoNewLine;
28         Write err_file, ","error_title": "" + titlestr +
            ""\NoNewLine;
29         Write err_file, ","error_string": "" + string1 +
            ""\NoNewLine;
30         Write err_file, ","error_string2": "" + string2 +
            ""\NoNewLine;
31         Write err_file, "}";

```

```

32         Close err_file;
33     ENDIF
34
35
36     ENDTRAP

```

Para guardar correctamente los errores se opta por establecer que si el disparo del error es válido, siendo el valor de secuencia de este mayor que 0, los errores detectados son almacenados en un archivo JSON que incluye en número de secuencia, el dominio y el número del error, además de descripciones adicionales si las tuviese. Todo ello para su registro y posterior análisis.

motion_program_logger

Este módulo se dedica a leer y almacenar la información obtenida a partir de la ejecución de movimientos del robot, registrando en todo momento el estado de sus articulaciones. A la hora de almacenarlos los guarda en un archivo log para su posterior análisis. El desempeño de esta funcionalidad se aborda de la siguiente manera:

En el procedimiento **[main]** se llevan a cabo las principales funciones de lectura y gestión del archivo de registro. Durante un bucle cerrado, para muestrear constantemente, se leen los datos de posición de todas las articulaciones robóticas mediante 'CJointT' y se inicializa un reloj para adquirir la identidad temporal de la lectura. Del mismo modo en este procedimiento se establece frecuencia de muestreo que se activa solo cuando existe un archivo log abierto para la escritura. Y mediante la gestión de excepciones, concretamente la de mensajes, también se encarga de la apertura y cierre de los documentos de registro dependiendo de las ordenes recibidas.

```

1  PROC motion_program_logger_main()
2  ...
3      WHILE TRUE DO
4          clk_now:=ClkRead(time_stamp_clock \HighRes);
5          motion_program_state{1}.clk_time:=clk_now;
6          motion_program_state{1}.joint_position:=CJointT(\TaskRef:=
              T_ROB1Id);
7          ...
8          IF log_file_open THEN
9              clk_diff:= loop_count*0.004 - clk_now;
10             loop_count := loop_count+1;
11             IF clk_diff > 0 THEN
12                 WaitTime clk_diff;
13             ENDIF
14         ELSE
15             loop_count := 0;
16             ClkStop time_stamp_clock;
17             ClkReset time_stamp_clock;

```

```

18         WaitTime 0.004;
19         ClkStart time_stamp_clock;
20     ENDIF
21     ...
22 END WHILE
23 ...
24 ENDPROC

```

Para el correcto manejo de los archivos log estos deben abrirse y cerrarse de forma segura para evitar posibles errores o fallas en el registro de datos, siendo **[motion-program-log-open]** y **[motion-program-log-close]** los encargados de ello.

El **[motion-program-log-open]** es usado principalmente para abrir nuevos archivos logs con todos los datos de inicialización, como la versión del archivo o los nombres de las articulaciones, necesarios para establecer su formato; aunque en su procedimiento también se aplican los procesos de transformación, o empaquetamiento, de datos necesarios para su escritura en los archivos. Mientras que **[motion-program-log-close]** solo se encarga de cerrar el archivo.

```

1  PROC motion_program_log_open()
2      VAR string log_filename;
3      VAR string header_str:="timestamp,cmdnum,J1,J2,J3,J4,J5,J6";
4      VAR num header_str_len;
5      VAR num rmq_timestamp_len;
6      VAR rawbytes header_bytes;
7
8      IF robot_count >= 2 THEN
9          header_str:="timestamp,cmdnum,J1,J2,J3,J4,J5,J6,J1_2,
10             J2_2,J3_2,J4_2,J5_2,J6_2";
11      ENDIF
12
13      header_str_len:=StrLen(header_str);
14      rmq_timestamp_len:=StrLen(rmq_timestamp);
15      log_filename := "log-" + rmq_filename + ".bin";
16
17      pack_num motion_program_file_version, header_bytes;
18      pack_num rmq_timestamp_len, header_bytes;
19      pack_str rmq_timestamp, header_bytes;
20      pack_num header_str_len, header_bytes;
21      pack_str header_str, header_bytes;
22      Open "RAMDISK:" \File:=log_filename, log_io_device, \Write\
23          Bin;
24      WriteRawBytes log_io_device, header_bytes;
25      ErrWrite \I, "Motion Program Log File Opened", "Motion
26          Program Log File Opened with filename: " + log_filename;
27      log_file_open := TRUE;
28  ENDPROC
29
30  PROC motion_program_log_close()

```

```

28         Close log_io_device;
29         log_file_open := FALSE;
30         ErrWrite \I, "Motion Program Log File Closed", "Motion
           Program Log File Closed";
31     ERROR
32         SkipWarn;
33         TRYNEXT;
34     ENDPROC

```

Los datos leídos a partir del movimiento del robot, para poder ser enviados a través de la comunicación EGM, deben someterse a una serie de transformaciones que conviertan los diferentes tipos de datos en cadenas de bytes. Estas operaciones son llevadas a cabo en **[pack-sum]** para los valores numéricos, **[pack-str]** para cadenas de datos, **[pack-jointtarget]** para los datos correspondientes a las articulaciones del robot, transformando los datos de cada articulación individualmente.

```

1  PROC pack_num(num val, VAR rawbytes b)
2      PackRawBytes val, b, (RawBytesLen(b)+1)\Float4;
3      ENDPROC
4
5  PROC pack_str(string val, VAR rawbytes b)
6      PackRawBytes val, b, (RawBytesLen(b)+1)\ASCII;
7      ENDPROC
8
9  PROC pack_jointtarget(jointtarget jt, VAR rawbytes b)
10     pack_num jt.robax.rax_1, b;
11     pack_num jt.robax.rax_2, b;
12     pack_num jt.robax.rax_3, b;
13     pack_num jt.robax.rax_4, b;
14     pack_num jt.robax.rax_5, b;
15     pack_num jt.robax.rax_6, b;
16     ENDPROC

```

Otro de los procedimientos vitales para el graficado de datos es **[motion-program-log-data]**. Se le asigna la función de capturar los datos leídos en el **[main]**, tanto de posición y estado, como de reloj; para empaquetarlos posteriormente en una variable "data-byte" que pasará a contener toda la información requerida del proceso de operación para posteriormente escribirlo en el archivo log mediante 'WriteRawBytes'.

```

1  PROC motion_program_log_data()
2      VAR rawbytes data_bytes;
3      pack_num ClkRead(time_stamp_clock \HighRes), data_bytes;
4      pack_num motion_program_state{1}.current_cmd_num, data_bytes
5      ;
6      pack_jointtarget motion_program_state{1}.joint_position,
7      data_bytes;
8      IF robot_count >= 2 THEN
9          pack_jointtarget motion_program_state{2}.joint_position,
10         data_bytes;
11     ENDIF
12     ENDPROC

```

```

data_bytes;
8      ENDIF
9      WriteRawBytes log_io_device, data_bytes;
10     ENDPROC

```

El sistema de gestión de excepciones también se inicializa en el **[main]** para dispararse al detectar su desencadenante. Estos procesos son usados para monitorizar de errores durante la operación, procurando la integridad de los datos recopilados hasta el momento, y el manejo de mensajes externos que comanden la apertura o cierre de logs.

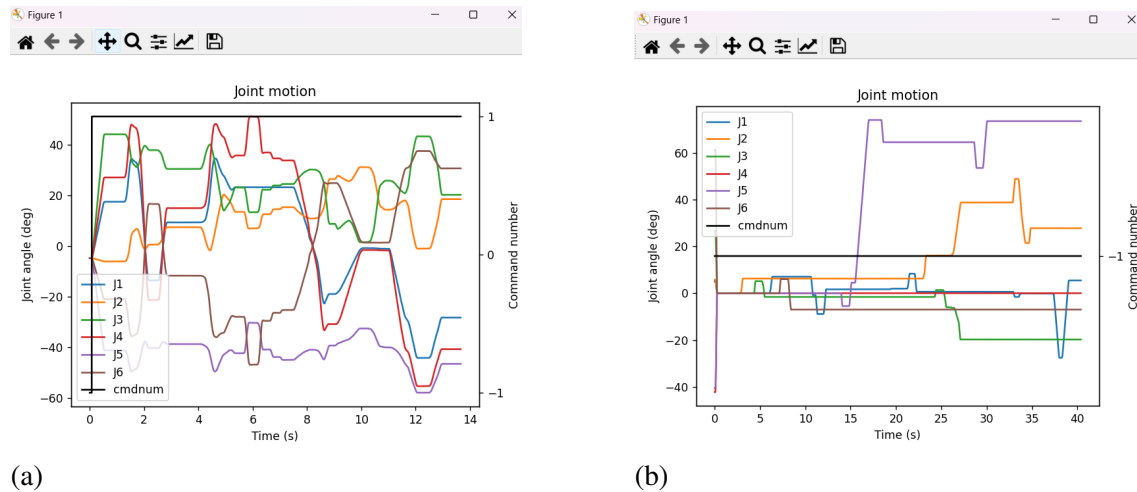


Fig. 4.5. Izquierda(a): Log de una operación de movimiento cartesiano; Derecha(b): Log de una operación de movimiento articular

motion_program_exec_egm

Módulo principal de la comunicación y gestión de ordenes para la teleoperación del brazo robótico. Cuenta con procedimientos y funciones que trabajan junto con la interfaz de comunicación EGM de ABB para la comunicación y transmisión de datos con aplicaciones externas en tiempo real [27].

Para comandar a la unidad mecánica por medio de ordenes provenientes de una aplicación externa primero es necesario inicializar la conexión EGM y asegurarse de que se establece correctamente en un canal seleccionado. En **[motion-program-egm-init]** configura la comunicación UDP mediante el uso de 'EGMSetupUC', y se asegura que, mientras el estado de la conexión indique que se encuentra ejecutando, esta no finalice, pero al pasar a otra categoría de estado cierre la conexión para evitar el cierres inesperados durante la operación. Además se establece una variable de estados que indique la existencia de una comunicación activa y funcionando para futuras gestiones.

```

1  PROC motion_program_egm_init()
2
3      BookErrNo ERR_NO_EGM;

```



```

4
5     IF egm_symbol_fail THEN
6         RETURN ;
7     ENDIF
8
9     EGMReset egmID1;
10    WaitTime 0.005;
11    EGMGetId egmID1;
12    egmSt1:=EGMGetState(egmID1);
13
14
15    EGMSetupUC ROB_1, egmID1, "joint", "UCdevice"\Joint\CommTimeout
        :=1000000;
16
17
18    ! IF egmST1=EGM_STATE_RUNNING THEN
19    !     EGMStop egmID1, EGM_STOP_HOLD;
20    ! ENDIF
21
22    EGMStreamStop egmID1;
23
24    IF NOT have_egm THEN
25        have_egm:=TRUE;
26    ENDIF
27    ENDPROC

```

La habilitación del recibimiento de datos a través de la conexión es realizada por medio de **[motion-program-egm-start-stream]** con el comando EGM de nombre similar 'EGMStartStream'. Mientras que para la selección del método de operación se lleva a cabo por el procedimiento **[motion-program-egm-enable]**, guiando a sus respectivas configuraciones dependiendo del valor del mensaje de recibido por el programa externo. Posteriormente se deben de realizar más ajustes a la conexión EGM para poder operar en método especificado; de lo cual se toma en cuenta en los procedimientos **[motion-program-egm-enable-joint]**, **[motion-program-egm-enable-pose]** y **[motion-program-egm-enable-corr]**. En ellos se contiene una estructura de funcionamiento muy similar en la que se asegura que la lectura de los datos de configuración recibidos es correcta y se establece el protocolo 'EGMActX' que es lo que permitirá su movimiento. Es necesario incluir que todos estos sistemas de movimiento necesitan de un desplazamiento inicial para activarse dentro de la configuración EGM [27]; por ello esta también se lleva a cabo en estos procedimientos por medio de :

```

1    joints:=CJointT();
2    joints.robax.rax_6:=joints.robax.rax_6+.0001;
3    MoveAbsj joints, v1000, fine, motion_program_tool\Wobj:=
        motion_program_wobj;

```

Fijarse en posibles errores dentro de esta etapa es crucial, debido a que indica algún

fallo en el formato de los datos configurados, los cuales son muy importantes tener en regla y en caso del disparo de alguno de estos errores, es conveniente revisar el formato de los datos enviados a través del programa externo.

Por otro lado se han creado una serie de funciones que lidian con el procesamiento de todos los datos recibidos durante la comunicación EGM, como **[try-read-egm-minmax]** y **[try-read-egm-frtype]** con los que se tratan de leer de forma sólida los parámetros de los que reciben el nombre; los **[try-motion-program-egmmoveX]** establecer movimientos puntuales con los protocolos 'EGMMoveC' y 'EGMMoveL' ;y los **[try-motion-program-egmrunX]** con las que se configura y ejecutan los protocolos 'EGMRunX' dependiendo del tipo de movimiento requerido para su ejecución en tiempo real, para ser posteriormente enviados al robot y hacerlo operar.

motion_program_exec

Este módulo es el nodo principal de gestión de las funcionalidades del entorno robótico. Recibiendo los archivos y datos de ejecución y ejecutando los procedimientos y/o módulos correspondientes para actuar acorde requiera la operación. Este módulo al ser adaptado a partir de otro repositorio [39] cuenta con funcionalidades que solo son útiles en otro contexto de control con otra clase de archivos, pero no realmente necesarias para la operación vía EGM

La inicialización principal de parámetros necesarios para la ejecución de la operación remota se lleva a cabo en **[motion-program-init]**, donde además se inicializa la conexión EGM en caso de estar disponible. Otras variables y registros son inicializados aquí.

```

1      PROC motion_program_init()
2          VAR string taskname;
3          BookErrNo ERR_INVALID_MP_VERSION;
4          BookErrNo ERR_INVALID_MP_FILE;
5          BookErrNo ERR_MISSED_PREEMPT;
6          BookErrNo ERR_INVALID_OPCODE;
7          taskname:=GetTaskName();
8          IF taskname="T_ROB1" THEN
9              motion_program_filename:="motion_program.bin";
10             task_ind:=1;
11         ELSEIF taskname="T_ROB2" THEN
12             motion_program_filename:="motion_program2.bin";
13             task_ind:=2;
14         ENDIF
15         IF task_ind=1 THEN
16             SetAO motion_program_preempt,0;
17             SetAO motion_program_preempt_current,0;
18             SetAO motion_program_preempt_cmd_num,-1;
19             SetAO motion_program_current_cmd_num,-1;
20             SetAO motion_program_queued_cmd_num,-1;
21             SetAO motion_program_seqno,-1;

```

```

22         ENDIF
23         motion_program_state{task_ind}.current_cmd_num:=-1;
24         motion_program_state{task_ind}.queued_cmd_num:=-1;
25         motion_program_state{task_ind}.preempt_current:=0;
26         motion_current_cmd_ind:=0;
27         motion_max_cmd_ind:=0;
28         IDelete motion_trigg_intno;
29         CONNECT motion_trigg_intno WITH motion_trigg_trap;
30         TriggInt motion_trigg_data,0.001,\Start,motion_trigg_intno;
31         RMQFindSlot logger_rmq,"RMQ_logger";
32         try_motion_program_egm_init;
33     ENDPROC

```

```

1     PROC try_motion_program_egm_init()
2         motion_program_egm_init;
3     ERROR
4         IF ERRNO=ERR_REFUNKPRC THEN
5             SkipWarn;
6             motion_program_have_egm:=FALSE;
7             TRYNEXT;
8         ENDIF
9     ENDPROC

```

Cuando un archivo llega al programa, para abrirlo se recurre a [**run-motion-program-file**] y [**open-motion-program-file**]. Juntos se encargan de abrir el archivo asegurándose de que todos los parámetros escritos en ellos sean correcto, y en caso de que no se correspondan con el formato de datos esperado disparan un error al sistema. También comprueban si el archivo recibido gestiona la comunicación EGM, en caso de que esto sea así se busca la activación del protocolo [**motion-program-egm-enable**] del módulo anterior para poder cargar los datos de configuración en el contexto adecuado.

```

1     PROC open_motion_program_file(string filename, bool preempt)
2         VAR num ver;
3         VAR tooldata mtool;
4         VAR wobjdata mwobj;
5         VAR loaddata mgripload;
6         VAR string timestamp;
7         VAR num egm_cmd;
8         VAR num seqno;
9
10        motion_program_state{task_ind}.motion_program_filename:=
11            filename;
12        motion_program_clear_bytes;
13        Open "RAMDISK:"\File:=filename,motion_program_io_device,\
14            Read\Bin;
15        IF NOT try_motion_program_read_num(ver) THEN
16            RAISE ERR_FILESIZE;
17        ENDIF

```

```

16      ....
17      ....
18      ....
19      IF NOT preempt THEN
20          motion_program_tool:=mtool;
21          motion_program_wobj:=mwobj;
22          motion_program_gripload:=mgripload;
23
24          SetSysData motion_program_tool;
25          SetSysData motion_program_wobj;
26          GripLoad motion_program_gripload;
27
28          IF motion_program_have_egm THEN
29              motion_program_egm_enable;
30          ELSE
31              IF NOT try_motion_program_read_num(egm_cmd) THEN
32                  RAISE ERR_INVALID_MP_FILE;
33              ENDIF
34              IF egm_cmd<>0 THEN
35                  RAISE ERR_INVALID_MP_FILE;
36              ENDIF
37          ENDIF
38      ELSE
39          IF NOT try_motion_program_read_num(egm_cmd) THEN
40              RAISE ERR_INVALID_MP_FILE;
41          ENDIF
42          IF egm_cmd<>0 THEN
43              RAISE ERR_INVALID_MP_FILE;
44          ENDIF
45      ENDIF
46
47      ENDPROC

```

El orden de toma de acción de comandos es llevado a cabo por medio de **[try-motion-program-run-next-cmd]**, se ocupa de leer el siguiente comando de movimiento y decidir la lógica de ejecución dependiendo del tipo de comando. Sin embargo para la comunicación EGM se opta por el envío de comandos simultáneos de aumento o disminución en los valores de posición, más que una serie de movimientos predefinidos con los que seguir un orden, pero es útil a la hora de enviar comandos en múltiples ejes o articulaciones de manera ordenada. Cabe incluir que esta función reconoce a un archivo de comunicación EGM gracias a que estos siempre empiezan por 50000, por lo que al hacerlo en vez de optar accede a las opciones de movimientos para EGM configuradas en **[motion_program_exec_egm]** Para la operación en tiempo real es necesario reemplazar las ordenes de comando rápidamente una vez estas cambien, para encargarse de eso se utiliza el procedimiento **[motion-program-do-preempt]** que se implementa para funcionar continuamente durante la ejecución del movimiento dado por el programa en **[motion-program-run]**. El Preempt, o pre-vaciado permite, en el momento que detecta la entrada

de una nueva orden, almacenar las dos ordenes, eliminar los datos de el archivo que se encuentra ejecutándose, y sustituir su información con la nueva orden inmediatamente, asegurando la ejecución de una nueva secuencia sin detener y reiniciar el programa.

```

1  PROC motion_program_run()
2
3      VAR bool keepgoing:=TRUE;
4      VAR num cmd_num;
5      VAR num cmd_op;
6      motion_program_state{task_ind}.current_cmd_num:=-1;
7      IF task_ind=1 THEN
8          SetAO motion_program_current_cmd_num,-1;
9      ENDIF
10     motion_program_state{task_ind}.running:=TRUE;
11     WHILE keepgoing DO
12         motion_program_do_preempt;
13         keepgoing:=try_motion_program_run_next_cmd(cmd_num,
14             cmd_op);
15     ENDWHILE
16     WaitRob\ZeroSpeed;
17     motion_program_state{task_ind}.running:=FALSE;
18     ERROR
19     motion_program_state{task_ind}.running:=FALSE;
20     RAISE ;
21 ENDPROC

```

```

1  FUNC bool try_motion_program_run_next_cmd(INOUT num cmd_num,INOUT
2      num cmd_op)
3
4      VAR num local_cmd_ind;
5
6      IF NOT (try_motion_program_read_num(cmd_num) AND
7          try_motion_program_read_num(cmd_op)) THEN
8          RETURN FALSE;
9      ENDIF
10
11     !motion_program_state.current_cmd_num:=cmd_num;
12     motion_max_cmd_ind:=motion_max_cmd_ind+1;
13     motion_program_state{task_ind}.queued_cmd_num:=
14         motion_max_cmd_ind;
15     IF task_ind=1 THEN
16         SetAO motion_program_queued_cmd_num,motion_max_cmd_ind;
17     ENDIF
18     local_cmd_ind:=((motion_max_cmd_ind-1) MOD 128)+1;
19
20     IF (cmd_op DIV 100000)=5 THEN
21         ! EGM command numbers start at 500000
22         motion_cmd_num_history{local_cmd_ind}:=-1;
23         RETURN try_motion_program_run_egm_cmd(cmd_num,cmd_op);

```

```
21         ENDIF
22         . . .
23         . . .
24
25     ENDFUNC
```

Para el apoyar los procedimientos de movimiento y reconocimiento de archivos, se crean unas pseudo-bibliotecas de movimientos que reconozcan en los datos del archivo su funcionalidad para así poder transformar esos datos en movimientos para la unidad mecánica y trabajan acorde con triggers para asegurar que el comando se lleva a cabo correctamente. También es generada una larga lista de funciones que abarcan todos los modos de lectura y tipos de archivo con los que se lidia durante esta clase de operaciones. De forma muy similar a los otros módulos estas funciones se encargan de que las lecturas sean a prueba de fallos, disparando excepciones si existe algún error, o asegurándose que se lee el segmento deseado del archivo; de este modo si existe algún error en la lectura del archivo se puede tratar de asegurar que es debido a una falla en el formato del mismo, y no en las funciones de lectura del programa.

De este modo se lleva a cabo el envío de datos su interpretación y lógica de ejecución para realizar movimientos en la unidad robótica correctamente.

5. RESULTADOS

5.1. Ejecución

En este apartado se describe como poner en funcionamiento el sistema teleoperado por medio de las aplicaciones presentadas para su correcto funcionamiento.

La configuración del entorno robótico que se va a explicar a continuación solo debe de realizarse una vez en el ordenador donde se vaya a ejecutar el programa. Primero es necesario contar un software de RobotStudio.

Ver: <http://new.abb.com/products/robotics/robotstudio/downloads>

Una vez instalado el software de ABB el siguiente paso es revisar el catálogo virtual en Add-Ins para descargar el RobotWare correspondiente al controlador IRC5. En este proyecto se ha utilizado la versión 6.14.00.01, aunque versiones más actualizadas de RobotWare para dicho controlador pueden funcionar.

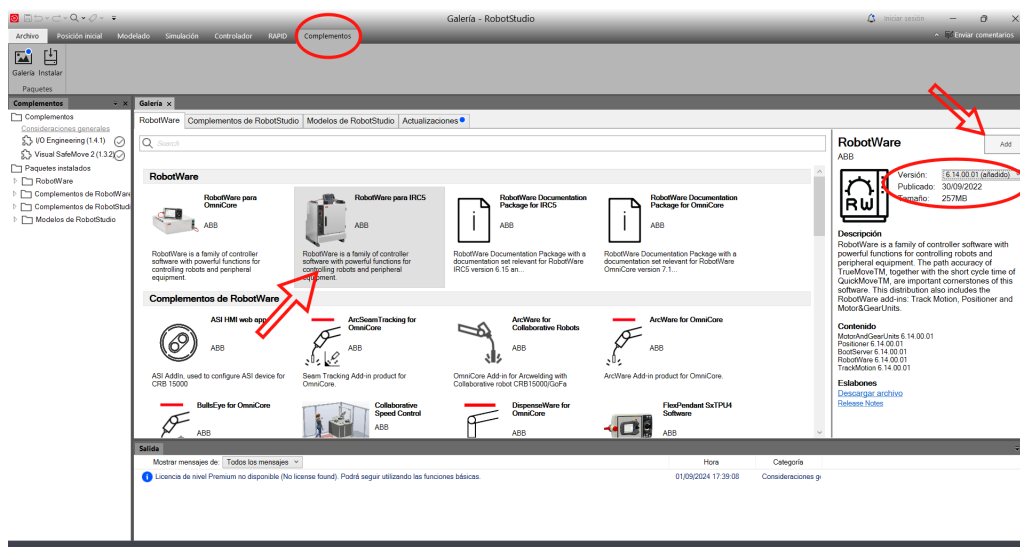


Fig. 5.1. Configuración del entorno robótico 1

A la hora de crear un nuevo proyecto asegurarse de especificar el correcto controlador y versión. Para el modelo del brazo robótico se ha empleado un modelo IRB-1200, aunque cualquier robot de la misma familia con 6 GDL debería ser igualmente adecuado.

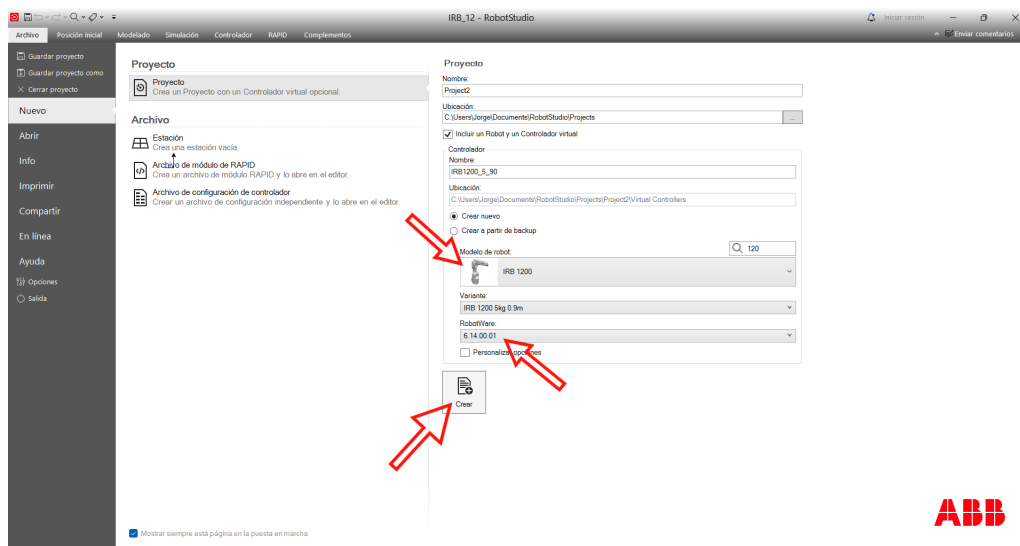


Fig. 5.2. Configuración del entorno robótico 2

Una vez generado el entorno virtual y cargado el controlador, se deberá ir a la pestaña de 'Controlador' (Controller), dentro de este espacio, bajo 'Estación Actual' (Current Station), se encontrará el controlador virtual con el nombre asignado durante la creación del proyecto. Con click derecho sobre él de abrirá una lista donde se deberá buscar y seleccionar 'Cambiar Opciones' (Change Options).

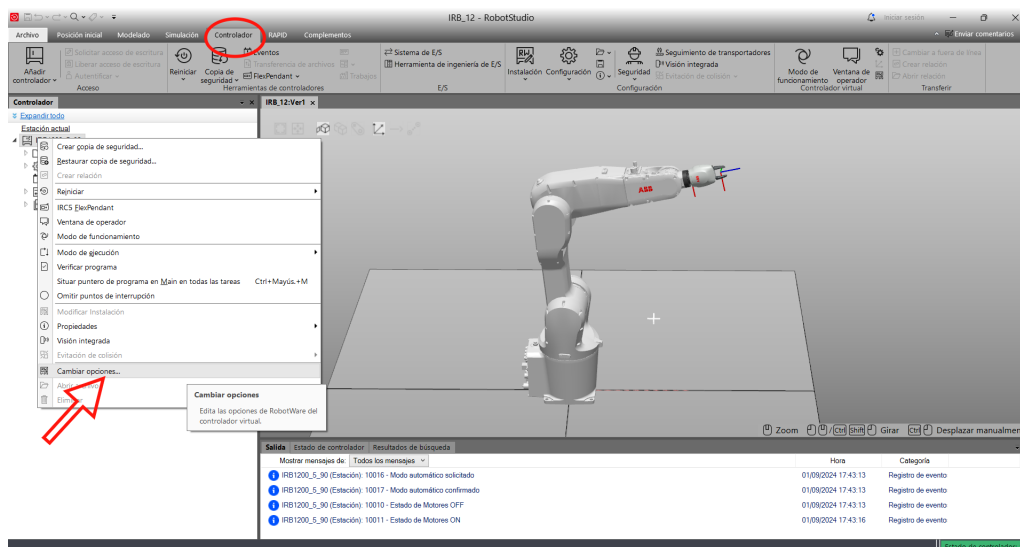


Fig. 5.3. Configuración del entorno robótico 3

Al seleccionarla se abrirá una ventana donde se podrán encontrar diversas categorías a su izquierda. Entre ellas se deben buscar: 'Comunicación' (Communication) donde aparecerá la opción 'PC Interface'; y 'Herramientas de Ingeniería' (Engineering Tools) donde aparecerán las opciones de 'Multitasking' y 'Externally Guided Motion (EGM)'. Se tendrán que activar estas opciones y pulsar 'OK' para confirmar su instalación.

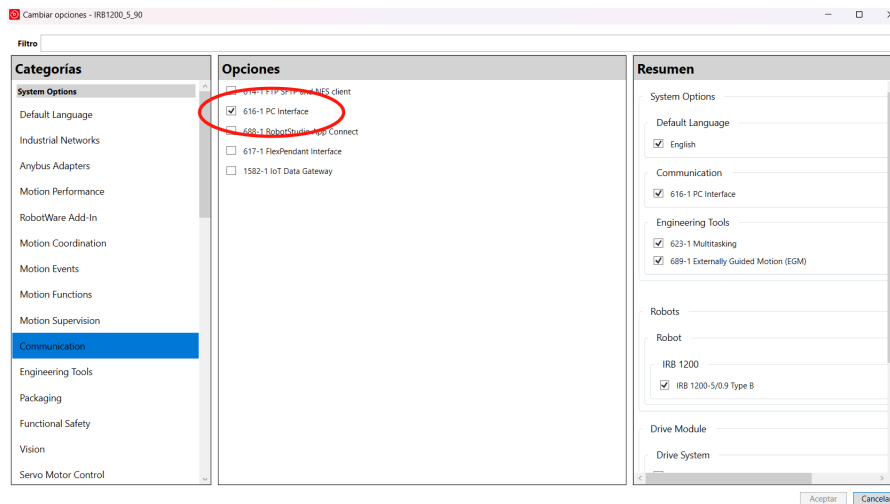


Fig. 5.4. Configuración del entorno robótico 4.1

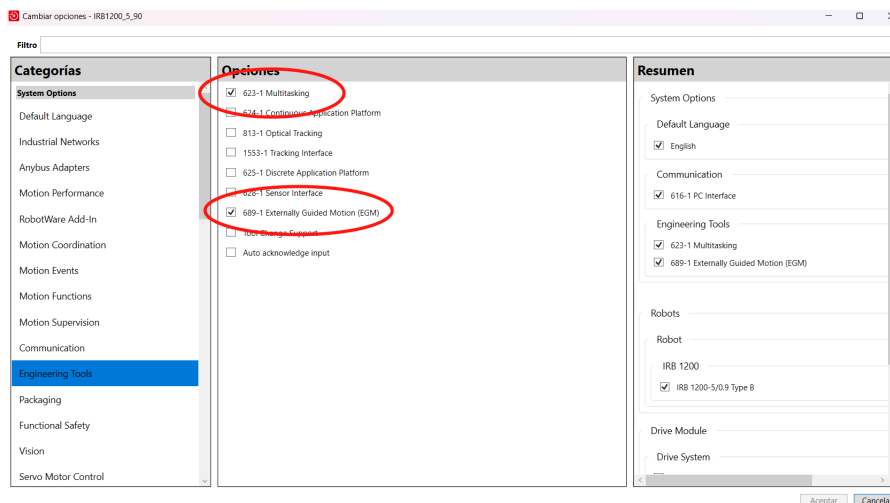


Fig. 5.5. Configuración del entorno robótico 4.2

Una vez configurado el entorno robótico se deberán descargar e instalar los archivos correspondientes. En el mismo lugar del controlador virtual en su interior debe de aparecer la carpeta 'HOME', en ella habría que usar el click derecho y seleccionar 'Abrir Carpeta'(Open Folder) que abrirá su ubicación en el ordenador. En su interior se tendrán que copiar los archivos '[error_reporter.mod]', '[motion_program_logger.mod]', '[motion_program_exec.mod]' y '[motion_program_exec_egm.mod]', asegurándose posteriormente de que todos los archivos han sido añadidos a la carpeta.

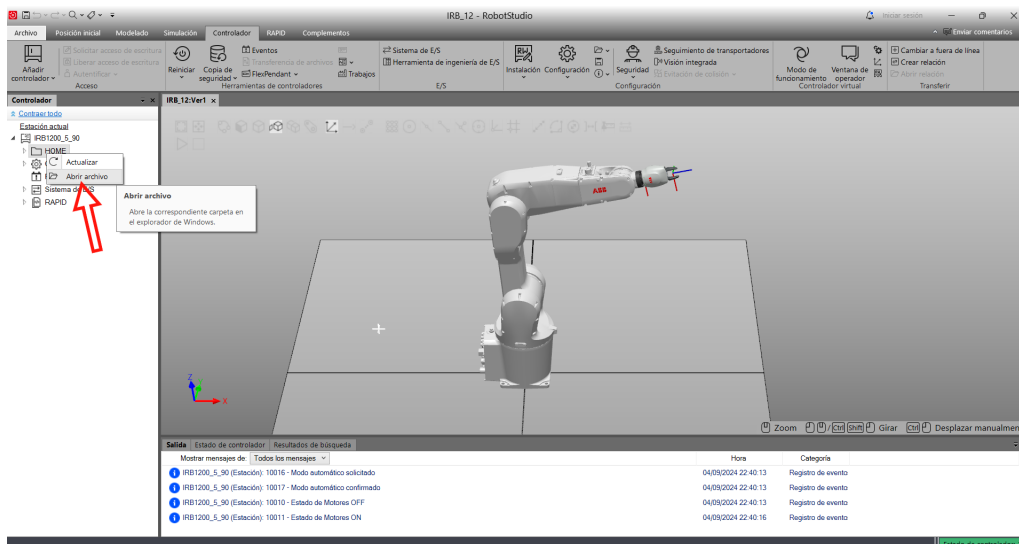


Fig. 5.6. Configuración del entorno robótico 5

De nuevo bajo el controlador virtual, se debe de hacer click derecho sobre 'Configuración' (Configuration) y seleccionar 'Cargar Parámetros' (Load Parameters). Se deberán subir uno a uno los archivos de '[SYS.cfg]', '[EIO.cfg]' y '[MOC.cfg]' asegurándose de que la opción 'Cargar parámetros y reemplazar duplicados' (Load parameters and replace duplicates) se encuentre seleccionado. Entonces una vez seguidos estos pasos se deberá reiniciar el controlador, preferiblemente con 'Reset RAPID (P-Start)'.

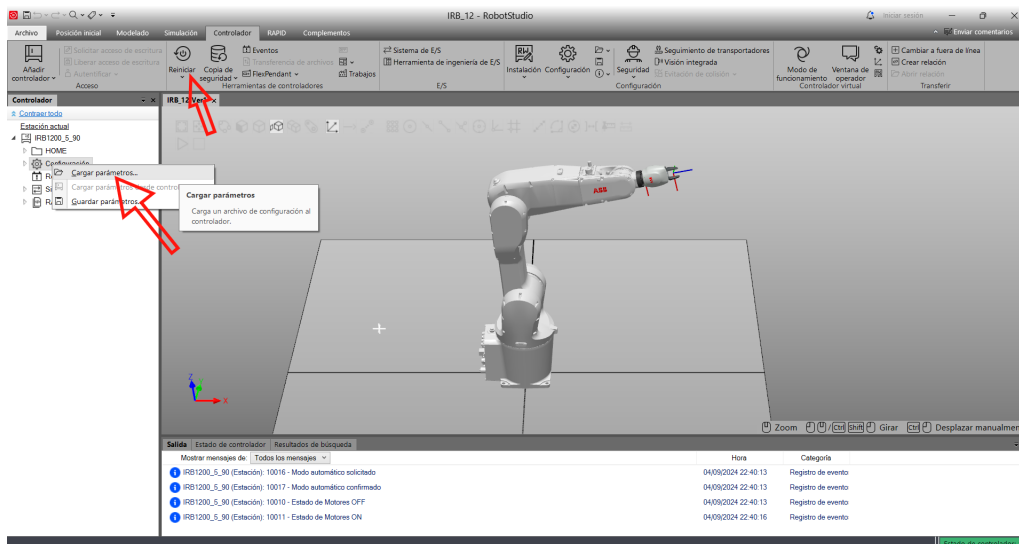


Fig. 5.7. Configuración del entorno robótico 7

Una vez hecho esto la estación robótica estará lista para funcionar y comunicarse por medio de EGM. En cualquier caso se recomienda entrar en las tablas de 'Configuración' para comprobar que las activaciones de envío de señales está configurado de forma adecuada.

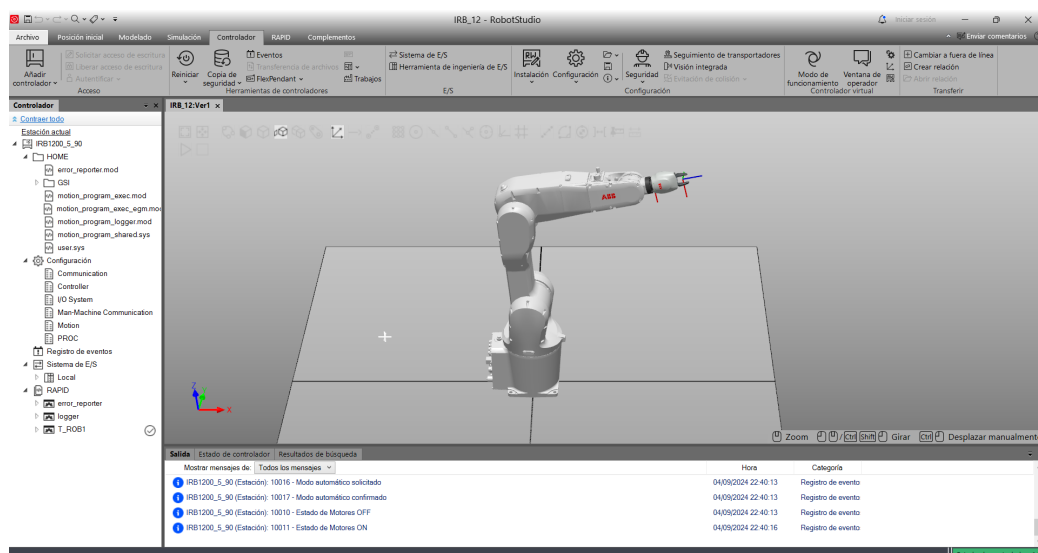


Fig. 5.8. Estado del controlador robótico tras la configuración

Si se han seguido los pasos anteriores correctamente ahora solo falta ejecutar las aplicaciones de Python mientras el software de la estación de trabajo se encuentre activo y con el controlador virtual completamente cargado. Para la puesta en marcha del sistema se debe de iniciar primero el servidor de comunicación del controlador seleccionado, apareciendo en la consola dos mensajes de espera de conexión, uno para el display y otro para la conexión EGM (no es necesaria la conexión del display para el funcionamiento del sistema pero se recomienda para un soporte visual), después se debe de iniciar el cliente EGM correspondiente al controlador empleado. Al inicializarlo el brazo robótico debería de situarse en una posición predeterminada, indicando que el la conexión EGM se ha realizado correctamente y el sistema se encuentra listo para ser teleoperado.

Durante la operación deben de tenerse en cuenta dos factores de seguridad fundamentales. Si los movimientos comandados al robot suponen una excesiva carga dinámica, aun siendo en una simulación virtual, la interfaz dejará de operar llevando al reinicio del entorno robótico y las aplicaciones. Esto mismo pasa si se llevan a las articulaciones de la unidad mecánica a un punto de singularidad.

5.2. Producto del proyecto

Se ha conseguido operar el brazo robótico por medio del mando xbox, lo que ello supone que se a alcanzado a transmitir los datos correctamente por protocolo TCP a las aplicaciones de display y cliente EGM sin perdidas de datos aparente, estableciendo simultáneamente dos ejecuciones servidor-cliente sin que estas interfieran entre sí. También se ha llegado a la operación exitosa por EGM a través de estos módulos, y se ha sido capaz de recurrir al uso del movimiento articular y cartesiano del robot durante la misma sesión, gracias a la versatilidad de sus elementos de acción.

Una de las mayores limitaciones de este apartado, refiriéndose al lector del controlador de xbox, es su estructuración basada en una API exclusiva de Windows (XInput) restringiendo el uso de esta aplicación en concreto a dispositivos que funcionen bajo este sistema operativo.

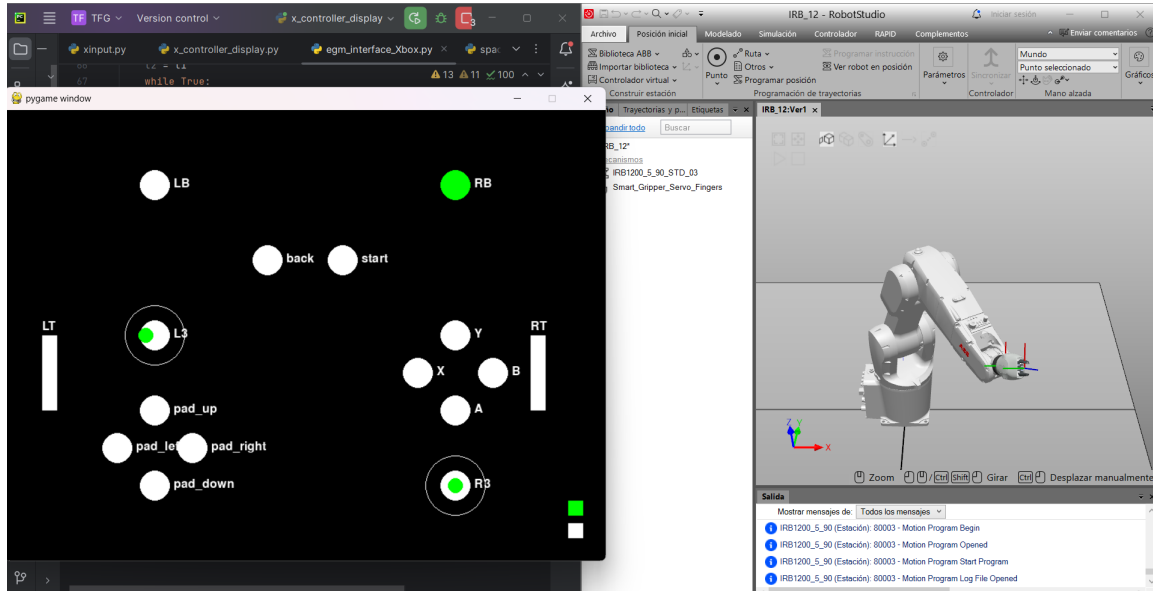


Fig. 5.9. Operación del brazo IRB_1200 con el dispositivo Xbox

En el caso de spacenavigator también ha sido posible establecer robustas comunicaciones de servidor-cliente simultaneas sin ninguna perdida de datos significativa. En el método de teleoperación, de igual modo, se ha logrado operar con los dos comandos de movimiento disponibles pero a causa del diseño del controlador se ha limitado a emplear solo un método por sesión.

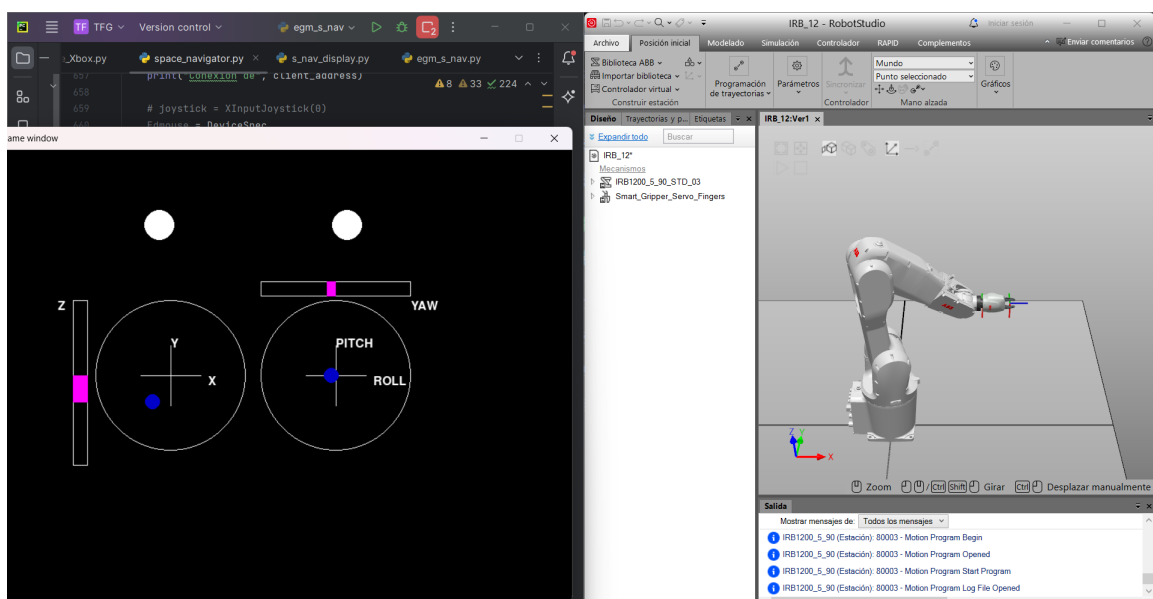


Fig. 5.10. Operación del brazo IRB_1200 con el dispositivo spacenavigator

Durante el desarrollo del proyecto y uso de ambos controladores se ha podido observar cierta diferencia en el desempeño de la teleoperación con cada uno de ellos. Aunque que el Spacenavigator resulta mucho más inmersivo en el control del brazo por medio de movimientos cartesianos, dispone de una gran limitación a la hora de implementar otros comandos debido a su falta de elementos de control, haciendo poco viable implementar la función de cambio de operación. Por otro lado, el controlador de xbox, resulta un elemento mucho más versátil a la hora de programar comandos o atajos para modificar ciertos parámetros durante la sesión; sin embargo el considerable número de botones, palancas y gatillos hacen necesario alguna clase de guía para que el operario sea capaz de orientarse con la funcionalidad de cada uno.

Para más información sobre la guía de operación referente a este sistema consultar el anexo (A.1)

Tanto los archivos concretos que deben de ser descargados para la configuración del entorno robótico, como las aplicaciones .py de la interfaz han sido recopilados en el siguiente repositorio: <https://github.com/jbs-515/teleoperation-ABB-interface>

6. CONCLUSIONES

A partir de la realización del proyecto y las pruebas que se han llevado a cabo, se han podido determinar diversas conclusiones acorde a sus ventajas e inconvenientes y respecto a las ampliaciones del proyecto y su modularidad.

Por medio del estudio y la realización de pruebas sobre los modelos de teleoperación diseñados, se refleja el éxito a la hora de integrar tecnologías modulares a un sistema de control remoto. Se ha establecido un sistema de operación a tiempo real gracias al desarrollo de aplicaciones de lectura para los controladores, mediante los cuales se envían las ordenes a través del sistema interactuando con las diversas etapas. A su vez, se han implementado diversos protocolos de comunicación en paralelo, haciendo posible la operación remota de un elemento robótico por medio de una conexión de protocolo EGM. Además, para proveer una mejor visualización de las comunicaciones entre los sistemas, se han integrado interfaces multimodales a través de GUIs, permitiendo ejercer un control más exhaustivo sobre los datos enviados. Por último, al final de cada sesión, el entorno robótico es capaz de ofrecer al entorno local información de retroalimentación por medio de un informe que representa los movimientos realizados durante la sesión de la aplicación, con el fin de documentarla en caso de diagnóstico o reconocimiento de irregularidades.

A pesar de haber logrado alcanzar los objetivos a los que se apuntaba con este proyecto, durante el desarrollo se han revelado la existencia de múltiples márgenes de mejora. Entre ellos la operación de brazos robóticos industriales presenta un grave problema a la hora de lidiar con los espacios de singularidad, provocando que el robot entre en descontrol, o provocando que, por cuestiones de seguridad, el controlador robótico termine la operación bloqueando los motores de la unidad mecánica o llevándola. Sumado a esto, el uso de conexiones EGM para guiar externamente a una unidad robótica supone la desactivación de todos los protocolos de seguridad asociados a esta si es que tuviese. Esto vuelve extremadamente peligrosa la intervención humana en el entorno teleoperado durante el uso de las aplicaciones diseñadas, requiriendo que todas las sesiones se realicen con las restricciones de espacio necesarias. Ante este inconveniente la solución planteada es la implementación de estrategias pasivas de seguridad dentro de la aplicación haciendo de esta manera que los protocolos de operación resulten no dañinos tanto para el operador como para el entorno, pudiendo convertir el entorno en el que se desenvuelve la operación en un espacio colaborativo con el trabajador humano.

Dentro de los márgenes de mejora surgen planteamientos de ampliación que son considerados a partir del futuro desarrollo del proyecto logrado, expandiendo su alcance.

Uno de los primeros planes de mejora que surgen es el de la ampliación en el repertorio de controladores implementados en la estructura del sistema, ofreciendo otras perspectivas de uso en la teleoperación o cumpliendo con las necesidades de cualquier persona sea cual sea su situación. Facilitando el acceso a estas tecnologías a multitud de personas en

multitud de situaciones. Entre ellos está la aplicación de controladores capaces de generar señales destinadas al usuario dándole contexto de los eventos sucedidos mientras la sesión se encuentra en marcha, generando de este modo una interfaz háptica, ofreciendo un mayor control sobre el entorno remoto al recibir retroalimentación sensorial y acentuando la telepresencia del operador, mejorando así la transparencia del sistema.

En los planteamientos iniciales se discutió el uso de un brazalete Myo de Mindrove, con el cual se habrían explorado otros marcos de aplicación para sistemas teleoperados, inclusive su utilidad dentro del campo biónico, concretamente para el uso de dispositivos protésicos. Tratándose de un controlador singular, pues opera por medio de la lectura de señales electromiográficas del cuerpo humano, volviendo realmente interesante el estudio y método de sus aplicaciones como controlador y reconocedor de gestos. Sin embargo su operabilidad se vio comprometida en un momento previo al desarrollo del proyecto.

También es necesario plantear en uso de otros protocolos de comunicación a parte del EGM. Este método de conexión solo está disponible en los artículos y elementos de software de ABB, y a pesar de resultar útil y eficiente a la hora de operar una unidad mecánica a tiempo real, su exclusividad limita enormemente el repertorio de campos de aplicación a los que se pueden extrapolar el resto de etapas y procedimientos del sistema. Alternativas útiles se presentan como ROS que, al tratarse de una plataforma de software de código abierto enfocada en la robótica, proporciona infraestructuras y bibliotecas aprovechables para la comunicación y operación de multitud de dispositivos.

Finalmente, al hablar sobre la implementación de aplicaciones modulares, estas han supuesto uno de los factores más interesantes que abordar durante el desarrollo del proyecto. El uso de sistemas con estas características abre la puerta a la posibilidad de reutilización de cualquier etapa presente en los proyectos creados, o las interfaces diseñadas a otras circunstancias y campos de aplicación. Esto favorece a la creación y preservación de una comunidad tecnológica en la que se comparten los logros y descubrimientos realizados, permitiendo que sean usados por otros y continuando la cadena del progreso. Además abre el acceso de múltiples campos y actividades tecnológicas a infinidad de personas, ajustándose a sus preferencias y/o necesidades para poder desempeñar su labor.

7. GESTIÓN DEL PROYECTO

Durante la realización del proyecto se han empleado diferentes tipos de herramientas tanto de software como de hardware los cuales han inquirido en una debida inversión económica, además de la implicación del tiempo que ha conllevado completarlo. Por lo tanto la inversión del proyecto puede resumirse en:

7.1. Presupuesto

Producto	Suministrador	Cantidad	Precio unitario	Precio total
Licencia de SW RobotStudio	ABB	1	1500	1500
Controlador Spacenavigator	3D connexion (Logitech)	1	132.05	132.05
Controlador Xbox	Nacon	1	21.99	21.99
Ordenador Portátil	Lenovo	1	629.99	629.66
Estudiante		325	15	4.875
				7158.7 €

TABLA 7.1. PRESUPUESTO DEL PROYECTO

7.2. Diagrama GANTT

	Abril	Mayo				Junio				Julio				Agosto				Septiembre	
	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2
Investigación y documentación																			
Estado del arte																			
Selección de hardware																			
Selección de software																			
Desarrollo del software	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Códigos de controladores																			
Códigos de display																			
Código de conexión EGM																			
Código del entorno robótico																			
Pruebas y arreglar errores																			
Escritura de la memoria																			

TABLA 7.2. DIAGRAMA GANTT

BIBLIOGRAFÍA

- [1] T. B. Sheridan, *Telerobotics, Automation and Human Supervisory Control*. Londres, Inglaterra: MIT Press, 1992.
- [2] D. M. y D. M. J. Harris, *Robotics: An Introduction*. Nueva York, NY, Estados Unidos de América: Springer Science+Business Media, 1986.
- [3] B. Siciliano, “Springer handbook of robotics,” *Springer-Verlag google schola*, vol. 2, pp. -, 2008.
- [4] L. Basañez y R. Suárez, “Teleoperation,” en *Springer Handbook of Automation*, Springer, 2009, pp. 449-468.
- [5] P. F. Hokayem, “Bilateral Teleoperation: An historical survey,” *Automa*, vol. 42, n.o 12, pp. 2035-2057, Dic 2006.
- [6] E. N. Ortega, “Teleoperación: técnicas, aplicaciones, entorno sensorial y teleoperación inteligente,” Tesis de mtría., UPC, 2004.
- [7] Telekino. [En línea]. Disponible en: <https://www.torresquevedo.org/LTQ10/index.php?title=Telecontrol>.
- [8] JJVelasco, “El Telekino, el primer control remoto de la historia,” *Hipertextual*, 2011. [En línea]. Disponible en: <https://hipertextual.com/2011/07/el-telekino-el-primer-control-remoto-de-la-historia>.
- [9] A. C. Correa, “SISTEMAS ROBOTICOS TELEOPERADOS,” Tesis doct., Universidad Militar Nueva Granada, Colombia, 2005.
- [10] EMpresa de robótica KUKA. [En línea]. Disponible en: <https://www.kuka.com/es-es/empresa/iimagazine/2023/12/homeofrobotik-robotikgeschichte>.
- [11] T. B. Sheridan, “Human–Robot Interaction: Status and Challenges,” *Human Factors*, vol. 58, n.º 4, pp. 525-532, 2016, PMID: 27098262. doi: 10.1177/0018720816644364. [En línea]. Disponible en: <https://doi.org/10.1177/0018720816644364>.
- [12] UNDERSEA United States Naval Museum. [En línea]. Disponible en: <https://navalunderseamuseum.org/curv-iii/>.
- [13] T. Fong, “Vehicle Teleoperation Interfaces. Autonomous Robots,” *11*, 9–18, (2001).
- [14] J. G. Jiménez, “Asistencia a la teleoperación en tareas de búsqueda y rescate mediante un robot hexápodo con navegación basada en odometría visua,” Tesis de mtría., UPM, 2018.
- [15] Reportaje sobre la implementación de sistema quirurgico Da Vinci. [En línea]. Disponible en: <http://www.icirugiarobotica.com/cirugia-robotica-da-vinci/>.

- [16] I. S. Navarro, “Prótesis biónicas, biología y tecnología,” *Panorama Actual del Medicamento*, vol. 42, n.o 411, pp. 256-259, 2018.
- [17] J. Cui, S. Tosunoglu, R. Roberts, C. Moore y D. W. Repperger, “A review of teleoperation system control,” en *Proceedings of the Florida conference on recent advances in robotics*, Citeseer, 2003, pp. 1-12.
- [18] C. M. Köllöd, N. J. Eftimiu, G. Márton e I. Ulbert, “Classification of semi-automated labeled MindRove armband recorded EMG data,” en *2022 IEEE 22nd International Symposium on Computational Intelligence and Informatics and 8th IEEE International Conference on Recent Achievements in Mechatronics, Automation, Computer Science and Robotics (CINTI-MACRo)*, IEEE, 2022, pp. 000 381-000 386.
- [19] S. E. Salcudean, “Control for teleoperation and haptic interfaces,” en *Control Problems in Robotics and Automation*, B. Siciliano y K. P. Valavanis, eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 51-66.
- [20] J. Montero Mata, *Diseño e implementación de un sistema de control por voz*, 2008.
- [21] R. Kress, J. Jansen y W. Hamel, “Modular robotics overview of the ‘state of the art’,” (ORNL/TM-12791). *United States*, 1996.
- [22] N. Ciencia, “Robots médicos: más precisión, menos errores, pero siempre vigilados por humanos,” *NOVACIENCIA*, Nov 11, 2021. [En línea]. Disponible en: <https://novaciencia.es/robots-medicos-uma/>.
- [23] *Esquema de botones, controlador xbox*. [En línea]. Disponible en: <https://gist.github.com/palmerj/586375bcc5bc83ccdaf00c6f5f863e86>.
- [24] *Manual Spacemavigator*. [En línea]. Disponible en: https://3dconnexion.com/manuals/spacemouse-compact/es/Manual_3Dconnexion-SpaceMouse-Compact_ES.pdf.
- [25] *Mindrove página oficial*. [En línea]. Disponible en: <https://mindrove.com/armband/>.
- [26] ABB Robot Studio page. [En línea]. Disponible en: <https://new.abb.com/products/robotics/es/robotstudio>.
- [27] ABB Application Manual: External Guided Motion. [En línea]. Disponible en: <https://search.abb.com/library/Download.aspx?DocumentID=3HAC073319-001&LanguageCode=en&DocumentPartId=&Action=Launch>.
- [28] *Documentación IRB₁200*. [En línea]. Disponible en: <https://search.abb.com/library/Download.aspx?DocumentID=9AKK106103A6066&LanguageCode=en&DocumentPartId=&Action=Launch>.
- [29] *Presentación ABB robot IRB₁200*. [En línea]. Disponible en: <https://search.abb.com/library/Download.aspx?DocumentID=9AKK106103A6065&LanguageCode=en&DocumentPartId=&Action=Launch>.
- [30] R. González Duque, *Python para todos*, 2011.

- [31] *pygame documentation*. [En línea]. Disponible en: <https://pygame.readthedocs.io/en/latest/index.html>.
- [32] *pygame página oficial*. [En línea]. Disponible en: <https://www.pygame.org/news>.
- [33] *socket communication documentation*. [En línea]. Disponible en: <https://docs.python.org/3/library/socket.html>.
- [34] *abb-robot-client documentation*. [En línea]. Disponible en: <https://abb-robot-client.readthedocs.io/en/latest/>.
- [35] *abb-motion-program documentation*. [En línea]. Disponible en: <https://abb-motion-program-exec.readthedocs.io/en/latest/>.
- [36] Repositorio de github, usuario 'r4dian'. [En línea]. Disponible en: <https://github.com/r4dian/Xbox-Controller-for-Python/tree/master>.
- [37] Introducción a XInput en aplicaciones Windows. [En línea]. Disponible en: <https://learn.microsoft.com/es-es/windows/win32/xinput/getting-started-with-xinput?redirectedfrom=MSDN#controller-layout>.
- [38] Repositorio de github, usuario 'pyspacenavigator'. [En línea]. Disponible en: <https://github.com/johnhw/pyspacenavigator/tree/master>.
- [39] Repositorio de github, usuario 'rpiRobotics'. [En línea]. Disponible en: https://github.com/rpiRobotics/abb_motion_program_exec/tree/master.

ANEXO 1: MANUAL DE LOS CONTROLADORES

Control del mando xbox

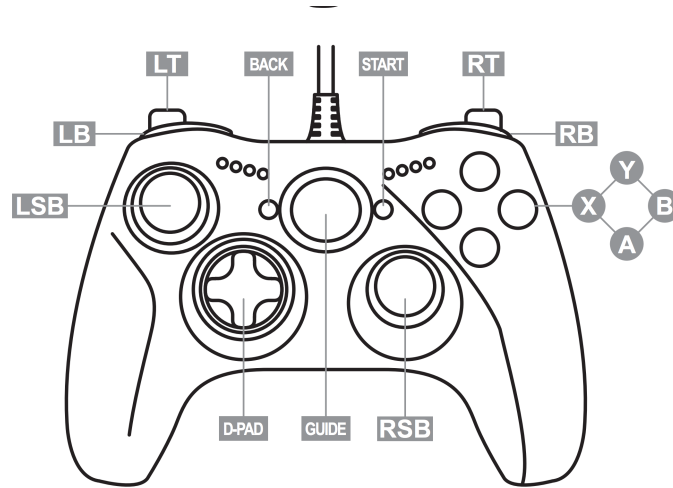


Fig. 7.1. Diagrama para el manual de control xbox

TABLA 7.3. ASIGNACIÓN DE ACCIONES PARA OPERACIÓN CON XBOX

T-O CARTESIANA		T-O ARTICULAR	
CONTROL	MOVIMIENTO	CONTROL	ARTICULACIÓN
LS_X	x-axis	LS_X	J1
LS_Y	y-axis	LS_Y	J2
RT	+z-axis	RS_Y	J3
LT	-z-axis	RS_X	J4
RS_X	q1	D-PAD_UP	+J5
RS_Y	q2	D-PAD_DOWN	-J5
D-PAD_RIGHT	+q3	RB	+J6
D-PAD_LEFT	-q3	LB	-J6
RSB	+speed	RSB	+speed
LSB	-speed	LSB	-speed
BACK	change mode	BACK	change mode
START	stop	START	stop

*Debe de tenerse en cuenta que al cambiar de modo de operación debe de cerrarse la ventana de log antes de poder proceder al siguiente modo.

Control de spacenavigator

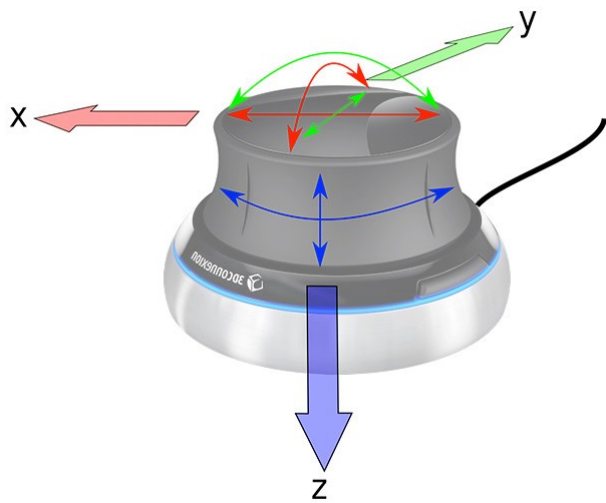


Fig. 7.2. Diagrama para el manual de control space-navigator

TABLA 7.4. ASIGNACIÓN DE ACCIONES PARA OPERACIÓN CON SPACENAVIGATOR

T-O CARTESIANA	
CONTROL	MOVIMIENTO
X	x-axis
Y	y-axis
Z	z-axis
TILT_SIDE	q1
TILT_FRONT	q2
ROTATE	q3
B_2	+speed
B_1	-speed
B_1 + B_2	stop

ANEXO 2: CÓDIGOS DE LECTURA

xinput

CÓDIGO 7.1. Código del lector de controlador Xbox

```
1  """
2  Un modulo para obtener los datos de entrada de un controlador Xbox
   por medio de
3  la libreria XInput de Windows http://msdn.microsoft.com/en-gb/
   library/windows/desktop/ee417001%28v=vs.85%29.aspx
4
5
6  Adaptado de 'Jason R. Coombs' codigo aqu :
7  http://pydoc.net/Python/jaraco.input/1.0.1/jaraco.input.win32.xinput
   /
8  under the MIT licence terms
9  Y de 'r4dian' codigo aqu :
10 https://github.com/r4dian/Xbox-Controller-for-Python/tree/master
11
12 Implementado en Python 3
13 Modificado para aadir se ales de seguimiento y disponer de
   conexi n TCP
14 para su comunicaci n con otras aplicaciones
15 Solo requiere Pyglet 1.2alpha1 o superior:
16 pip install --upgrade http://pyglet.googlecode.com/archive/tip.zip
17 """
18
19 import ctypes
20 import sys
21 import time
22 from operator import itemgetter, attrgetter
23 from itertools import count, starmap
24 from pyglet import event
25 import json
26 import socket
27 import threading
28
29
30 # biblioteca de eventos
31 class XINPUT_GAMEPAD(ctypes.Structure):
32     _fields_ = [
33         ('buttons', ctypes.c_ushort), # wButtons
34         ('left_trigger', ctypes.c_ubyte), # bLeftTrigger
35         ('right_trigger', ctypes.c_ubyte), # bLeftTrigger
36         ('l_thumb_x', ctypes.c_short), # sThumbLX
37         ('l_thumb_y', ctypes.c_short), # sThumbLY
```

```

38         ('r_thumb_x', ctypes.c_short), # sThumbRx
39         ('r_thumb_y', ctypes.c_short), # sThumbRy
40     ]
41
42
43 class XINPUT_STATE(ctypes.Structure):
44     _fields_ = [
45         ('packet_number', ctypes.c_ulong), # dwPacketNumber
46         ('gamepad', XINPUT_GAMEPAD), # Gamepad
47     ]
48
49
50 class XINPUT_VIBRATION(ctypes.Structure):
51     _fields_ = [("wLeftMotorSpeed", ctypes.c_ushort),
52                 ("wRightMotorSpeed", ctypes.c_ushort)]
53
54 class XINPUT_BATTERY_INFORMATION(ctypes.Structure):
55     _fields_ = [("BatteryType", ctypes.c_ubyte),
56                 ("BatteryLevel", ctypes.c_ubyte)]
57
58
59 xinput = ctypes.windll.xinput1_4
60 # xinput = ctypes.windll.xinput9_1_0 # this is the Win 8 version ?
61 # xinput1_2, xinput1_1 (32-bit Vista SP1)
62 # xinput1_3 (64-bit Vista SP1)
63
64
65 def struct_dict(struct):
66     """
67     take a ctypes.Structure and return its field/value pairs
68     as a dict.
69
70     >>> 'buttons' in struct_dict(XINPUT_GAMEPAD)
71     True
72     >>> struct_dict(XINPUT_GAMEPAD)['buttons'].__class__.__name__
73     'CField'
74     """
75     get_pair = lambda field_type: (
76         field_type[0], getattr(struct, field_type[0]))
77     return dict(list(map(get_pair, struct._fields_)))
78
79
80 def get_bit_values(number, size=32):
81     """
82     (Obtiene los valores de bit como una lista para un numero dado)
83     Get bit values as a list for a given number
84
85     >>> get_bit_values(1) == [0]*31 + [1]
86     True
87
88     >>> get_bit_values(0xDEADBEEF)

```

```

89     [1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1,
90         1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1]
91
92     (Se puede sobrepasar el tamaño estandar de 32-bits para
93     ajustarse a
94     tu aplicación)
95     You may override the default word size of 32-bits to match your
96     actual
97     application.
98     >>> get_bit_values(0x3, 2)
99     [1, 1]
100
101     >>> get_bit_values(0x3, 4)
102     [0, 0, 1, 1]
103     """
104     res = list(gen_bit_values(number))
105     res.reverse()
106     # 0-pad the MSB
107     res = [0] * (size - len(res)) + res
108     return res
109
110 def gen_bit_values(number):
111     """
112     Devuelve cero o uno por cada bit de un valor numerico hasta el
113     MSB, empezando por el LSB.
114     Return a zero or one for each bit of a numeric value up to the
115     most
116     significant 1 bit, beginning with the least significant bit.
117     """
118     number = int(number)
119     while number:
120         yield number & 0x1
121         number >>= 1
122
123 ERROR_DEVICE_NOT_CONNECTED = 1167
124 ERROR_SUCCESS = 0
125
126 class XInputJoystick(event.EventDispatcher):
127     """
128     XInputJoystick
129
130     Un gestor de estados, usa el modelo de eventos pyglet, conecta
131     con un dispositivo
132     y analiza los eventos cuando el estado cambia.
133     .
134
135     Example:

```



```

134     controller_one = XInputJoystick(0)
135     """
136     max_devices = 4
137
138     def __init__(self, device_number, normalize_axes=True):
139         values = vars()
140         del values['self']
141         self.__dict__.update(values)
142
143         super(XInputJoystick, self).__init__()
144
145         self._last_state = self.get_state()
146         self.received_packets = 0
147         self.missed_packets = 0
148
149         # Establece el m todo que se emplear para la
150         # normalizaci n.
151         choices = [self.translate_identity, self.
152                   translate_using_data_size]
153         self.translate = choices[normalize_axes]
154         # //////////////////////////////////
155         self.cartesian_mode = True
156
157     def translate_using_data_size(self, value, data_size):
158         # normaliza los datos ana gicos en un rango de [0, 1] para
159         # los rangos estrictamente positivos
160         # y [-0.5,0.5] para los datos con simbolo
161         data_bits = 8 * data_size
162         return float(value) / (2 ** data_bits - 1)
163
164     def translate_identity(self, value, data_size=None):
165         return value
166
167     def get_state(self):
168         "Obtiene el estado del controlador representado por este
169         objeto"
170         state = XINPUT_STATE()
171         res = xinput.XInputGetState(self.device_number, ctypes.byref
172                                     (state))
173         if res == ERROR_SUCCESS:
174             return state
175         if res != ERROR_DEVICE_NOT_CONNECTED:
176             raise RuntimeError(
177                 "Unknown error %d attempting to get state of device
178                 %d" % (res, self.device_number))
179         # else return None (no hay dispositivo conectado)
180
181     def is_connected(self):
182         return self._last_state is not None
183
184     @staticmethod

```

```

179 def enumerate_devices():
180     "Devuelve el dato de # de dispositivos conectados"
181     devices = list(
182         map(XInputJoystick, list(range(XInputJoystick.
183             max_devices))))
184     return [d for d in devices if d.is_connected()]
185
186 def set_vibration(self, left_motor, right_motor):
187     "Controla la velocidad de ambos motores independientemente"
188     # Set up function argument types and return type
189     XInputSetState = xinput.XInputSetState
190     XInputSetState.argtypes = [ctypes.c_uint, ctypes.POINTER(
191         XINPUT_VIBRATION)]
192     XInputSetState.restype = ctypes.c_uint
193
194     vibration = XINPUT_VIBRATION(
195         int(left_motor * 65535), int(right_motor * 65535))
196     XInputSetState(self.device_number, ctypes.byref(vibration))
197
198 def get_battery_information(self):
199     "Detecta el tipo de bater a y el nivel de carga"
200     BATTERY_DEVTYPE_GAMEPAD = 0x00
201     BATTERY_DEVTYPE_HEADSET = 0x01
202     XInputGetBatteryInformation = xinput.
203         XInputGetBatteryInformation
204     XInputGetBatteryInformation.argtypes = [ctypes.c_uint,
205         ctypes.c_ubyte, ctypes.POINTER(
206             XINPUT_BATTERY_INFORMATION)]
207     XInputGetBatteryInformation.restype = ctypes.c_uint
208
209     battery = XINPUT_BATTERY_INFORMATION(0,0)
210     XInputGetBatteryInformation(self.device_number,
211         BATTERY_DEVTYPE_GAMEPAD, ctypes.byref(battery))
212
213     #define BATTERY_TYPE_DISCONNECTED      0x00
214     #define BATTERY_TYPE_WIRED            0x01
215     #define BATTERY_TYPE_ALKALINE         0x02
216     #define BATTERY_TYPE_NIMH            0x03
217     #define BATTERY_TYPE_UNKNOWN          0xFF
218     #define BATTERY_LEVEL_EMPTY           0x00
219     #define BATTERY_LEVEL_LOW             0x01
220     #define BATTERY_LEVEL_MEDIUM          0x02
221     #define BATTERY_LEVEL_FULL            0x03
222     batt_type = "Unknown" if battery.BatteryType == 0xFF else [
223         "Disconnected", "Wired", "Alkaline", "Nimh"][battery.
224         BatteryType]
225     level = ["Empty", "Low", "Medium", "Full"][battery.
226         BatteryLevel]
227     return batt_type, level
228
229 def dispatch_events(self):

```

```

221         "Bucle principal de eventos de joystick"
222         state = self.get_state()
223         if not state:
224             raise RuntimeError(
225                 "Joystick %d is not connected" % self.device_number)
226         if state.packet_number != self._last_state.packet_number:
227             # si el estado cambia lo maneja
228             self.update_packet_count(state)
229             self.handle_changed_state(state)
230         self._last_state = state
231
232     def update_packet_count(self, state):
233         "Hace seguimiento de los paquetes recibidos y perdidos (
234             modificaci n del rendimiento)"
235         self.received_packets += 1
236         missed_packets = state.packet_number - \
237             self._last_state.packet_number - 1
238         if missed_packets:
239             self.dispatch_event('on_missed_packet', missed_packets)
240         self.missed_packets += missed_packets
241
242     def handle_changed_state(self, state):
243         "Maneja varios eventos resultantes de cambios de estados"
244         self.dispatch_event('on_state_changed', state)
245         self.dispatch_axis_events(state)
246         self.dispatch_button_events(state)
247
248     def dispatch_axis_events(self, state):
249         # axis fields se refieren a tod0 menos los botones
250         axis_fields = dict(XINPUT_GAMEPAD._fields_)
251         axis_fields.pop('buttons')
252         for axis, type in list(axis_fields.items()):
253             old_val = getattr(self._last_state.gamepad, axis)
254             new_val = getattr(state.gamepad, axis)
255             data_size = ctypes.sizeof(type)
256             old_val = self.translate(old_val, data_size)
257             new_val = self.translate(new_val, data_size)
258
259             # establece zonas muertas: minimas
260             # ultimos ajustes probados 18/08/24
261             if ((old_val != new_val and (new_val >
262                 0.080000000000000000 or new_val <
263                 -0.080000000000000000) and abs(old_val - new_val) >
264                 0.000000000500000000) or
265                 (axis == 'right_trigger' or axis == 'left_trigger')
266                 and new_val == 0 and abs(old_val - new_val) >
267                 0.000000000500000000):
268                 self.dispatch_event('on_axis', axis, new_val)
269
270     def dispatch_button_events(self, state):

```

```

265         changed = state.gamepad.buttons ^ self._last_state.gamepad.
            buttons
266         changed = get_bit_values(changed, 16)
267         buttons_state = get_bit_values(state.gamepad.buttons, 16)
268         changed.reverse()
269         buttons_state.reverse()
270         button_numbers = count(1)
271         changed_buttons = list(
272             filter(itemgetter(0), list(zip(changed, button_numbers,
                buttons_state))))
273         tuple(starmap(self.dispatch_button_event, changed_buttons))
274
275
276     def dispatch_button_event(self, changed, number, pressed):
277         self.dispatch_event('on_button', number, pressed)
278         ''
279         # ////////////////////////////////// (este parece que funciona mejor
            8/8/24)
280         if number == 6 and pressed:
281             self.cartesian_mode = not self.cartesian_mode
282             print(f"Modo cambiado a {'cartesiano' if self.
                cartesian_mode else 'polar'}")
283         ''
284
285     # stub para manejar los eventos
286     def on_state_changed(self, state):
287         pass
288
289     def on_axis(self, axis, value):
290         pass
291
292     def on_button(self, button, pressed):
293         pass
294
295     def on_missed_packet(self, number):
296         pass
297
298
299     list(map(XInputJoystick.register_event_type, [
300         'on_state_changed',
301         'on_axis',
302         'on_button',
303         'on_missed_packet',
304     ]))
305
306
307     def determine_optimal_sample_rate(joystick=None):
308         """
309         Acciona lentamente la palanca, monitoriza el flujo de paquetes
            para
310         observar si hay perdida de paquetes, indicando que el ratio de

```

```

311     muestreo es demasiado lento para los paquetes perdidos. Esto
        supone
312     informaci n perdida para actualizar el estado del joystick.
313     Al detectar perdidas de paquetes, incrementa el ratio de
        muestreo
314     hasta que se haya llegado al estado deseado.
315     """
316     # recomendado probar entre 200-2000Hz
317     if joystick is None:
318         joystick = XInputJoystick.enumerate_devices()[0]
319
320     j = joystick
321
322     print("Move the joystick or generate button events
        characteristic of your app")
323     print("Hit Ctrl-C or press button 6 (<, Back) to quit.")
324
325
326     # epezar a 1Hz y aumentar el ratio hasta que los mensajes
        perdidos son eliminados
327     j.probe_frequency = 1 # Hz
328     j.quit = False
329     j.target_reliability = .99 # perder 1 mensaje de 100 es
        aceptable
330
331     @j.event
332     def on_button(button, pressed):
333         # flag the process to quit if the < button ('back') is
            pressed.
334         j.quit = (button == 6 and pressed)
335
336     @j.event
337     def on_missed_packet(number):
338         print('missed %(number)d packets' % vars())
339         total = j.received_packets + j.missed_packets
340         reliability = j.received_packets / float(total)
341         if reliability < j.target_reliability:
342             j.missed_packets = j.received_packets = 0
343             j.probe_frequency *= 1.5
344
345     while not j.quit:
346         j.dispatch_events()
347         time.sleep(1.0 / j.probe_frequency)
348     print("final probe frequency was %s Hz" % j.probe_frequency)
349
350
351     def sample_first_joystick():
352         """
353         Utiliza el primer mando disponible, mostrando los cambios
354         que se realicen en 1 en la consola. Los triggers LT y RT
355         activan los motores de vibraci n

```

```

356     """
357     joysticks = XInputJoystick.enumerate_devices()
358     device_numbers = list(map(attrgetter('device_number'), joysticks
359                                     ))
360
361     print('found %d devices: %s' % (len(joysticks), device_numbers))
362
363     if not joysticks:
364         sys.exit(0)
365
366     j = joysticks[0]
367     print('using %d' % j.device_number)
368
369     battery = j.get_battery_information()
370     print(battery)
371
372     @j.event
373     def on_button(button, pressed):
374         print('button', button, pressed)
375
376     left_speed = 0
377     right_speed = 0
378
379     @j.event
380     def on_axis(axis, value):
381         left_speed = 0
382         right_speed = 0
383
384         print('axis', axis, value)
385         if axis == "left_trigger":
386             left_speed = value
387         elif axis == "right_trigger":
388             right_speed = value
389         j.set_vibration(left_speed, right_speed)
390
391     while True:
392         j.dispatch_events()
393         time.sleep(.01)
394
395     # //////////////////////////////////////
396     def run_xinput_server(address='localhost', port=5000):
397         server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
398                                     )
399         server_socket.bind((address, port))
400         server_socket.listen(1)
401         print("Esperando connexion")
402         connection, client_address = server_socket.accept()
403         print("Conexion de", client_address)
404
405         joystick = XInputJoystick(0)

```

```

405
406     try:
407         while True:
408             joystick.dispatch_events() # para leer a tiempo real
409                                     # los eventos internos del joystick
410             # (como el cartesian_mode)
411             state = joystick.get_state()
412             if state:
413                 data = {
414                     'buttons': state.gamepad.buttons,
415                     'left_trigger': state.gamepad.left_trigger /
416                                     255.0,
417                     'right_trigger': state.gamepad.right_trigger /
418                                     255.0,
419                     'l_thumb_x': state.gamepad.l_thumb_x / 32767.0,
420                     'l_thumb_y': state.gamepad.l_thumb_y / 32767.0,
421                     'r_thumb_x': state.gamepad.r_thumb_x / 32767.0,
422                     'r_thumb_y': state.gamepad.r_thumb_y / 32767.0,
423                     'cartesian_mode': joystick.cartesian_mode
424                 }
425                 message = json.dumps(data) + '\n'
426                 connection.sendall(message.encode('utf-8'))
427                 # # connection.sendall(json.dumps(data).encode('utf
428                 # -8'))
429                 # connection.sendall(data.encode('utf-8'))
430                 time.sleep(0.01) # limita frec de envio a 100Hz
431             finally:
432                 connection.close()
433                 server_socket.close()
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

449 def handle_client(connection):
450     joystick = XInputJoystick(0)
451
452     try:
453         while True:
454             joystick.dispatch_events() # para leer a tiempo real
455                                     # los eventos internos del joystick
456             # (como el cartesian_mode)
457             state = joystick.get_state()
458             if state:
459                 data = {
460                     'buttons': state.gamepad.buttons,
461                     'left_trigger': state.gamepad.left_trigger /
462                                     255.0,
463                     'right_trigger': state.gamepad.right_trigger /
464                                     255.0,
465                     'l_thumb_x': state.gamepad.l_thumb_x / 32767.0,
466                     'l_thumb_y': state.gamepad.l_thumb_y / 32767.0,
467                     'r_thumb_x': state.gamepad.r_thumb_x / 32767.0,
468                     'r_thumb_y': state.gamepad.r_thumb_y / 32767.0,
469                     'cartesian_mode': joystick.cartesian_mode
470                 }
471                 message = json.dumps(data) + '\n'
472                 connection.sendall(message.encode('utf-8'))
473                 # # connection.sendall(json.dumps(data).encode('utf-8'))
474                 # connection.sendall(data.encode('utf-8'))
475                 time.sleep(0.01) # limita frec de envio a 100Hz
476             finally:
477                 connection.close()
478                 # server_socket.close()
479
480 # //////////////////////////////////////
481
482 if __name__ == "__main__":
483
484     server_thread = threading.Thread(target=run_xinput_server)
485     ABB_thread = threading.Thread(target=run_xinput_server_to_egm,
486                                   daemon=True)
487
488     server_thread.start()
489     ABB_thread.start()
490
491     sample_first_joystick()

```

space_navigator


```

1  from time import sleep
2  import pywinusb.hid as hid
3  from collections import namedtuple
4  import timeit
5  import copy
6  from pywinusb.hid import usage_pages, helpers, winapi
7  import socket
8  import json
9  import threading
10 import time
11
12 # current version number
13 __version__ = "0.2.3"
14
15 # reloj para medir tiempos
16 high_acc_clock = timeit.default_timer
17
18 GENERIC_PAGE = 0x1
19 BUTTON_PAGE = 0x9
20 LED_PAGE = 0x8
21 MULTI_AXIS_CONTROLLER_CAP = 0x8
22
23 HID_AXIS_MAP = {
24     0x30: "x",
25     0x31: "y",
26     0x32: "z",
27     0x33: "roll",
28     0x34: "pitch",
29     0x35: "yaw",
30 }
31
32 import pprint
33
34 # el mapeado de los ejes se especifica como:
35 # [channel, byte1, byte2, scale]; scale es por lo grneral -1 o 1 y
36 # multiplica el resultado por este valor
37 # (pero el escalado per-axis tambien puede ser establecido de porma
38 # manual)
39 # byte1 y byte2 son indices hacia el HID array indicando a los dos
40 # bytes que se leen a formar el valor para este eje
41 # Para el SpaceNavigator, these are consecutive bytes following the
42 # channel number.
43 AxisSpec = namedtuple("AxisSpec", ["channel", "byte1", "byte2", "
44     scale"])
45
46 # los estados de los botones son especificados como:
47 # [channel, data byte, bit of byte, index to write to]
48 # Si un mensaje es recibido en el canal especificado, el valordel
49 # data byte es establecido en el bit array del boton
50 ButtonSpec = namedtuple("ButtonSpec", ["channel", "byte", "bit"])

```

```

45
46
47 # convertir 2 bytes 8-bit a un int con signo de 16-bit
48 def to_int16(y1, y2):
49     x = (y1) | (y2 << 8)
50     if x >= 32768:
51         x = -(65536 - x)
52     return x
53
54
55 # tuppla para los resultados de los 6GDL
56 SpaceNavigator = namedtuple(
57     "SpaceNavigator", ["t", "x", "y", "z", "roll", "pitch", "yaw", "
58         buttons"]
59 )
60
61 class ButtonState(list):
62     def __int__(self):
63         return sum((b << i) for (i, b) in enumerate(reversed(self)))
64
65
66 class DeviceSpec(object):
67     """Contiene las especificaciones de un solo dispositivo 3
68         Dconnexion"""
69
70     def __init__(
71         self, name, hid_id, led_id, mappings, button_mapping,
72         axis_scale=350.0
73     ):
74         self.name = name
75         self.hid_id = hid_id
76         self.led_id = led_id
77         self.mappings = mappings
78         self.button_mapping = button_mapping
79         self.axis_scale = axis_scale
80
81         self.led_usage = hid.get_full_usage_id(led_id[0], led_id[1])
82         # se inicializa en un vector de reposo "0" para cada estado
83         self.dict_state = {
84             "t": -1,
85             "x": 0,
86             "y": 0,
87             "z": 0,
88             "roll": 0,
89             "pitch": 0,
90             "yaw": 0,
91             "buttons": ButtonState([0] * len(self.button_mapping)),
92         }
93         self.tuple_state = SpaceNavigator(**self.dict_state)

```

```

93         # start in disconnected state
94         self.device = None
95         self.callback = None
96         self.button_callback = None
97
98     def describe_connection(self):
99         """Devuelve una cadena representativa del dispositivo,
100            incluyendo
101            el estado de conexión"""
102         if self.device == None:
103             return "%s [disconnected]" % (self.name)
104         else:
105             return "%s connected to %s %s version: %s [serial: %s]"
106                 % (
107                     self.name,
108                     self.vendor_name,
109                     self.product_name,
110                     self.version_number,
111                     self.serial_number,
112                 )
113
114     @property
115     def connected(self):
116         """Si es True el dispositivo ha sido conectado"""
117         return self.device is not None
118
119     @property
120     def state(self):
121         """Devuelve el estado actual de read ()
122
123         Devuelve: state: {t,x,y,z,pitch,yaw,roll,button} namedtuple
124                    0 nada si el dispositivo no está conectado.
125         """
126         return self.read()
127
128     def open(self):
129         """Abre una conexión con el dispositivo, si es posible"""
130         if self.device:
131             self.device.open()
132             # copy in product details
133             self.product_name = self.device.product_name
134             self.vendor_name = self.device.vendor_name
135             self.version_number = self.device.version_number
136             # no parece funcionar
137             # el número de serie es una cadena de bytes, las convertimos
138             # a hexa
139             self.serial_number = "".join(

```

```

140 def set_led(self, state):
141     """Establece el estado del LED"""
142     if self.connected:
143         reports = self.device.find_output_reports()
144         for report in reports:
145             if self.led_usage in report:
146                 report[self.led_usage] = state
147                 report.send()
148
149 def close(self):
150     """Cierra la conexi n si est  abierta"""
151     if self.connected:
152         self.device.close()
153         self.device = None
154
155 def read(self):
156     """Devuelve el estado actual de este controlador.
157
158     Devuelve:
159         state: {t,x,y,z,pitch,yaw,roll,button} namedtuple
160         0 nada si no est  abierto.
161     """
162     if self.connected:
163         return self.tuple_state
164     else:
165         return None
166
167 def process(self, data):
168     """
169     Actualiza el estado basado en los datos recibidos
170
171     Esta funcion actualiza el estado del objeto, dan do valores
172     por cada
173     eje [x,y,z,roll,pitch,yaw] en un rango de [-1.0, 1.0]
174     teoricamente-> no es cierto, parece que rangos max
175     [-1.5, 1.5]
176     El estado de la tupla es establecido solo cuando los 6GDL
177     han sido leidos correctamente
178
179     Si se proporciona un callback, es llamado con una copia del
180     estado de la tupla actual
181     Si button_callback, solo se le llama cuando existe un cambio
182     en el estado del boton con los argumentos (state,
183     button_state)
184
185     Parametros:
186         data      The data for this HID event, as returned by the
187                   HID callback
188
189     """
190     button_changed = False

```

```

183     for name, (chan, b1, b2, flip) in self.mappings.items():
184         if data[0] == chan:
185             self.dict_state[name] = (
186                 flip * to_int16(data[b1], data[b2]) / float(
187                     self.axis_scale)
188             )
189
190     for button_index, (chan, byte, bit) in enumerate(self.
191         button_mapping):
192         if data[0] == chan:
193             button_changed = True
194             # update the button vector
195             mask = 1 << bit
196             self.dict_state["buttons"][button_index] = (
197                 1 if (data[byte] & mask) != 0 else 0
198             )
199
200     self.dict_state["t"] = high_acc_clock()
201
202     # debe de recibir ambas partes del estado de los 6GDL antes
203     # de devolver el diccionario de estado
204     if len(self.dict_state) == 8:
205         self.tuple_state = SpaceNavigator(**self.dict_state)
206
207     # llama cualquier llamada relacionada
208     if self.callback:
209         self.callback(self.tuple_state)
210
211     # solo llama a la llamada de los botones si el estado de los
212     # botones ha cambiado
213     if self.button_callback and button_changed:
214         self.button_callback(self.tuple_state, self.tuple_state.
215             buttons)
216
217     #
218     //////////////////////////////////////
219
220     # Los identificadores de los dispositivos de 3Dconnection con los
221     # que tiene compatibilidad
222     # Cada identificador mapea un nombre a un objeto DeviceSpec
223     device_specs = {
224         "SpaceNavigator": DeviceSpec(
225             name="SpaceNavigator",
226             # vendor ID and product ID
227             hid_id=[0x46D, 0xC626],
228             # LED HID usage code pair
229             led_id=[0x8, 0x4B],
230             mappings={
231                 "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),

```

```

226         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
227         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
228         "pitch": AxisSpec(channel=2, byte1=1, byte2=2, scale=-1)
229     ,
230     "roll": AxisSpec(channel=2, byte1=3, byte2=4, scale=-1),
231     "yaw": AxisSpec(channel=2, byte1=5, byte2=6, scale=1),
232 },
233 button_mapping=[
234     ButtonSpec(channel=3, byte=1, bit=0),
235     ButtonSpec(channel=3, byte=1, bit=1),
236 ],
237 axis_scale=350.0,
238 ),
239 "SpaceMouse Compact": DeviceSpec(
240     name="SpaceMouse Compact",
241     # vendor ID and product ID
242     hid_id=[0x256F, 0xC635],
243     # LED HID usage code pair
244     led_id=[0x8, 0x4B],
245     mappings={
246         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
247         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
248         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
249         "pitch": AxisSpec(channel=2, byte1=1, byte2=2, scale=-1)
250     ,
251         "roll": AxisSpec(channel=2, byte1=3, byte2=4, scale=-1),
252         "yaw": AxisSpec(channel=2, byte1=5, byte2=6, scale=1),
253     },
254     button_mapping=[
255         ButtonSpec(channel=3, byte=1, bit=0),
256         ButtonSpec(channel=3, byte=1, bit=1),
257     ],
258     axis_scale=350.0,
259 ),
260 "SpaceMouse Pro Wireless": DeviceSpec(
261     name="SpaceMouse Pro Wireless",
262     # vendor ID and product ID
263     hid_id=[0x256F, 0xC632],
264     # LED HID usage code pair
265     led_id=[0x8, 0x4B],
266     mappings={
267         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
268         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
269         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
270         "pitch": AxisSpec(channel=1, byte1=7, byte2=8, scale=-1)
271     ,
272         "roll": AxisSpec(channel=1, byte1=9, byte2=10, scale=-1)
273     ,
274         "yaw": AxisSpec(channel=1, byte1=11, byte2=12, scale=1),
275     },
276     button_mapping=[

```

```

273         ButtonSpec(channel=3, byte=1, bit=0), # MENU
274         ButtonSpec(channel=3, byte=3, bit=7), # ALT
275         ButtonSpec(channel=3, byte=4, bit=1), # CTRL
276         ButtonSpec(channel=3, byte=4, bit=0), # SHIFT
277         ButtonSpec(channel=3, byte=3, bit=6), # ESC
278         ButtonSpec(channel=3, byte=2, bit=4), # 1
279         ButtonSpec(channel=3, byte=2, bit=5), # 2
280         ButtonSpec(channel=3, byte=2, bit=6), # 3
281         ButtonSpec(channel=3, byte=2, bit=7), # 4
282         ButtonSpec(channel=3, byte=2, bit=0), # ROLL CLOCKWISE
283         ButtonSpec(channel=3, byte=1, bit=2), # TOP
284         ButtonSpec(channel=3, byte=4, bit=2), # ROTATION
285         ButtonSpec(channel=3, byte=1, bit=5), # FRONT
286         ButtonSpec(channel=3, byte=1, bit=4), # REAR
287         ButtonSpec(channel=3, byte=1, bit=1),
288     ], # FIT
289     axis_scale=350.0,
290 ),
291 # identico pero con el ID 0xc631
292 "SpaceMouse Pro Wireless": DeviceSpec(
293     name="SpaceMouse Pro Wireless",
294     # vendor ID and product ID
295     hid_id=[0x256F, 0xC631],
296     # LED HID usage code pair
297     led_id=[0x8, 0x4B],
298     mappings={
299         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
300         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
301         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
302         "pitch": AxisSpec(channel=1, byte1=7, byte2=8, scale=-1)
303         ,
304         "roll": AxisSpec(channel=1, byte1=9, byte2=10, scale=-1)
305         ,
306         "yaw": AxisSpec(channel=1, byte1=11, byte2=12, scale=1),
307     },
308     button_mapping=[
309         ButtonSpec(channel=3, byte=1, bit=0), # MENU
310         ButtonSpec(channel=3, byte=3, bit=7), # ALT
311         ButtonSpec(channel=3, byte=4, bit=1), # CTRL
312         ButtonSpec(channel=3, byte=4, bit=0), # SHIFT
313         ButtonSpec(channel=3, byte=3, bit=6), # ESC
314         ButtonSpec(channel=3, byte=2, bit=4), # 1
315         ButtonSpec(channel=3, byte=2, bit=5), # 2
316         ButtonSpec(channel=3, byte=2, bit=6), # 3
317         ButtonSpec(channel=3, byte=2, bit=7), # 4
318         ButtonSpec(channel=3, byte=2, bit=0), # ROLL CLOCKWISE
319         ButtonSpec(channel=3, byte=1, bit=2), # TOP
320         ButtonSpec(channel=3, byte=4, bit=2), # ROTATION
321         ButtonSpec(channel=3, byte=1, bit=5), # FRONT
322         ButtonSpec(channel=3, byte=1, bit=4), # REAR
323         ButtonSpec(channel=3, byte=1, bit=1),

```

```

322     ], # FIT
323     axis_scale=350.0,
324 ),
325 "SpaceMouse Pro": DeviceSpec(
326     name="SpaceMouse Pro",
327     # vendor ID and product ID
328     hid_id=[0x46D, 0xC62b],
329     led_id=[0x8, 0x4B],
330     mappings={
331         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
332         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
333         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
334         "pitch": AxisSpec(channel=2, byte1=1, byte2=2, scale=-1)
335         ,
336         "roll": AxisSpec(channel=2, byte1=3, byte2=4, scale=-1),
337         "yaw": AxisSpec(channel=2, byte1=5, byte2=6, scale=1),
338     },
339     button_mapping=[
340         ButtonSpec(channel=3, byte=1, bit=0), # MENU
341         ButtonSpec(channel=3, byte=3, bit=7), # ALT
342         ButtonSpec(channel=3, byte=4, bit=1), # CTRL
343         ButtonSpec(channel=3, byte=4, bit=0), # SHIFT
344         ButtonSpec(channel=3, byte=3, bit=6), # ESC
345         ButtonSpec(channel=3, byte=2, bit=4), # 1
346         ButtonSpec(channel=3, byte=2, bit=5), # 2
347         ButtonSpec(channel=3, byte=2, bit=6), # 3
348         ButtonSpec(channel=3, byte=2, bit=7), # 4
349         ButtonSpec(channel=3, byte=2, bit=0), # ROLL CLOCKWISE
350         ButtonSpec(channel=3, byte=1, bit=2), # TOP
351         ButtonSpec(channel=3, byte=4, bit=2), # ROTATION
352         ButtonSpec(channel=3, byte=1, bit=5), # FRONT
353         ButtonSpec(channel=3, byte=1, bit=4), # REAR
354         ButtonSpec(channel=3, byte=1, bit=1), # FIT
355     ],
356     axis_scale=350.0,
357 ),
358 "SpaceMouse Wireless": DeviceSpec(
359     name="SpaceMouse Wireless",
360     # vendor ID and product ID
361     hid_id=[0x256F, 0xC62E],
362     # LED HID usage code pair
363     led_id=[0x8, 0x4B],
364     mappings={
365         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
366         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
367         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
368         "pitch": AxisSpec(channel=1, byte1=7, byte2=8, scale=-1)
369         ,
370         "roll": AxisSpec(channel=1, byte1=9, byte2=10, scale=-1)
371         ,
372         "yaw": AxisSpec(channel=1, byte1=11, byte2=12, scale=1),

```



```

370     },
371     button_mapping=[
372         ButtonSpec(channel=3, byte=1, bit=0), # LEFT
373         ButtonSpec(channel=3, byte=1, bit=1), # RIGHT
374     ], # FIT
375     axis_scale=350.0,
376 ),
377 "3Dconnexion Universal Receiver": DeviceSpec(
378     name="3Dconnexion Universal Receiver",
379     # vendor ID and product ID
380     hid_id=[0x256F, 0xC652],
381     # LED HID usage code pair
382     led_id=[0x8, 0x4B],
383     mappings={
384         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
385         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),
386         "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
387         "pitch": AxisSpec(channel=1, byte1=7, byte2=8, scale=-1)
388         ,
389         "roll": AxisSpec(channel=1, byte1=9, byte2=10, scale=-1)
390         ,
391         "yaw": AxisSpec(channel=1, byte1=11, byte2=12, scale=1),
392     },
393     button_mapping=[
394         ButtonSpec(channel=3, byte=1, bit=0), # MENU
395         ButtonSpec(channel=3, byte=3, bit=7), # ALT
396         ButtonSpec(channel=3, byte=4, bit=1), # CTRL
397         ButtonSpec(channel=3, byte=4, bit=0), # SHIFT
398         ButtonSpec(channel=3, byte=3, bit=6), # ESC
399         ButtonSpec(channel=3, byte=2, bit=4), # 1
400         ButtonSpec(channel=3, byte=2, bit=5), # 2
401         ButtonSpec(channel=3, byte=2, bit=6), # 3
402         ButtonSpec(channel=3, byte=2, bit=7), # 4
403         ButtonSpec(channel=3, byte=2, bit=0), # ROLL CLOCKWISE
404         ButtonSpec(channel=3, byte=1, bit=2), # TOP
405         ButtonSpec(channel=3, byte=4, bit=2), # ROTATION
406         ButtonSpec(channel=3, byte=1, bit=5), # FRONT
407         ButtonSpec(channel=3, byte=1, bit=4), # REAR
408         ButtonSpec(channel=3, byte=1, bit=1),
409     ], # FIT
410     axis_scale=350.0,
411 ),
412 "SpacePilot Pro": DeviceSpec(
413     name="SpacePilot Pro",
414     # vendor ID and product ID
415     hid_id=[0x46D, 0xC629],
416     # LED HID usage code pair
417     led_id=[0x8, 0x4B],
418     mappings={
419         "x": AxisSpec(channel=1, byte1=1, byte2=2, scale=1),
420         "y": AxisSpec(channel=1, byte1=3, byte2=4, scale=-1),

```

```

419     "z": AxisSpec(channel=1, byte1=5, byte2=6, scale=-1),
420     "pitch": AxisSpec(channel=2, byte1=1, byte2=2, scale=-1)
421     ,
422     "roll": AxisSpec(channel=2, byte1=3, byte2=4, scale=-1),
423     "yaw": AxisSpec(channel=2, byte1=5, byte2=6, scale=1),
424 },
425 button_mapping=[
426     ButtonSpec(channel=3, byte=4, bit=0), # SHIFT
427     ButtonSpec(channel=3, byte=3, bit=6), # ESC
428     ButtonSpec(channel=3, byte=4, bit=1), # CTRL
429     ButtonSpec(channel=3, byte=3, bit=7), # ALT
430     ButtonSpec(channel=3, byte=3, bit=1), # 1
431     ButtonSpec(channel=3, byte=3, bit=2), # 2
432     ButtonSpec(channel=3, byte=2, bit=6), # 3
433     ButtonSpec(channel=3, byte=2, bit=7), # 4
434     ButtonSpec(channel=3, byte=3, bit=0), # 5
435     ButtonSpec(channel=3, byte=1, bit=0), # MENU
436     ButtonSpec(channel=3, byte=4, bit=6), # -
437     ButtonSpec(channel=3, byte=4, bit=5), # +
438     ButtonSpec(channel=3, byte=4, bit=4), # DOMINANT
439     ButtonSpec(channel=3, byte=4, bit=3), # PAN/ZOOM
440     ButtonSpec(channel=3, byte=4, bit=2), # ROTATION
441     ButtonSpec(channel=3, byte=2, bit=0), # ROLL CLOCKWISE
442     ButtonSpec(channel=3, byte=1, bit=2), # TOP
443     ButtonSpec(channel=3, byte=1, bit=5), # FRONT
444     ButtonSpec(channel=3, byte=1, bit=4), # REAR
445     ButtonSpec(channel=3, byte=2, bit=2), # ISO
446     ButtonSpec(channel=3, byte=1, bit=1), # FIT
447 ],
448 axis_scale=350.0,
449 ),
450 }
451 #
452 ///////////////////////////////////////////////////
453 # [Para el SpaceNavigator]
454 # El formato del HID es
455 # [id, a, b, c, d, e, f]
456 # cada par (a,b), (c,d), (e,f) es un valor con signo de 16-bit
457 # representando el estado absoluto del dispositivo [from -350 to
458 # 350]
459
460 # si id==1, el mapeo es
461 # (a,b) = y translation
462 # (c,d) = x translation
463 # (e,f) = z translation
464
465 # si id==2 el mapeo es
466 # (a,b) = x tilting (roll)
467 # (c,d) = y tilting (pitch)

```

```

465 # (d,e) = z tilting (yaw)
466
467 # si id==3 el mapeo es
468 # a = button. Bit 1 = button 1, bit 2 = button 2
469
470 # Cada movimiento del dispositivo siempre causa dos eventos de HID
471 # uno con id 1 y otro con id 2, para generarse un tras otro.
472
473
474 supported_devices = list(device_specs.keys())
475 _active_device = None
476
477
478 def close():
479     """Cierra el dispositivo activo"""
480     if _active_device is not None:
481         _active_device.close()
482
483
484 def read():
485     """Devuelve el estado actual del spacenavigator conectado.
486
487     Devuelve:
488         state: {t,x,y,z,pitch,yaw,roll,button} namedtuple
489         0 nada si no est abierto.
490     """
491     if _active_device is not None:
492         return _active_device.tuple_state
493     else:
494         return None
495
496
497 def list_devices():
498     """Devuelve una lista de dispositivos compatibles conectados
499
500     Devuelve:
501         Una lista de los elementos con soporte que sean encontrados.
502     """
503     devices = []
504     all_hids = hid.find_all_hid_devices()
505     if all_hids:
506         for index, device in enumerate(all_hids):
507             for device_name, spec in device_specs.items():
508                 if (
509                     device.vendor_id == spec.hid_id[0]
510                     and device.product_id == spec.hid_id[1]
511                 ):
512                     devices.append(device_name)
513     return devices
514
515

```

```

516 def open(callback=None, button_callback=None, device=None,
517         DeviceNumber=0):
518     """
519     Abre uno de los dispositivos spcadenavigator. Hace este
520     dispositivo el dispositivo activo actual, lo cual habilita
521     read() u close()
522     Para multiples dispositivos usa el read() y close() de los
523     respectivos objetos.
524
525     Parametros:
526         callback: Si se provee un callback, es llamado a cada
527         actualizacion de HID con una copia del estado actual de
528         namedtuple
529         button_callback: Si button_callback se recibe, es llamado
530         cada vez que se pulsa un boton, con los argumentos (
531         state_tuple, button_state)
532         device: nombre del dispositivo que se debe abrir. Debe de
533         poseer los valores de alguno de los dispositivos de la
534         lista.
535         si no encuentra, elige el primer dispositivo
536         compatible encontrado.
537         DeviceNumber: usa el primer dispositivo (DeviceNumber=0) que
538         encuentre.
539
540     Devuelve:
541         Un objeto del dispositivo si se ha abierto correctamente
542         0 nada si no esta abierto
543     """
544     # only used if the module-level functions are used
545     global _active_device
546
547     # if no device name specified, look for any matching device and
548     # choose the first
549     if device == None:
550         all_devices = list_devices()
551         if len(all_devices) > 0:
552             device = all_devices[0]
553         else:
554             return None
555
556     found_devices = []
557     all_hids = hid.find_all_hid_devices()
558     if all_hids:
559         for index, dev in enumerate(all_hids):
560             spec = device_specs[device]
561             if dev.vendor_id == spec.hid_id[0] and dev.product_id ==
562                 spec.hid_id[1]:
563                 found_devices.append({"Spec": spec, "HIDDevice": dev
564                                     })
565             print("%s found" % device)
566
567     else:

```

```

552         print("No HID devices detected")
553         return None
554
555     if len(found_devices) == 0:
556         print("No supported devices found")
557         return None
558     else:
559
560         if len(found_devices) <= DeviceNumber:
561             DeviceNumber = 0
562
563         if len(found_devices) > DeviceNumber:
564             # crea una copia de las especificaciones del dispositivo
565             spec = found_devices[DeviceNumber]["Spec"]
566             dev = found_devices[DeviceNumber]["HIDDevice"]
567             new_device = copy.deepcopy(spec)
568             new_device.device = dev
569
570             # establece los callbacks
571             new_device.callback = callback
572             new_device.button_callback = button_callback
573             # abre el dispositivo y establece el gestionador de
574             # datos
575             new_device.open()
576             dev.set_raw_data_handler(lambda x: new_device.process(x)
577                                     )
578             _active_device = new_device
579             return new_device
580
581         print("Unknown error occurred.")
582         return None
583
584 def print_state(state):
585     # llamada al print en consola
586     if state:
587         print(
588             " ".join(
589                 [
590                     "%4s %+.2f" % (k, getattr(state, k))
591                     for k in ["x", "y", "z", "roll", "pitch", "yaw",
592                             "t"]
593                 ]
594             )
595         )
596
597 def toggle_led(state, buttons):
598     print("".join(["buttons=", str(buttons)]))
599     # Activa los leds si B_1 y los desactiva si B_2
600     if buttons[0] == 1:

```

```

600         set_led(1)
601     if buttons[1] == 1:
602         set_led(0)
603
604
605 def set_led(state):
606     if _active_device:
607         _active_device.set_led(state)
608
609
610 # //////////////////////////////////////
611 def run_sn_server(address='localhost', port=65432):
612     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
613     )
614     server_socket.bind((address, port))
615     server_socket.listen(1)
616     print("Esperando conexion")
617     connection, client_address = server_socket.accept()
618     print("Conexion de", client_address)
619
620     # joystick = XInputJoystick(0)
621     Edmouse = DeviceSpec
622
623     try:
624         while True:
625             # state = joystick.get_state()
626             state = read()
627             if state:
628                 data = {
629                     # data = json.dumps({
630                     'x': state.x,
631                     'y': state.y,
632                     'z': state.z,
633                     'roll': state.roll,
634                     'pitch': state.pitch,
635                     'yaw': state.yaw,
636                     't': state.t,
637                     'buttons': list(state.buttons)
638                 }
639                 # })
640                 message = json.dumps(data) + '\n'
641                 connection.sendall(message.encode('utf-8'))
642                 # # connection.sendall(json.dumps(data).encode('utf
643                 -8'))
644                 # connection.sendall(data.encode('utf-8'))
645                 time.sleep(0.01) # limita frec de envio a 100Hz
646     finally:
647         connection.close()
648         server_socket.close()
649
650 # //////////////////////////////////////

```

```

649
650 # //////////////////////////////////////
651 def run_sn_egm_server(address='localhost', port=65433):
652     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
653     )
654     server_socket.bind((address, port))
655     server_socket.listen(1)
656     print("Esperando conexion")
657     connection, client_address = server_socket.accept()
658     print("Conexion de", client_address)
659
660     # joystick = XInputJoystick(0)
661     Edmouse = DeviceSpec
662
663     try:
664         while True:
665             # state = joystick.get_state()
666             state = read()
667             if state:
668                 data = {
669                     # data = json.dumps({
670                     'x': state.x,
671                     'y': state.y,
672                     'z': state.z,
673                     'roll': state.roll,
674                     'pitch': state.pitch,
675                     'yaw': state.yaw,
676                     't': state.t,
677                     'buttons': list(state.buttons)
678                 }
679                 # })
680                 message = json.dumps(data) + '\n'
681                 connection.sendall(message.encode('utf-8'))
682                 # # connection.sendall(json.dumps(data).encode('utf
683                 # -8'))
684                 # connection.sendall(data.encode('utf-8'))
685                 time.sleep(0.01) # limita freq de envio a 100Hz
686             finally:
687                 connection.close()
688                 server_socket.close()
689
690 # //////////////////////////////////////
691
692 if __name__ == "__main__":
693     # server_thread = threading.Thread(target=run_sn_server)
694     # server_thread.start()
695
696     print("Devices found:\n\t%s" % "\n\t".join(list_devices()))
697     dev = open(callback=print_state, button_callback=toggle_led)
698     print(dev.describe_connection())
699

```

```
698     if dev:
699         dev.set_led(0)
700         server_thread = threading.Thread(target=run_sn_server)
701         server_egm = threading.Thread(target=run_sn_egm_server)
702         server_thread.start()
703         server_egm.start()
704         while 1:
705             sleep(1)
706             dev.set_led(1)
707             sleep(1)
708             dev.set_led(0)
```


ANEXO 3: GUI:

x_input_display

CÓDIGO 7.3. Codigo para la GUI xinput

```
1  import socket
2  import json
3  import pygame
4  import sys
5
6
7  # //////////////////////////////////////
8  BUTTON_MAP = {
9      0x0001: 'pad_up', 0x0002: 'pad_down', 0x0004: 'pad_left', 0x0008
10         : 'pad_right',
11      0x0010: 'start', 0x0020: 'back', 0x0040: 'L3', 0x0080: 'R3',
12      0x0100: 'LB', 0x0200: 'RB', 0x1000: 'A', 0x2000: 'B', 0x4000: 'X
13         ', 0x8000: 'Y'
14  }
15
16  BUTTON_POSITIONS = {
17      0x0001: (200, 400), 0x0002: (200, 500), 0x0004: (150, 450),
18      0x0008: (250, 450),
19      0x0010: (450, 200), 0x0020: (350, 200), 0x0040: (200, 300),
20      0x0080: (600, 500),
21      0x0100: (200, 100), 0x0200: (600, 100), 0x1000: (600, 400),
22      0x2000: (650, 350),
23      0x4000: (550, 350), 0x8000: (600, 300)
24  }
25
26  LEFT_TRIGGER_POS = (50, 300)
27  RIGHT_TRIGGER_POS = (700, 300)
28  LEFT_JOYSTICK_POS = (200, 300)
29  RIGHT_JOYSTICK_POS = (600, 500)
30
31  STATE_POS_CAR = (750, 520)
32  STATE_POS_POL = (750, 550)
33
34  # //////////////////////////////////////
35
36  def draw_text(screen, text, position, font, color=(255, 255, 255)):
37      text_surface = font.render(text, True, color)
38      screen.blit(text_surface, position)
39
40  def run_display_client(address='localhost', port=5000):
```

```

37     # pygame.init()
38     # screen_width, screen_height = 800, 600
39     # screen = pygame.display.set_mode((screen_width, screen_height)
40         )
41
42     # Establecer la conexi n
43     client_socket_ = socket.socket(socket.AF_INET, socket.
44         SOCK_STREAM)
45     client_socket_.connect((address, port))
46     print("conexi n de servidor")
47     return client_socket_
48
49 # //////////////////////////////////////
50 def update_display(screen_, joystick_state):
51     # L gica para dibujar los botones y joysticks basada en el
52     estado recibido
53     screen.fill((0, 0, 0))
54     for button, position in BUTTON_POSITIONS.items():
55         # el grupo BUTTON_POSITIONS tiene dos elementos en un orden
56         y en el for establezco que representan en ese orden
57         color = (0, 255, 0) if joystick_state['buttons'] & button
58         else (255, 255, 255)
59         pygame.draw.circle(screen_, color, position, 20)
60         draw_text(screen_, BUTTON_MAP[button], (position[0] + 25,
61             position[1] - 10), font)
62
63     draw_trigger(screen_, joystick_state['left_trigger'],
64         LEFT_TRIGGER_POS)
65     draw_trigger(screen_, joystick_state['right_trigger'],
66         RIGHT_TRIGGER_POS)
67
68     draw_joystick(screen_, joystick_state['l_thumb_x'],
69         joystick_state['l_thumb_y'], LEFT_JOYSTICK_POS)
70     draw_joystick(screen_, joystick_state['r_thumb_x'],
71         joystick_state['r_thumb_y'], RIGHT_JOYSTICK_POS)
72
73     draw_tog(screen_, joystick_state['cartesian_mode'],
74         STATE_POS_CAR)
75     draw_tog(screen_, not joystick_state['cartesian_mode'],
76         STATE_POS_POL)
77
78     pygame.display.update()
79
80     # pass # Aqu ir a el c digo para dibujar basado en
81     joystick_state
82
83 def draw_trigger(screen_, value, position):
84     pygame.draw.rect(screen_, (255, 255, 255), (*position, 20, 100))

```

```

74     pygame.draw.rect(screen_, (0, 255, 0), (*position, 20, int(100 *
       value)))
75     if position == LEFT_TRIGGER_POS:
76         draw_text(screen_, 'LT', (position[0], position[1] - 20),
           font)
77     else:
78         draw_text(screen_, 'RT', (position[0], position[1] - 20),
           font)
79
80
81 def draw_joystick(screen_, x_value, y_value, position):
82     pygame.draw.circle(screen_, (255, 255, 255), position, 40, 1)
83     pygame.draw.circle(screen_, (0, 255, 0),
84         (position[0] + int(40 * x_value), position[1]
           - int(40 * y_value)), 10)
85
86
87 def draw_tog(screen_, state_, position):
88     color = (0, 255, 0) if state_ else (255, 255, 255)
89     pygame.draw.rect(screen_, color, (*position, 20, 20))
90
91
92 if __name__ == "__main__":
93     pygame.init()
94     screen = pygame.display.set_mode((800, 600))
95     font = pygame.font.Font(None, 24)
96     # run_display_client()
97     client_socket = run_display_client()
98
99     buffer = ""
100    try:
101        while True:
102            data = client_socket.recv(4096).decode('utf-8') # antes
               recv() era 1024
103            if not data:
104                break
105            buffer += data
106            while '\n' in buffer:
107                message, buffer = buffer.split('\n', 1)
108                state = json.loads(message)
109                update_display(screen, state)
110                # state = json.loads(data.decode('utf-8'))
111                # update_display(screen, state)
112
113            for event in pygame.event.get():
114                if event.type == pygame.QUIT:
115                    pygame.quit()
116                    sys.exit()
117
118            # pygame.time.delay(10)
119    finally:

```

s_nav_display

CÓDIGO 7.4. Código para la GUI de spacenavigator

```

1  import socket
2  import json
3  import pygame
4  import sys
5  import threading
6
7  # //////////////////////////////////////
8  BUTTON_MAP = {
9      0x0001: 'B_1', 0x0002: 'B_2'
10 }
11
12 BUTTON_POSITIONS = {
13     0x0001: (100, 20), 0x0002: (100, 280)
14 }
15 Z_TRIGGER_POS = (135, 200)
16 YAW_TRIGGER_POS = (385, 175)
17 LEFT_JOYSTICK_POS = (100, 500)
18 RIGHT_JOYSTICK_POS = (300, 500)
19
20 # //////////////////////////////////////
21
22
23 def run_display_client(address='localhost', port=65432):
24     # pygame.init()
25     # screen_width, screen_height = 800, 600
26     # screen = pygame.display.set_mode((screen_width, screen_height)
27     #                                     )
28     # pygame.display.set_caption("Display de Controlador Xbox")
29
30     # Establecer la conexión
31     client_socket_ = socket.socket(socket.AF_INET, socket.
32                                   SOCK_STREAM)
33     client_socket_.connect((address, port))
34     print("conexión de servidor")
35     return client_socket_
36
37 # //////////////////////////////////////
38 def update_display(screen_, joystick_state):
39     # Lógica para dibujar los botones y joysticks basada en el
40     # estado recibido
41     screen_.fill((0, 0, 0))

```

```

40
41 # dibujar botones
42 for i, button in enumerate(state['buttons']):
43     # el grupo BUTTON_POSITIONS tiene dos elementos en un orden
44     y en el for establezco que representan en ese orden
45     color = (0, 0, 255) if button else (255, 255, 255)
46     pygame.draw.circle(screen_, color, (250 * (1 + i), 100), 20)
47     # draw_text(screen_, BUTTON_MAP[button], (250 * (1 + i) -
48         10, 100 - 10), font)
49
50 # dibujo de barras
51 draw_bar(screen_, joystick_state['z'], Z_TRIGGER_POS, "Z",
52     horizontal=False)
53 draw_bar(screen_, joystick_state['yaw'], YAW_TRIGGER_POS, "YAW",
54     horizontal=True)
55
56 # dibujo de etiquetas
57 pygame.draw.line(screen_, (255, 255, 255), (265 - 40, 300), (265
58     + 40, 300))
59 pygame.draw.line(screen_, (255, 255, 255), (265, 300 - 40),
60     (265, 300 + 40))
61 pygame.draw.line(screen_, (255, 255, 255), (485 - 40, 300), (485
62     + 40, 300))
63 pygame.draw.line(screen_, (255, 255, 255), (485, 300 - 40),
64     (485, 300 + 40))
65 x_label = font.render("X", True, (255, 255, 255))
66 y_label = font.render("Y", True, (255, 255, 255))
67 screen_.blit(x_label, (265 + 50, 300))
68 screen_.blit(y_label, (265, 300 - 50))
69 pitch_label = font.render("ROLL", True, (255, 255, 255))
70 roll_label = font.render("PITCH", True, (255, 255, 255))
71 screen_.blit(pitch_label, (485 + 50, 300))
72 screen_.blit(roll_label, (485, 300 - 50))
73
74 #joysticks
75 pygame.draw.circle(screen_, (255, 255, 255), (265, 300), 100, 1)
76 pygame.draw.circle(screen_, (0, 0, 200), (int(265 + state['x'] *
77     80), int(300 - state['y'] * 80)), 10)
78
79 pygame.draw.circle(screen_, (255, 255, 255), (485, 300), 100, 1)
80 pygame.draw.circle(screen_, (0, 0, 200), (int(485 + state['roll
81     ']' * 80), int(300 - state['pitch'] * 80)), 10)
82
83 pygame.display.update()
84
85 def draw_bar(screen_, value, position, label, horizontal=False):
86     label_text = font.render(label, True, (255, 255, 255))
87     if horizontal:
88         pygame.draw.rect(screen_, (255, 255, 255), (*position, 200,
89             20), 1)

```

```

80         if value >= 0:
81             pygame.draw.rect(screen_, (0, 0, 255), (position[0] +
82                 100, position[1], int(value * 100), 20))
83         else:
84             pygame.draw.rect(screen_, (255, 0, 255), (position[0] +
85                 100 + int(value * 100), position[1], -int(value *
86                 100), 20))
87             screen_.blit(label_text, (position[0] + 200, position[1] +
88                 25))
89         else:
90             pygame.draw.rect(screen_, (255, 255, 255), (*position, 20,
91                 220), 1)
92             if value >= 0:
93                 pygame.draw.rect(screen_, (0, 0, 255), (position[0],
94                     position[1] + 100 - int(value * 100), 20, int(value
95                         * 100)))
96             else:
97                 pygame.draw.rect(screen_, (255, 0, 255), (position[0],
98                     position[1] + 100, 20, -int(value * 100)))
99             screen_.blit(label_text, (position[0] - 20, position[1]))
100
101 if __name__ == "__main__":
102     pygame.init()
103     screen = pygame.display.set_mode((800, 600))
104
105     font = pygame.font.Font(None, 24)
106
107     # run_display_client()
108     client_socket = run_display_client()
109
110     buffer = ""
111     try:
112         while True:
113             data = client_socket.recv(4096).decode('utf-8') # antes
114                 recv() era 1024
115             if not data:
116                 break
117             buffer += data
118             while '\n' in buffer:
119                 message, buffer = buffer.split('\n', 1)
120                 state = json.loads(message)
121                 update_display(screen, state)
122             # state = json.loads(data.decode('utf-8'))
123             # update_display(screen, state)
124
125             for event in pygame.event.get():
126                 if event.type == pygame.QUIT:
127                     pygame.quit()
128                     sys.exit()

```

```
122         # pygame.time.delay(10)
123     finally:
124         client_socket.close()
```

ANEXO 4: EGM

egm_interface_Xbox

CÓDIGO 7.5. Codido para onexión EGM xbox

```
1  from abb_robot_client.egm import EGM
2  import abb_motion_program_exec as abb
3  import time
4  import numpy as np
5  import copy
6  import socket
7  import json
8  import pygame
9  import sys
10
11  v_global = 0
12  previous_state_x = False
13  previous_state_y = False
14
15
16  def clamp(value, min_value, max_value):
17      return max(min_value, min(value, max_value))
18
19
20  def normalize_quaternion(q):
21      norm = np.linalg.norm(q)
22      return [q[0] / norm, q[1] / norm, q[2] / norm, q[3] / norm]
23
24
25  def mix_target():
26      global v_global
27
28      while True:
29          if v_global == 0:
30              egm_pose_target()
31          elif v_global == 1:
32              egm_joint_target()
33          elif v_global == 2:
34              print("Finalizaci n del programa por completo")
35              break
36          else:
37              print("Valor no manejado, finalizando programa")
38              break
39
40
41
```



```

89         previous_state_y = current_state_y
90
91         # Actualizaci n de las articulaciones seg n el
          mando
92         joints.robax[0] += state['l_thumb_x'] * gain #
          Articulaci n 1
93         joints.robax[1] += state['l_thumb_y'] * gain #
          Articulaci n 2
94         joints.robax[2] += state['r_thumb_y'] * gain #
          Articulaci n 3
95         joints.robax[3] += state['r_thumb_x'] * gain #
          Articulaci n 4
96
97         if state['buttons'] & 0x0001: # PAD_UP
98             joints.robax[4] += gain
99         if state['buttons'] & 0x0002: # PAD_DOWN
100             joints.robax[4] -= gain
101
102         if state['buttons'] & 0x0100: # LB
103             joints.robax[5] -= gain
104         if state['buttons'] & 0x0200: # RB
105             joints.robax[5] += gain
106
107         # Aplicar restricciones a las articulaciones
108         joints.robax[0] = clamp(joints.robax[0], -170, 170)
          # Articulaci n 1
109         joints.robax[1] = clamp(joints.robax[1], -70, 90) #
          Articulaci n 2
110         joints.robax[2] = clamp(joints.robax[2], -60, 40) #
          Articulaci n 3
111         joints.robax[3] = clamp(joints.robax[3], -90, 90) #
          Articulaci n 4
112         joints.robax[4] = clamp(joints.robax[4], -80, 90) #
          Articulaci n 5
113         joints.robax[5] = clamp(joints.robax[5], -180, 180)
          # Articulaci n 6
114
115         # Enviar los valores de las articulaciones al robot
116         egm.send_to_robot(joints.robax)
117
118         if state['buttons'] & 0x0010: # Bot n START para
          salir
119             v_global = 3
120             break
121
122         if state['buttons'] & 0x0020:
123             v_global = 0
124             break
125
126         # egm.send_to_robot(np.ones((6,)) * np.sin(t2 - t1) * 5)
127

```

```

128     client.stop_egm()
129
130     while client.is_motion_program_running():
131         time.sleep(0.05)
132
133     log_results = client.read_motion_program_result_log(lognum)
134
135     # log_results.data is a numpy array
136     import matplotlib.pyplot as plt
137     fig, ax1 = plt.subplots()
138     lns1 = ax1.plot(log_results.data[:, 0], log_results.data[:, 2:])
139     ax1.set_xlabel("Time (s)")
140     ax1.set_ylabel("Joint angle (deg)")
141     ax2 = ax1.twinx()
142     lns2 = ax2.plot(log_results.data[:, 0], log_results.data[:, 1],
143                     '-k')
144     ax2.set_ylabel("Command number")
145     ax2.set_yticks(range(-1, int(max(log_results.data[:, 1])) + 1))
146     ax1.legend(lns1 + lns2, log_results.column_headers[2:] + ["
147         cmdnum"])
148     ax1.set_title("Joint motion")
149     plt.show()
150     print("Operaci n terminada")
151
152 def egm_pose_target(gain=10):
153     global v_global, previous_state_x, previous_state_y
154     # config de limites de correccion
155     mm = abb.egm_minmax(-1e-3, 1e-3)
156
157     # config de los marcos de referencia
158     corr_frame = abb.pose([0, 0, 0], [1, 0, 0, 0])
159     corr_fr_type = abb.egmframetype.EGM_FRAME_WOBJ
160     sense_frame = abb.pose([0, 0, 0], [1, 0, 0, 0])
161     sense_fr_type = abb.egmframetype.EGM_FRAME_WOBJ
162     egm_offset = abb.pose([0, 0, 0], [1, 0, 0, 0])
163
164     # config EGM para orientaci n en el espacio de trabajo
165     egm_config = abb.EGMPoseTargetConfig(corr_frame, corr_fr_type,
166                                           sense_frame, sense_fr_type,
167                                           mm, mm, mm, mm, mm, mm,
168                                           1000, 1000
169                                           )
170
171     # establece mov definido
172     r1 = abb.robtarget([400, 100, 600], [0.7071068, 0., 0.7071068,
173         0.], abb.confdata(0, 0, 0, 1), [0] * 6)
174     # r3 = abb.robtarget([500, 50, 600], [0.7071068, 0., 0.7071068,
175         0.], abb.confdata(0, 0, 0, 1), [0] * 6)
176
177     # creaci n del programa de mov
178     mp = abb.MotionProgram(egm_config=egm_config)

```

```

173 mp.MoveJ(r1, abb.v1000, abb.fine)
174 # mp.MoveJ(r3, abb.v1000, abb.fine)
175 mp.EGMRunPose(10, 0.05, 0.05, egm_offset)
176
177 # envio de datos al robot
178 client = abb.MotionProgramExecClient(base_url="http
      ://127.0.0.1:80")
179 lognum = client.execute_motion_program(mp, wait=False)
180
181 # conexion con mando
182 xinput_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
      )
183 xinput_socket.connect(('localhost', 5001))
184 buffer = ""
185
186 # recepci n y envio de correcciones en bucle
187 t1 = time.perf_counter()
188
189 r2 = copy.copy(r1)
190 # r4 = copy.copy(r3)
191
192 egm = EGM()
193 t2 = t1
194 while True:
195     t2 = time.perf_counter()
196     res, feedback = egm.receive_from_robot(timeout=0.05)
197     if res:
198         data = xinput_socket.recv(4096).decode('utf-8')
199         buffer += data
200         if '\n' in buffer:
201             message, buffer = buffer.split('\n', 1)
202             state = json.loads(message)
203
204             # Detectar el evento de pulsaci n del bot n X (
                Disminuir gain)
205             current_state_x = state['buttons'] & 0x0040
206             if current_state_x and not previous_state_x:
207                 gain = max(0.5, gain - 0.5)
208                 print(f"Gain disminuido a: {gain}")
209             previous_state_x = current_state_x
210
211             # Detectar el evento de pulsaci n del bot n Y (
                Aumentar gain)
212             current_state_y = state['buttons'] & 0x0080
213             if current_state_y and not previous_state_y:
214                 gain += 0.5
215                 print(f"Gain aumentado a: {gain}")
216             previous_state_y = current_state_y
217
218             # actualizaci n del robot
219             r2.trans[0] += state['l_thumb_x'] * gain

```

```

220     r2.trans[1] += state['l_thumb_y'] * gain
221     r2.trans[2] += (state['right_trigger'] - state['
        left_trigger']) * gain
222
223     # Actualizaci n de la orientaci n del robot
224     q1 = clamp(r2.rot[0] + state['r_thumb_x'] * 0.01,
        -0.7071068, 0.7071068) # Q1
225     q2 = clamp(r2.rot[1] + state['r_thumb_y'] * 0.01,
        -0.7071068, 0.7071068) # Q2
226     q3 = clamp(r2.rot[2] + (state['buttons'] & 0x0004 -
        state['buttons'] & 0x0008) * 0.01, -0.7071068,
        0.7071068) # Q3
227     if state['buttons'] & 0x0004: # PAD_LEFT
228         q3 = clamp(r2.rot[2] - 0.01 * gain, -0.7071068,
        0.7071068) # Disminuye Q3
229     if state['buttons'] & 0x0008: # PAD_RIGHT
230         q3 = clamp(r2.rot[2] + 0.01 * gain, -0.7071068,
        0.7071068) # Aumenta Q3
231
232
233     # Recalcular el cuarto componente del cuaterni n (
        Q4) para mantenerlo unitario
234     #q4 = np.sqrt(1.0 - (q1 ** 2 + q2 ** 2 + q3 ** 2))
235
236     # Asegurar que Q1^2 + Q2^2 + Q3^2 <= 1 para evitar
        valores inv lidos
237     if q1 ** 2 + q2 ** 2 + q3 ** 2 <= 1.0:
238         q4 = np.sqrt(1.0 - (q1 ** 2 + q2 ** 2 + q3 ** 2)
        )
239     else:
240         q4 = r2.rot[3] # Mantener el valor anterior de
        Q4 si los otros valores son inv lidos
241
242     r2.rot[0] = q1
243     r2.rot[1] = q2
244     r2.rot[2] = q3
245     r2.rot[3] = q4
246
247     if state['buttons'] & 0x0010:
248         v_global = 3
249         break
250
251     if state['buttons'] & 0x0020:
252         v_global = 1
253         break
254
255     egm.send_to_robot_cart(r2.trans, r2.rot)
256     # r4.trans[1] = (t2 - t1) * 100.
257     # egm.send_to_robot_cart(r4.trans, r4.rot)
258
259     # detenci n de EGM y graficar datos botenidos
260     client.stop_egm()

```

```

261
262     while client.is_motion_program_running():
263         time.sleep(0.05)
264
265     log_results = client.read_motion_program_result_log(lognum)
266
267     # log_results.data is a numpy array
268     import matplotlib.pyplot as plt
269     fig, ax1 = plt.subplots()
270     lns1 = ax1.plot(log_results.data[:, 0], log_results.data[:, 2:])
271     ax1.set_xlabel("Time (s)")
272     ax1.set_ylabel("Joint angle (deg)")
273     ax2 = ax1.twinx()
274     lns2 = ax2.plot(log_results.data[:, 0], log_results.data[:, 1],
275                     '-k')
276     ax2.set_ylabel("Command number")
277     ax2.set_yticks(range(-1, int(max(log_results.data[:, 1])) + 1))
278     ax1.legend(lns1 + lns2, log_results.column_headers[2:] + ["
279         cmdnum"])
280     ax1.set_title("Joint motion")
281     plt.show()
282     print("Operaci n terminada")
283
284 if __name__ == "__main__":
285     mix_target()

```

egm_s_nav

CÓDIGO 7.6. Ccodigo para control EGM spacenavigator

```

1  from abb_robot_client.egm import EGM
2  import abb_motion_program_exec as abb
3  import time
4  import numpy as np
5  import copy
6  import socket
7  import json
8  import pygame
9  import sys
10
11
12  def clamp(value, min_value, max_value):
13      return max(min_value, min(value, max_value))
14
15
16  def normalize_quaternion(q):
17      norm = np.linalg.norm(q)

```

```

18     return [q[0] / norm, q[1] / norm, q[2] / norm, q[3] / norm]
19
20
21 def apply_deadzone(value, threshold):
22     if abs(value) < threshold:
23         return 0
24     return value
25
26
27 def egm_pose_target(gain=1):
28     # config de limites de correccion
29     mm = abb.egm_minmax(-1e-3, 1e-3)
30
31     # config de los marcos de referencia
32     corr_frame = abb.pose([0, 0, 0], [1, 0, 0, 0])
33     corr_fr_type = abb.egmframetype.EGM_FRAME_WOBJ
34     sense_frame = abb.pose([0, 0, 0], [1, 0, 0, 0])
35     sense_fr_type = abb.egmframetype.EGM_FRAME_WOBJ
36     egm_offset = abb.pose([0, 0, 0], [1, 0, 0, 0])
37
38     # config EGM para orientaci n en el espacio de trabajo
39     egm_config = abb.EGMPoseTargetConfig(corr_frame, corr_fr_type,
40                                         sense_frame, sense_fr_type,
41                                         mm, mm, mm, mm, mm, mm,
42                                         1000, 1000
43                                         )
44
45     # establece mov definido
46     r1 = abb.robtarg([400, 100, 600], [0.7071068, 0., 0.7071068,
47     0.], abb.confdata(0, 0, 0, 1), [0] * 6)
48     # r3 = abb.robtarg([500, 50, 600], [0.7071068, 0., 0.7071068,
49     0.], abb.confdata(0, 0, 0, 1), [0] * 6)
50
51     # creaci n del programa de mov
52     mp = abb.MotionProgram(egm_config=egm_config)
53     mp.MoveJ(r1, abb.v1000, abb.fine)
54     # mp.MoveJ(r3, abb.v1000, abb.fine)
55     mp.EGMRunPose(10, 0.05, 0.05, egm_offset)
56
57     # envio de datos al robot
58     client = abb.MotionProgramExecClient(base_url="http
59     ://127.0.0.1:80")
60     lognum = client.execute_motion_program(mp, wait=False)
61
62     # conexi on con mando
63     xinput_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM
64     )
65     xinput_socket.connect(('localhost', 65433))
66     buffer = ""
67
68     # recepci n y envio de correcciones en bucle
69     t1 = time.perf_counter()

```

```

63
64     r2 = copy.copy(r1)
65     # r4 = copy.copy(r3)
66
67     egm = EGM()
68     t2 = t1
69     while True:
70         t2 = time.perf_counter()
71         res, feedback = egm.receive_from_robot(timeout=0.05)
72         if res:
73             data = xinput_socket.recv(4096).decode('utf-8')
74             buffer += data
75             if '\n' in buffer:
76                 message, buffer = buffer.split('\n', 1)
77                 state = json.loads(message)
78
79                 if state['buttons'][0] == 1: # Bot n X para
                    disminuir ganancia
80                     gain = max(0.5, gain - 0.5)
81                     print(f"Gain disminuido a: {gain}")
82                 if state['buttons'][1] == 1: # Bot n Y para
                    aumentar ganancia
83                     gain += 0.5
84                     print(f"Gain aumentado a: {gain}")
85
86                 x = apply_deadzone(state['x'], 0.5)
87                 y = apply_deadzone(state['y'], 0.5)
88                 z = apply_deadzone(state['z'], 0.5)
89
90                 r2.trans[0] += x * gain
91                 r2.trans[1] += y * gain
92                 r2.trans[2] += z * gain
93
94                 # Actualizaci n de la orientaci n del robot
95                 q1 = clamp(r2.rot[0] + state['roll'] * 0.01,
                    -0.7071068, 0.7071068) # Q1
96                 q2 = clamp(r2.rot[1] + state['pitch'] * 0.01,
                    -0.7071068, 0.7071068) # Q2
97                 q3 = clamp(r2.rot[2] + state['yaw'] * 0.01,
                    -0.7071068,
98                     0.7071068) # Q3
99
100                 # Recalcular el cuarto componente del cuaterni n (
                    Q4) para mantenerlo unitario
101                 #q4 = np.sqrt(1.0 - (q1 ** 2 + q2 ** 2 + q3 ** 2))
102
103                 # Asegurar que Q1^2 + Q2^2 + Q3^2 <= 1 para evitar
                    valores inv lidos
104                 if q1 ** 2 + q2 ** 2 + q3 ** 2 <= 1.0:
105                     q4 = np.sqrt(1.0 - (q1 ** 2 + q2 ** 2 + q3 ** 2)

```



```

106         else:
107             q4 = r2.rot[3] # Mantener el valor anterior de
                             Q4 si los otros valores son inv lidos
108
109             r2.rot[0] = q1
110             r2.rot[1] = q2
111             r2.rot[2] = q3
112             r2.rot[3] = q4
113
114             if state['buttons'][0] == 1 and state['buttons'][1]
               == 1:
115                 print("Ambos botones presionados, saliendo...")
116                 break
117
118             egm.send_to_robot_cart(r2.trans, r2.rot)
119             # r4.trans[1] = (t2 - t1) * 100.
120             # egm.send_to_robot_cart(r4.trans, r4.rot)
121
122             # detenci n de EGM y graficar datos botenidos
123             client.stop_egm()
124
125             while client.is_motion_program_running():
126                 time.sleep(0.05)
127
128             log_results = client.read_motion_program_result_log(lognum)
129
130             # log_results.data is a numpy array
131             import matplotlib.pyplot as plt
132             fig, ax1 = plt.subplots()
133             lns1 = ax1.plot(log_results.data[:, 0], log_results.data[:, 2:])
134             ax1.set_xlabel("Time (s)")
135             ax1.set_ylabel("Joint angle (deg)")
136             ax2 = ax1.twinx()
137             lns2 = ax2.plot(log_results.data[:, 0], log_results.data[:, 1],
                             '-k')
138             ax2.set_ylabel("Command number")
139             ax2.set_yticks(range(-1, int(max(log_results.data[:, 1])) + 1))
140             ax1.legend(lns1 + lns2, log_results.column_headers[2:] + ["
               cmdnum"])
141             ax1.set_title("Joint motion")
142             plt.show()
143             print("Operaci n terminada")
144
145
146 if __name__ == "__main__":
147     egm_pose_target()

```

ANEXO 5: RAPID

error_reporter

```
1  MODULE error_reporter
2
3      VAR intnum motion_program_err_interrupt;
4
5      PROC main()
6          CONNECT motion_program_err_interrupt WITH
              motion_program_trap_err;
7          IError COMMON_ERR, TYPE_ERR, motion_program_err_interrupt;
8          WHILE TRUE DO
9              WaitTime 1;
10         ENDWHILE
11
12     ENDPROC
13
14     TRAP motion_program_trap_err
15         VAR trapdata err_data;
16         VAR errdomain err_domain;
17         VAR num err_number;
18         VAR errtype err_type;
19         VAR string titlestr;
20         VAR string string1;
21         VAR string string2;
22         VAR num seqno;
23         VAR string err_filename;
24         VAR iodev err_file;
25
26         GetTrapData err_data;
27         ReadErrData err_data, err_domain, err_number, err_type \
            Title:=titlestr \Str1:=string1 \Str2:=string2;
28
29         ErrWrite \W, "Motion Program Failed", "Motion Program Failed
            at command number " + NumToStr(motion_program_state{1}.
            current_cmd_num,0)
30         \RL2:= "with error code " + NumToStr(err_domain*10000+
            err_number,0)
31         \RL3:= "error title '" + titlestr + "'"
32         \RL4:= "error string '" + string1 + "'";
33
34         seqno := motion_program_seqno_started;
35         IF seqno > 0 THEN
36             err_filename := "motion_program_err---seqno-" + NumToStr
                (seqno,0) + ".json";
37             Open "RAMDISK:"\File:=err_filename,err_file\Write;
```

```

38         Write err_file, "{"seqno": " + NumToStr(seqno,0)\
        NoNewLine;
39         Write err_file, ","error_domain": " + NumToStr(
        err_domain,0)\NoNewLine;
40         Write err_file, ","error_number": " + NumToStr(
        err_number,0)\NoNewLine;
41         Write err_file, ","error_title": "" + titlestr +
        """"\NoNewLine;
42         Write err_file, ","error_string": "" + string1 +
        """"\NoNewLine;
43         Write err_file, ","error_string2": "" + string2 +
        """"\NoNewLine;
44         Write err_file, "}";
45         Close err_file;
46     ENDIF
47
48
49     ENDTRAP
50 ENDMODULE

```

motion_program_logger

```

1  MODULE motion_program_logger
2
3      LOCAL VAR bool log_file_open:=FALSE;
4      LOCAL VAR iodev log_io_device;
5      LOCAL VAR clock time_stamp_clock;
6
7      LOCAL VAR intnum rmqint_open;
8      LOCAL VAR string rmq_timestamp;
9      LOCAL VAR string rmq_filename;
10     LOCAL VAR string rmq_req{2};
11
12     LOCAL VAR intnum logger_err_interrupt;
13     LOCAL VAR num robot_count:=0;
14
15
16     PROC motion_program_logger_main()
17
18         VAR num clk_now;
19         VAR num c:=0;
20         VAR num loop_count:=0;
21         VAR num clk_diff;
22
23         VAR num mechunit_listnum:=0;
24         VAR string mechunit_name:="";
25
26         CONNECT rmqint_open WITH rmq_message_string;

```

```

27     IRMQMessage rmq_req, rmqint_open;
28
29     CONNECT logger_err_interrupt WITH err_handler;
30     IError COMMON_ERR, TYPE_ERR, logger_err_interrupt;
31
32     WHILE GetNextMechUnit(mechunit_listnum, mechunit_name) DO
33         robot_count:=robot_count+1;
34     ENDWHILE
35
36     ClkReset time_stamp_clock;
37     ClkStart time_stamp_clock;
38     WHILE TRUE DO
39         clk_now:=ClkRead(time_stamp_clock \HighRes);
40         motion_program_state{1}.clk_time:=clk_now;
41         motion_program_state{1}.joint_position:=CJointT(\TaskRef
           :=T_ROB1Id);
42         IF robot_count >= 2 THEN
43             motion_program_state{2}.joint_position:=CJointT(\
               TaskName:="T_ROB2");
44         ENDIF
45         IF log_file_open THEN
46             clk_diff:= loop_count*0.004 - clk_now;
47             loop_count := loop_count+1;
48             IF clk_diff > 0 THEN
49                 WaitTime clk_diff;
50             ENDIF
51         ELSE
52             loop_count := 0;
53             ClkStop time_stamp_clock;
54             ClkReset time_stamp_clock;
55             WaitTime 0.004;
56             ClkStart time_stamp_clock;
57         ENDIF
58
59         IDisable;
60         IF motion_program_executing <> 0 THEN
61             IF log_file_open THEN
62                 motion_program_log_data;
63             ENDIF
64         ENDIF
65         IEnable;
66
67     ENDWHILE
68 ENDPROC
69
70 PROC pack_num(num val, VAR rawbytes b)
71     PackRawBytes val, b, (RawBytesLen(b)+1)\Float4;
72 ENDPROC
73
74 PROC pack_str(string val, VAR rawbytes b)
75     PackRawBytes val, b, (RawBytesLen(b)+1)\ASCII;

```

```

76     ENDPROC
77
78     PROC pack_jointtarget(jointtarget jt, VAR rawbytes b)
79         pack_num jt.robax.rax_1, b;
80         pack_num jt.robax.rax_2, b;
81         pack_num jt.robax.rax_3, b;
82         pack_num jt.robax.rax_4, b;
83         pack_num jt.robax.rax_5, b;
84         pack_num jt.robax.rax_6, b;
85     ENDPROC
86
87     PROC motion_program_log_open()
88         VAR string log_filename;
89         VAR string header_str:="timestamp,cmdnum,J1,J2,J3,J4,J5,J6";
90         VAR num header_str_len;
91         VAR num rmq_timestamp_len;
92         VAR rawbytes header_bytes;
93
94         IF robot_count >= 2 THEN
95             header_str:="timestamp,cmdnum,J1,J2,J3,J4,J5,J6,J1_2,
96                 J2_2,J3_2,J4_2,J5_2,J6_2";
97         ENDIF
98
99         header_str_len:=StrLen(header_str);
100        rmq_timestamp_len:=StrLen(rmq_timestamp);
101        log_filename := "log-" + rmq_filename + ".bin";
102
103        pack_num motion_program_file_version, header_bytes;
104        pack_num rmq_timestamp_len, header_bytes;
105        pack_str rmq_timestamp, header_bytes;
106        pack_num header_str_len, header_bytes;
107        pack_str header_str, header_bytes;
108        Open "RAMDISK:" \File:=log_filename, log_io_device, \Write\
109            Bin;
110        WriteRawBytes log_io_device, header_bytes;
111        ErrWrite \I, "Motion Program Log File Opened", "Motion
112            Program Log File Opened with filename: " + log_filename;
113        log_file_open := TRUE;
114    ENDPROC
115
116    PROC motion_program_log_close()
117        Close log_io_device;
118        log_file_open := FALSE;
119        ErrWrite \I, "Motion Program Log File Closed", "Motion
120            Program Log File Closed";
121    ERROR
122        SkipWarn;
123        TRYNEXT;
124    ENDPROC
125
126    PROC motion_program_log_data()

```

```

123     VAR rawbytes data_bytes;
124     pack_num ClkRead(time_stamp_clock \HighRes), data_bytes;
125     pack_num motion_program_state{1}.current_cmd_num, data_bytes
126     ;
127     pack_jointtarget motion_program_state{1}.joint_position,
128     data_bytes;
129     IF robot_count >= 2 THEN
130         pack_jointtarget motion_program_state{2}.joint_position,
131         data_bytes;
132     ENDIF
133     WriteRawBytes log_io_device, data_bytes;
134 ENDPROC
135
136 TRAP rmq_message_string
137     VAR rmqmessage rmqmsg;
138     IDisable;
139     RMQGetMessage rmqmsg;
140     RMQGetMsgData rmqmsg, rmq_req;
141     rmq_timestamp:=rmq_req{1};
142     rmq_filename:=rmq_req{2};
143     IF log_file_open THEN
144         motion_program_log_close;
145     ENDIF
146     IF StrLen(rmq_timestamp) > 0 AND motion_program_log_motion >
147     0 THEN
148         motion_program_log_open;
149     ENDIF
150     IEnable;
151 ENDTRAP
152
153 TRAP err_handler
154
155     VAR trapdata err_data;
156     VAR errdomain err_domain;
157     VAR num err_number;
158     VAR errtype err_type;
159     IDisable;
160     ISleep logger_err_interrupt;
161
162     GetTrapData err_data;
163     ReadErrData err_data, err_domain, err_number, err_type;
164     IF log_file_open THEN
165         motion_program_log_close;
166     ENDIF
167
168     IWatch logger_err_interrupt;
169     IEnable;
170
171 ENDTRAP

```

motion_program_exec_egm

```

1  MODULE motion_program_exec_egm
2
3      CONST num egm_sample_rate:=4;
4
5      TASK PERS bool egm_symbol_fail:=FALSE;
6      TASK PERS bool have_egm:=TRUE;
7      LOCAL VAR egmident egmID1;
8      LOCAL VAR egmstate egmSt1;
9
10     LOCAL VAR errnum ERR_NO_EGM:=-1;
11
12     CONST num MOTION_PROGRAM_CMD_EGMJOINT:=50001;
13     CONST num MOTION_PROGRAM_CMD_EGMPOSE:=50002;
14     CONST num MOTION_PROGRAM_CMD_EGMMOVEL:=50003;
15     CONST num MOTION_PROGRAM_CMD_EGMMOVEC:=50004;
16
17     PROC motion_program_egm_init()
18
19         BookErrNo ERR_NO_EGM;
20
21         IF egm_symbol_fail THEN
22             RETURN ;
23         ENDIF
24
25         EGMReset egmID1;
26         WaitTime 0.005;
27         EGMGetId egmID1;
28         egmSt1:=EGMGetState(egmID1);
29
30
31         EGMSetupUC ROB_1,egmID1,"joint","UCdevice"\Joint\CommTimeout
           :=100000;
32
33
34         !IF egmSt1=EGM_STATE_RUNNING THEN
35             ! EGMStop egmID1,EGM_STOP_HOLD;
36         !ENDIF
37
38         EGMStreamStop egmID1;
39
40         IF NOT have_egm THEN
41             have_egm:=TRUE;
42         ENDIF
43     ENDPROC

```

```

44
45 PROC motion_program_egm_start_stream()
46     IF have_egm THEN
47         EGMStreamStart egmID1\SampleRate:=egm_sample_rate;
48     ENDIF
49 ENDPROC
50
51 PROC motion_program_egm_enable()
52     VAR num egm_cmd;
53     IF NOT try_motion_program_read_num(egm_cmd) THEN
54         RAISE ERR_INVALID_MP_FILE;
55     ENDIF
56
57     ! EGM no command specified, enable streaming
58     IF egm_cmd=0 THEN
59         IF have_egm THEN
60             EGMStreamStart egmID1\SampleRate:=egm_sample_rate;
61         ENDIF
62         RETURN ;
63     ENDIF
64
65     IF NOT have_egm THEN
66         RAISE ERR_NO_EGM;
67     ENDIF
68
69     ! EGM joint position command
70     IF egm_cmd=1 THEN
71         motion_program_egm_enable_joint;
72         RETURN ;
73     ENDIF
74
75     ! EGM pose command
76     IF egm_cmd=2 THEN
77         motion_program_egm_enable_pose;
78         RETURN ;
79     ENDIF
80
81     ! EGM pose command
82     IF egm_cmd=3 THEN
83         motion_program_egm_enable_corr;
84         RETURN ;
85     ENDIF
86
87     RAISE ERR_INVALID_OPCODE;
88
89 ENDPROC
90
91 PROC motion_program_egm_enable_joint()
92     VAR egm_minmax minmax{6};
93     VAR num pos_dev;
94     VAR num speed_dev;

```



```

95     VAR num i;
96     VAR jointtarget joints;
97     FOR i FROM 1 TO 6 DO
98         IF NOT try_read_egm_minmax(minmax{i}) THEN
99             RAISE ERR_INVALID_MP_FILE;
100         ENDIF
101     ENDFOR

102
103     IF NOT try_motion_program_read_num(pos_dev) AND
104         try_motion_program_read_num(speed_dev) THEN
105         RAISE ERR_INVALID_MP_FILE;
106     ENDIF

107     ! "Move" a tiny amount to start EGM
108     joints:=CJointT();
109     joints.robax.rax_6:=joints.robax.rax_6+.0001;
110     MoveAbsj joints,v1000,fine,motion_program_tool\WObj:=
111         motion_program_wobj;

112     EGMActJoint egmID1,\Tool:=motion_program_tool,\WObj:=
113         motion_program_wobj\J1:=minmax{1}
114         \J2:=minmax{2}\J3:=minmax{3}\J4:=minmax{4}\J5:=minmax{5}\J6
115         :=minmax{6}\MaxPosDeviation:=pos_dev\MaxSpeedDeviation:=
116         speed_dev;
117
118 ENDPROC

119
120 PROC motion_program_egm_enable_pose()
121     VAR pose posecor;
122     VAR egmframetype posecor_frtype;
123     VAR pose posesens;
124     VAR egmframetype posesens_frtype;
125     VAR egm_minmax minmax{6};
126     VAR num pos_dev;
127     VAR num speed_dev;
128     VAR num i;
129     VAR jointtarget joints;

130
131     IF NOT (
132         try_motion_program_read_pose(posecor)
133         AND try_read_egm_frtype(posecor_frtype)
134         AND try_motion_program_read_pose(posesens)
135         AND try_read_egm_frtype(posesens_frtype)
136     ) THEN
137         RAISE ERR_INVALID_MP_FILE;
138     ENDIF

139
140     FOR i FROM 1 TO 6 DO
141         IF NOT try_read_egm_minmax(minmax{i}) THEN
142             RAISE ERR_INVALID_MP_FILE;
143         ENDIF
144     ENDFOR

```

```

141
142     IF NOT try_motion_program_read_num(pos_dev) AND
143         try_motion_program_read_num(speed_dev) THEN
144         RAISE ERR_INVALID_MP_FILE;
145     ENDIF
146
147     EGMSetupUC ROB_1, egmID1, "pose", "UCdevice"\Pose\CommTimeout
148         :=1000000;
149
150     ! "Move" a tiny amount to start EGM
151     joints:=CJointT();
152     joints.robax.rax_6:=joints.robax.rax_6+.0001;
153     MoveAbsj joints, v1000, fine, motion_program_tool\WObj:=
154         motion_program_wobj;
155
156     EGMActPose egmID1, \StreamStart, \Tool:=motion_program_tool, \
157         WObj:=motion_program_wobj, posecor, posecor_frtype,
158         posesens, posesens_frtype,
159         \X:=minmax{1}\Y:=minmax{2}\Z:=minmax{3}\Rx:=minmax{4}\Ry
160             :=minmax{5}\Rz:=minmax{6}\MaxPosDeviation:=pos_dev\
161             MaxSpeedDeviation:=speed_dev;
162
163 ENDPROC
164
165 PROC motion_program_egm_enable_corr()
166     VAR pose posesens;
167     VAR jointtarget joints;
168     IF NOT try_motion_program_read_pose(posesens) THEN
169         RAISE ERR_INVALID_MP_FILE;
170     ENDIF
171
172     EGMSetupUC ROB_1, egmID1, "path_corr", "UCdevice"\PathCorr\APTR
173         ;
174
175     joints:=CJointT();
176     joints.robax.rax_6:=joints.robax.rax_6+.0001;
177     MoveAbsj joints, v1000, fine, motion_program_tool\WObj:=
178         motion_program_wobj;
179
180     EGMActMove egmID1, posesens\SampleRate:=48;
181
182 ENDPROC
183
184 FUNC bool try_read_egm_minmax(INOUT egm_minmax minmax)
185     RETURN (try_motion_program_read_num(minmax.min) AND
186         try_motion_program_read_num(minmax.max));
187
188 ENDFUNC
189
190 FUNC bool try_read_egm_frtype(INOUT egmframetype frtype)
191     VAR num frtype_code;
192     IF NOT try_motion_program_read_num(frtype_code) THEN
193         RETURN FALSE;

```

```

182         ENDIF
183
184         IF frtype_code=1 THEN
185             frtype:=EGM_FRAME_TOOL;
186         ELSEIF frtype_code=2 THEN
187             frtype:=EGM_FRAME_WOBJ;
188         ELSEIF frtype_code=3 THEN
189             frtype:=EGM_FRAME_WORLD;
190         ELSEIF frtype_code=4 THEN
191             frtype:=EGM_FRAME_JOINT;
192         ELSE
193             frtype:=EGM_FRAME_BASE;
194         ENDIF
195         RETURN TRUE;
196     ENDFUNC
197
198     FUNC bool try_motion_program_egmrunjoint()
199         VAR num condtime;
200         VAR num rampin;
201         VAR num rampout;
202
203         IF NOT (
204             try_motion_program_read_num(condtime)
205             AND try_motion_program_read_num(rampin)
206             AND try_motion_program_read_num(rampout)
207         ) THEN
208             RETURN FALSE;
209         ENDIF
210
211         SetDO motion_program_stop_egm,0;
212         EGMRunJoint egmID1,EGM_STOP_HOLD,\NoWaitCond\J1\J2\J3\J4\J5\
213             J6\CondTime:=condtime\RampInTime:=rampin;
214         WaitDO motion_program_stop_egm,1;
215         EGMStop egmID1,EGM_STOP_HOLD\RampOutTime:=rampout;
216         EGMStreamStart egmID1\SampleRate:=egm_sample_rate;
217
218         RETURN TRUE;
219     ENDFUNC
220
221     FUNC bool try_motion_program_egmrunpose()
222         VAR num condtime;
223         VAR num rampin;
224         VAR num rampout;
225         VAR pose offset;
226
227         IF NOT (
228             try_motion_program_read_num(condtime)
229             AND try_motion_program_read_num(rampin)
230             AND try_motion_program_read_num(rampout)
231             AND try_motion_program_read_pose(offset)
232         ) THEN

```

```

232         RETURN FALSE;
233     ENDIF
234
235     SetDO motion_program_stop_egm,0;
236     EGMRunPose egmID1,EGM_STOP_HOLD,\NoWaitCond\x\y\z\Rx\Ry\Rz\
        CondTime:=condtime\RampInTime:=rampin\RampOutTime:=
        rampout\Offset:=offset;
237     WaitDO motion_program_stop_egm,1;
238     EGMStop egmID1,EGM_STOP_HOLD\RampOutTime:=rampout;
239     EGMStreamStart egmID1\SampleRate:=egm_sample_rate;
240     RETURN TRUE;
241 ENDFUNC
242
243 FUNC bool try_motion_program_run_egmmovel(num cmd_num)
244     VAR robtarget rt;
245     VAR speeddata sd;
246     VAR zonedata zd;
247     IF NOT (
248         try_motion_program_read_rt(rt)
249         AND try_motion_program_read_sd(sd)
250         AND try_motion_program_read_zd(zd)
251     ) THEN
252         RETURN FALSE;
253     ENDIF
254
255     EGMMoveL egmID1,rt,sd,zd,motion_program_tool\Wobj:=
        motion_program_wobj;
256
257     RETURN TRUE;
258
259 ENDFUNC
260
261 FUNC bool try_motion_program_run_egmmovec(num cmd_num)
262     VAR robtarget rt1;
263     VAR robtarget rt2;
264     VAR speeddata sd;
265     VAR zonedata zd;
266     IF NOT (
267         try_motion_program_read_rt(rt1)
268         AND try_motion_program_read_rt(rt2)
269         AND try_motion_program_read_sd(sd)
270         AND try_motion_program_read_zd(zd)
271     ) THEN
272         RETURN FALSE;
273     ENDIF
274
275     EGMMoveC egmID1,rt1,rt2,sd,zd,motion_program_tool\Wobj:=
        motion_program_wobj;
276
277     RETURN TRUE;
278

```

```

279     ENDFUNC
280
281     FUNC bool try_motion_program_run_egm_cmd(num cmd_num,num cmd_op)
282         TEST cmd_op
283         CASE MOTION_PROGRAM_CMD_EGMJOINT:
284             RETURN try_motion_program_egmrunjoint();
285         CASE MOTION_PROGRAM_CMD_EGMPOSE:
286             RETURN try_motion_program_egmrunpose();
287         CASE MOTION_PROGRAM_CMD_EGMMOVEL:
288             RETURN try_motion_program_run_egmmovel(cmd_num);
289         CASE MOTION_PROGRAM_CMD_EGMMOVEC:
290             RETURN try_motion_program_run_egmmovec(cmd_num);
291         DEFAULT:
292             RAISE ERR_INVALID_OPCODE;
293         ENDTEST
294     ENDFUNC
295
296     ENDMODULE

```

motion_program_exec

```

1  MODULE motion_program_exec
2
3      CONST num MOTION_PROGRAM_DRIVER_MODE:=0;
4
5      CONST num MOTION_PROGRAM_CMD_NOOP:=0;
6      CONST num MOTION_PROGRAM_CMD_MOVEABSJ:=1;
7      CONST num MOTION_PROGRAM_CMD_MOVEJ:=2;
8      CONST num MOTION_PROGRAM_CMD_MOVEL:=3;
9      CONST num MOTION_PROGRAM_CMD_MOVEC:=4;
10     CONST num MOTION_PROGRAM_CMD_WAIT:=5;
11     CONST num MOTION_PROGRAM_CMD_CIRMODE:=6;
12     CONST num MOTION_PROGRAM_CMD_SYNCMOVEON:=7;
13     CONST num MOTION_PROGRAM_CMD_SYNCMOVEOFF:=8;
14
15     LOCAL VAR iodev motion_program_io_device;
16     LOCAL VAR rawbytes motion_program_bytes;
17     LOCAL VAR num motion_program_bytes_offset;
18
19     TASK PERS tooldata motion_program_tool:=[TRUE
20         ,[[0,0,0],[1,0,0,0]],[0.001,[0,0,0.001],[1,0,0,0],[0,0,0]];
21     TASK PERS wobjdata motion_program_wobj:=[FALSE,TRUE
22         ,""],[[0,0,0],[1,0,0,0]],[[0,0,0],[1,0,0,0]];
23     TASK PERS loaddata motion_program_griplload:=[0.001, [0, 0,
24         0.001],[1, 0, 0, 0], [0, 0, 0, 0];
25
26     LOCAL VAR rmqslot logger_rmq;
27
28

```

```

25     LOCAL VAR intnum motion_trigg_intno;
26     LOCAL VAR triggdata motion_trigg_data;
27     LOCAL VAR num motion_cmd_num_history{128};
28     LOCAL VAR num motion_current_cmd_ind;
29     LOCAL VAR num motion_max_cmd_ind;
30
31     VAR errnum ERR_INVALID_MP_VERSION:=-1;
32     VAR errnum ERR_INVALID_MP_FILE:=-1;
33     VAR errnum ERR_MISSED_PREEMPT:=-1;
34     VAR errnum ERR_INVALID_OPCODE:=-1;
35
36     LOCAL PERS tasks task_list{2}:=[["T_ROB1"],["T_ROB2"]];
37
38     VAR syncident motion_program_sync1;
39     VAR syncident motion_program_sync2;
40
41     LOCAL VAR num task_ind;
42     LOCAL VAR string motion_program_filename;
43
44     VAR bool motion_program_have_egm:=TRUE;
45
46     LOCAL VAR num motion_program_driver_seqno;
47
48     LOCAL VAR intnum motion_program_driver_abort_into;
49
50     PROC motion_program_main()
51         IF MOTION_PROGRAM_DRIVER_MODE = 0 THEN
52             motion_program_init;
53             run_motion_program_file(motion_program_filename);
54             motion_program_fini;
55         ELSE
56             motion_program_main_driver_mode;
57         ENDIF
58     ENDPROC
59
60     PROC motion_program_init()
61         VAR string taskname;
62         BookErrNo ERR_INVALID_MP_VERSION;
63         BookErrNo ERR_INVALID_MP_FILE;
64         BookErrNo ERR_MISSED_PREEMPT;
65         BookErrNo ERR_INVALID_OPCODE;
66         taskname:=GetTaskName();
67         IF taskname="T_ROB1" THEN
68             motion_program_filename:="motion_program.bin";
69             task_ind:=1;
70         ELSEIF taskname="T_ROB2" THEN
71             motion_program_filename:="motion_program2.bin";
72             task_ind:=2;
73         ENDIF
74         IF task_ind=1 THEN
75             SetAO motion_program_preempt,0;

```

```

76         SetAO motion_program_preempt_current,0;
77         SetAO motion_program_preempt_cmd_num,-1;
78         SetAO motion_program_current_cmd_num,-1;
79         SetAO motion_program_queued_cmd_num,-1;
80         SetAO motion_program_seqno,-1;
81     ENDIF
82     motion_program_state{task_ind}.current_cmd_num:=-1;
83     motion_program_state{task_ind}.queued_cmd_num:=-1;
84     motion_program_state{task_ind}.preempt_current:=0;
85     motion_current_cmd_ind:=0;
86     motion_max_cmd_ind:=0;
87     IDelete motion_trigg_intno;
88     CONNECT motion_trigg_intno WITH motion_trigg_trap;
89     TriggInt motion_trigg_data,0.001,\Start,motion_trigg_intno;
90     RMQFindSlot logger_rmq,"RMQ_logger";
91     try_motion_program_egm_init;
92 ENDPROC
93
94 PROC motion_program_fini()
95     ErrWrite\I,"Motion Program Complete","Motion Program
          Complete";
96     IDelete motion_trigg_intno;
97 ENDPROC
98
99 PROC run_motion_program_file(string filename)
100     ErrWrite\I,"Motion Program Begin","Motion Program Begin";
101     close_motion_program_file;
102     open_motion_program_file filename, FALSE;
103     ErrWrite\I,"Motion Program Start Program","Motion Program
          Start Program timestamp: "+motion_program_state{task_ind
          }.program_timestamp;
104     IF task_ind=1 THEN
105         motion_program_req_log_start;
106     ENDIF
107     motion_program_run;
108     close_motion_program_file;
109     IF task_ind=1 THEN
110         motion_program_req_log_end;
111     ENDIF
112 ENDPROC
113
114 PROC open_motion_program_file(string filename, bool preempt)
115     VAR num ver;
116     VAR tooldata mtool;
117     VAR wobjdata mwobj;
118     VAR loaddata mgriplload;
119     VAR string timestamp;
120     VAR num egm_cmd;
121     VAR num seqno;
122

```

```

123     motion_program_state{task_ind}.motion_program_filename:=
        filename;
124     motion_program_clear_bytes;
125     Open "RAMDISK:"\File:=filename,motion_program_io_device,\
        Read\Bin;
126     IF NOT try_motion_program_read_num(ver) THEN
127         RAISE ERR_FILESIZE;
128     ENDIF
129
130     IF ver<>motion_program_file_version THEN
131         ErrWrite "Invalid Motion Program","Invalid motion
            program file version";
132         RAISE ERR_INVALID_MP_VERSION;
133     ENDIF
134
135     IF NOT try_motion_program_read_td(mtool) THEN
136         ErrWrite "Invalid Motion Program Tool","Invalid motion
            program tool";
137         RAISE ERR_INVALID_MP_FILE;
138     ENDIF
139
140     IF NOT try_motion_program_read_wd(mwobj) THEN
141         ErrWrite "Invalid Motion Program Wobj","Invalid motion
            program wobj";
142         RAISE ERR_INVALID_MP_FILE;
143     ENDIF
144
145     IF NOT try_motion_program_read_ld(mgripload) THEN
146         ErrWrite "Invalid Motion Program GripLoad","Invalid
            motion program gripload";
147         RAISE ERR_INVALID_MP_FILE;
148     ENDIF
149
150     IF NOT try_motion_program_read_string(timestamp) THEN
151         ErrWrite "Invalid Motion Program Timestamp","Invalid
            motion program timestamp";
152         RAISE ERR_INVALID_MP_FILE;
153     ENDIF
154
155     IF NOT try_motion_program_read_num(seqno) THEN
156         ErrWrite "Invalid Motion Program Timestamp","Invalid
            motion program timestamp";
157         RAISE ERR_INVALID_MP_FILE;
158     ENDIF
159
160     IF NOT preempt THEN
161         motion_program_state{task_ind}.program_seqno:=seqno;
162         motion_program_state{task_ind}.program_timestamp:=
            timestamp;
163         IF task_ind=1 THEN
164             SetAO motion_program_seqno,seqno;

```



```

165         ENDIF
166     ENDIF
167
168     ErrWrite\I,"Motion Program Opened","Motion Program Opened
        with timestamp: "+timestamp;
169
170     IF NOT preempt THEN
171         motion_program_tool:=mtool;
172         motion_program_wobj:=mwobj;
173         motion_program_griplload:=mgriplload;
174
175         SetSysData motion_program_tool;
176         SetSysData motion_program_wobj;
177         GriplLoad motion_program_griplload;
178
179         IF motion_program_have_egm THEN
180             motion_program_egm_enable;
181         ELSE
182             IF NOT try_motion_program_read_num(egm_cmd) THEN
183                 RAISE ERR_INVALID_MP_FILE;
184             ENDIF
185             IF egm_cmd<>0 THEN
186                 RAISE ERR_INVALID_MP_FILE;
187             ENDIF
188         ENDIF
189     ELSE
190         IF NOT try_motion_program_read_num(egm_cmd) THEN
191             RAISE ERR_INVALID_MP_FILE;
192         ENDIF
193         IF egm_cmd<>0 THEN
194             RAISE ERR_INVALID_MP_FILE;
195         ENDIF
196     ENDIF
197
198 ENDPROC
199
200 PROC close_motion_program_file()
201     VAR string filename;
202     filename:=motion_program_state{task_ind}.
        motion_program_filename;
203     motion_program_state{task_ind}.motion_program_filename:="";
204     Close motion_program_io_device;
205 ERROR
206     !SkipWarn;
207     TRYNEXT;
208 ENDPROC
209
210 PROC motion_program_run()
211
212     VAR bool keepgoing:=TRUE;
213     VAR num cmd_num;

```

```

214     VAR num cmd_op;
215     motion_program_state{task_ind}.current_cmd_num:=-1;
216     IF task_ind=1 THEN
217         SetAO motion_program_current_cmd_num,-1;
218     ENDIF
219     motion_program_state{task_ind}.running:=TRUE;
220     WHILE keepgoing DO
221         motion_program_do_preempt;
222         keepgoing:=try_motion_program_run_next_cmd(cmd_num,
223             cmd_op);
224     ENDWHILE
225     WaitRob\ZeroSpeed;
226     motion_program_state{task_ind}.running:=FALSE;
227 ERROR
228     motion_program_state{task_ind}.running:=FALSE;
229     RAISE ;
230 ENDPROC
231
232 FUNC bool try_motion_program_run_next_cmd(INOUT num cmd_num,
233     INOUT num cmd_op)
234
235     VAR num local_cmd_ind;
236
237     IF NOT (try_motion_program_read_num(cmd_num) AND
238         try_motion_program_read_num(cmd_op)) THEN
239         RETURN FALSE;
240     ENDIF
241
242     !motion_program_state.current_cmd_num:=cmd_num;
243     motion_max_cmd_ind:=motion_max_cmd_ind+1;
244     motion_program_state{task_ind}.queued_cmd_num:=
245         motion_max_cmd_ind;
246
247     IF task_ind=1 THEN
248         SetAO motion_program_queued_cmd_num,motion_max_cmd_ind;
249     ENDIF
250     local_cmd_ind:=((motion_max_cmd_ind-1) MOD 128)+1;
251
252     IF (cmd_op DIV 10000)=5 THEN
253         ! EGM command numbers start at 50000
254         motion_cmd_num_history{local_cmd_ind}:=-1;
255         RETURN try_motion_program_run_egm_cmd(cmd_num,cmd_op);
256     ENDIF
257
258     TEST cmd_op
259     CASE MOTION_PROGRAM_CMD_NOOP:
260         RETURN TRUE;
261     CASE MOTION_PROGRAM_CMD_MOVEABSJ:
262         motion_cmd_num_history{local_cmd_ind}:=-1;
263         RETURN try_motion_program_run_moveabsj(cmd_num);
264     CASE MOTION_PROGRAM_CMD_MOVEJ:
265         motion_cmd_num_history{local_cmd_ind}:=-1;

```

```

261         RETURN try_motion_program_run_movej(cmd_num);
262     CASE MOTION_PROGRAM_CMD_MOVEL:
263         motion_cmd_num_history{local_cmd_ind}:=cmd_num;
264         RETURN try_motion_program_run_movel(cmd_num);
265     CASE MOTION_PROGRAM_CMD_MOVEC:
266         motion_cmd_num_history{local_cmd_ind}:=cmd_num;
267         RETURN try_motion_program_run_movec(cmd_num);
268     CASE MOTION_PROGRAM_CMD_WAIT:
269         motion_cmd_num_history{local_cmd_ind}:= -1;
270         RETURN try_motion_program_wait(cmd_num);
271     CASE MOTION_PROGRAM_CMD_CIRMODE:
272         motion_cmd_num_history{local_cmd_ind}:= -1;
273         RETURN try_motion_program_set_cirmode(cmd_num);
274     CASE MOTION_PROGRAM_CMD_SYNCMOVEON:
275         motion_cmd_num_history{local_cmd_ind}:= -1;
276         RETURN try_motion_program_sync_move_on(cmd_num);
277     CASE MOTION_PROGRAM_CMD_SYNCMOVEOFF:
278         motion_cmd_num_history{local_cmd_ind}:= -1;
279         RETURN try_motion_program_sync_move_off(cmd_num);
280     DEFAULT:
281         RAISE ERR_INVALID_OPCODE;
282     ENDTEST
283
284
285     ENDFUNC
286
287     !!! All these FUNCS below are for the definition and execution of
288     movements
289     FUNC bool try_motion_program_run_moveabsj(num cmd_num)
290     VAR jointtarget j;
291     VAR speeddata sd;
292     VAR zonedata zd;
293     IF NOT (
294     try_motion_program_read_jt(j)
295     AND try_motion_program_read_sd(sd)
296     AND try_motion_program_read_zd(zd)
297     ) THEN
298         RETURN FALSE;
299     ENDIF
300     IF IsSyncMoveOn() THEN
301         MoveAbsJ j,\ID:=cmd_num,sd,zd,motion_program_tool\Wobj:=
302         motion_program_wobj;
303     ELSE
304         MoveAbsJ j,sd,zd,motion_program_tool\Wobj:=
305         motion_program_wobj;
306     ENDIF
307     RETURN TRUE;
308     ENDFUNC
309
310     FUNC bool try_motion_program_run_movej(num cmd_num)
311     VAR robtarget rt;

```

```

309     VAR speeddata sd;
310     VAR zonedata zd;
311     IF NOT (
312         try_motion_program_read_rt(rt)
313         AND try_motion_program_read_sd(sd)
314         AND try_motion_program_read_zd(zd)
315     ) THEN
316         RETURN FALSE;
317     ENDIF
318     IF IsSyncMoveOn() THEN
319         TriggJ rt,\ID:=cmd_num,sd,motion_trigg_data,zd,
320             motion_program_tool\Wobj:=motion_program_wobj;
321     ELSE
322         TriggJ rt,sd,motion_trigg_data,zd,motion_program_tool\
323             WObj:=motion_program_wobj;
324     ENDIF
325     RETURN TRUE;
326
327 ENDFUNC
328
329 FUNC bool try_motion_program_run_moveI(num cmd_num)
330     VAR robtarget rt;
331     VAR speeddata sd;
332     VAR zonedata zd;
333     IF NOT (
334         try_motion_program_read_rt(rt)
335         AND try_motion_program_read_sd(sd)
336         AND try_motion_program_read_zd(zd)
337     ) THEN
338         RETURN FALSE;
339     ENDIF
340     IF IsSyncMoveOn() THEN
341         TriggL rt,\ID:=cmd_num,sd,motion_trigg_data,zd,
342             motion_program_tool\Wobj:=motion_program_wobj;
343     ELSE
344         TriggL rt,sd,motion_trigg_data,zd,motion_program_tool\
345             WObj:=motion_program_wobj;
346     ENDIF
347     RETURN TRUE;
348
349 ENDFUNC
350
351 FUNC bool try_motion_program_run_moveC(num cmd_num)
352     VAR robtarget rt1;
353     VAR robtarget rt2;
354     VAR speeddata sd;
355     VAR zonedata zd;
356     IF NOT (
357         try_motion_program_read_rt(rt1)
358         AND try_motion_program_read_rt(rt2)
359         AND try_motion_program_read_sd(sd)

```

```

356     AND try_motion_program_read_zd(zd)
357 ) THEN
358     RETURN FALSE;
359 ENDIF
360 IF IsSyncMoveOn() THEN
361     TriggC rt1,rt2,\ID:=cmd_num,sd,motion_trigg_data,zd,
362         motion_program_tool\Wobj:=motion_program_wobj;
363 ELSE
364     TriggC rt1,rt2,sd,motion_trigg_data,zd,
365         motion_program_tool\Wobj:=motion_program_wobj;
366 ENDIF
367 RETURN TRUE;
368
369 ENDFUNC
370
371 FUNC bool try_motion_program_wait(num cmd_num)
372     VAR num t;
373     IF NOT try_motion_program_read_num(t) THEN
374         RETURN FALSE;
375     ENDIF
376     WaitRob\ZeroSpeed;
377     motion_program_state{task_ind}.current_cmd_num:=cmd_num;
378     IF task_ind=1 THEN
379         SetAO motion_program_current_cmd_num,cmd_num;
380     ENDIF
381     WaitTime t;
382     RETURN TRUE;
383 ENDFUNC
384
385 FUNC bool try_motion_program_set_cirmode(num cmd_num)
386     VAR num switch;
387     IF NOT try_motion_program_read_num(switch) THEN
388         RETURN FALSE;
389     ENDIF
390     motion_program_state{task_ind}.current_cmd_num:=cmd_num;
391     IF task_ind=1 THEN
392         SetAO motion_program_current_cmd_num,cmd_num;
393     ENDIF
394     TEST switch
395     CASE 1:
396         CirPathMode\PathFrame;
397     CASE 2:
398         CirPathMode\ObjectFrame;
399     CASE 3:
400         CirPathMode\CirPointOri;
401     CASE 4:
402         CirPathMode\Wrist45;
403     CASE 5:
404         CirPathMode\Wrist46;
405     CASE 6:
406         CirPathMode\Wrist56;

```

```

405         ENDTEST
406         RETURN TRUE;
407     ENDFUNC
408
409     FUNC bool try_motion_program_sync_move_on(num cmd_num)
410         SyncMoveOn motion_program_sync1,task_list;
411         RETURN TRUE;
412     ENDFUNC
413
414     FUNC bool try_motion_program_sync_move_off(num cmd_num)
415         SyncMoveOff motion_program_sync2;
416         RETURN TRUE;
417     ENDFUNC
418
419     !!! All the FUNCS below are for reading purposes
420
421     FUNC bool try_motion_program_read_jt(INOUT jointtarget j)
422         IF NOT (
423             try_motion_program_read_num(j.robax.rax_1)
424             AND try_motion_program_read_num(j.robax.rax_2)
425             AND try_motion_program_read_num(j.robax.rax_3)
426             AND try_motion_program_read_num(j.robax.rax_4)
427             AND try_motion_program_read_num(j.robax.rax_5)
428             AND try_motion_program_read_num(j.robax.rax_6)
429             AND try_motion_program_read_num(j.extax.eax_a)
430             AND try_motion_program_read_num(j.extax.eax_b)
431             AND try_motion_program_read_num(j.extax.eax_c)
432             AND try_motion_program_read_num(j.extax.eax_d)
433             AND try_motion_program_read_num(j.extax.eax_e)
434             AND try_motion_program_read_num(j.extax.eax_f)
435         ) THEN
436             RETURN FALSE;
437         ENDIF
438         RETURN TRUE;
439     ENDFUNC
440
441     FUNC bool try_motion_program_read_sd(INOUT speeddata sd)
442         IF NOT (
443             try_motion_program_read_num(sd.v_tcp)
444             AND try_motion_program_read_num(sd.v_ori)
445             AND try_motion_program_read_num(sd.v_leax)
446             AND try_motion_program_read_num(sd.v_reax)
447         ) THEN
448             RETURN FALSE;
449         ENDIF
450         RETURN TRUE;
451     ENDFUNC
452
453     FUNC bool try_motion_program_read_zd(INOUT zonedata zd)
454         VAR num finep_num:=0;
455         IF NOT (

```

```

456     try_motion_program_read_num(finep_num)
457     AND try_motion_program_read_num(zd.pzone_tcp)
458     AND try_motion_program_read_num(zd.pzone_ori)
459     AND try_motion_program_read_num(zd.pzone_eax)
460     AND try_motion_program_read_num(zd.zone_ori)
461     AND try_motion_program_read_num(zd.zone_leax)
462     AND try_motion_program_read_num(zd.zone_reax)
463 ) THEN
464     RETURN FALSE;
465 ENDIF
466 zd.finep:=finep_num<>0;
467 RETURN TRUE;
468 ENDFUNC
469
470 FUNC bool try_motion_program_read_pose(INOUT pose p)
471     IF NOT (
472         try_motion_program_read_num(p.trans.x)
473         AND try_motion_program_read_num(p.trans.y)
474         AND try_motion_program_read_num(p.trans.z)
475         AND try_motion_program_read_num(p.rot.q1)
476         AND try_motion_program_read_num(p.rot.q2)
477         AND try_motion_program_read_num(p.rot.q3)
478         AND try_motion_program_read_num(p.rot.q4)
479     ) THEN
480         RETURN FALSE;
481     ENDIF
482     RETURN TRUE;
483 ENDFUNC
484
485 FUNC bool try_motion_program_read_rt(INOUT robtarget rt)
486     IF NOT (
487         try_motion_program_read_num(rt.trans.x)
488         AND try_motion_program_read_num(rt.trans.y)
489         AND try_motion_program_read_num(rt.trans.z)
490         AND try_motion_program_read_num(rt.rot.q1)
491         AND try_motion_program_read_num(rt.rot.q2)
492         AND try_motion_program_read_num(rt.rot.q3)
493         AND try_motion_program_read_num(rt.rot.q4)
494         AND try_motion_program_read_num(rt.robconf.cf1)
495         AND try_motion_program_read_num(rt.robconf.cf4)
496         AND try_motion_program_read_num(rt.robconf.cf6)
497         AND try_motion_program_read_num(rt.robconf.cfx)
498         AND try_motion_program_read_num(rt.extax.eax_a)
499         AND try_motion_program_read_num(rt.extax.eax_b)
500         AND try_motion_program_read_num(rt.extax.eax_c)
501         AND try_motion_program_read_num(rt.extax.eax_d)
502         AND try_motion_program_read_num(rt.extax.eax_e)
503         AND try_motion_program_read_num(rt.extax.eax_f)
504     ) THEN
505         RETURN FALSE;
506     ENDIF

```

```

507         RETURN TRUE;
508     ENDFUNC
509
510     FUNC bool try_motion_program_read_td(INOUT tooldata td)
511         VAR num robhold_num;
512         IF NOT (
513             try_motion_program_read_num(robhold_num)
514             AND try_motion_program_read_num(td.tframe.trans.x)
515             AND try_motion_program_read_num(td.tframe.trans.y)
516             AND try_motion_program_read_num(td.tframe.trans.z)
517             AND try_motion_program_read_num(td.tframe.rot.q1)
518             AND try_motion_program_read_num(td.tframe.rot.q2)
519             AND try_motion_program_read_num(td.tframe.rot.q3)
520             AND try_motion_program_read_num(td.tframe.rot.q4)
521             AND try_motion_program_read_num(td.tload.mass)
522             AND try_motion_program_read_num(td.tload.cog.x)
523             AND try_motion_program_read_num(td.tload.cog.y)
524             AND try_motion_program_read_num(td.tload.cog.z)
525             AND try_motion_program_read_num(td.tload.aom.q1)
526             AND try_motion_program_read_num(td.tload.aom.q2)
527             AND try_motion_program_read_num(td.tload.aom.q3)
528             AND try_motion_program_read_num(td.tload.aom.q4)
529             AND try_motion_program_read_num(td.tload.ix)
530             AND try_motion_program_read_num(td.tload.iy)
531             AND try_motion_program_read_num(td.tload.iz)
532         )
533         THEN
534             RETURN FALSE;
535         ENDIF
536         td.robhold:=robhold_num<>0;
537         RETURN TRUE;
538     ENDFUNC
539
540     FUNC bool try_motion_program_read_ld(INOUT loaddata ld)
541         IF NOT (
542             try_motion_program_read_num(ld.mass)
543             AND try_motion_program_read_num(ld.cog.x)
544             AND try_motion_program_read_num(ld.cog.y)
545             AND try_motion_program_read_num(ld.cog.z)
546             AND try_motion_program_read_num(ld.aom.q1)
547             AND try_motion_program_read_num(ld.aom.q2)
548             AND try_motion_program_read_num(ld.aom.q3)
549             AND try_motion_program_read_num(ld.aom.q4)
550             AND try_motion_program_read_num(ld.ix)
551             AND try_motion_program_read_num(ld.iy)
552             AND try_motion_program_read_num(ld.iz)
553         )
554         THEN
555             RETURN FALSE;
556         ENDIF
557         RETURN TRUE;

```



```

558     ENDFUNC
559
560     FUNC bool try_motion_program_read_wd(INOUT wobjdata wd)
561         VAR num robhold_num;
562         VAR num ufprog_num;
563         IF NOT (
564             try_motion_program_read_num(robhold_num)
565             AND try_motion_program_read_num(ufprog_num)
566             AND try_motion_program_read_string(wd.ufmec)
567             AND try_motion_program_read_num(wd.uframe.trans.x)
568             AND try_motion_program_read_num(wd.uframe.trans.y)
569             AND try_motion_program_read_num(wd.uframe.trans.z)
570             AND try_motion_program_read_num(wd.uframe.rot.q1)
571             AND try_motion_program_read_num(wd.uframe.rot.q2)
572             AND try_motion_program_read_num(wd.uframe.rot.q3)
573             AND try_motion_program_read_num(wd.uframe.rot.q4)
574             AND try_motion_program_read_num(wd.oframe.trans.x)
575             AND try_motion_program_read_num(wd.oframe.trans.y)
576             AND try_motion_program_read_num(wd.oframe.trans.z)
577             AND try_motion_program_read_num(wd.oframe.rot.q1)
578             AND try_motion_program_read_num(wd.oframe.rot.q2)
579             AND try_motion_program_read_num(wd.oframe.rot.q3)
580             AND try_motion_program_read_num(wd.oframe.rot.q4)
581         )
582         THEN
583             RETURN FALSE;
584         ENDIF
585         wd.robhold:=robhold_num<>0;
586         wd.ufprog:=ufprog_num<>0;
587         RETURN TRUE;
588     ENDFUNC
589
590     FUNC bool try_motion_program_fill_bytes()
591         IF RawBytesLen(motion_program_bytes)=0 OR
592            motion_program_bytes_offset>RawBytesLen(
593            motion_program_bytes) THEN
594             ClearRawBytes motion_program_bytes;
595             motion_program_bytes_offset:=1;
596             ReadRawBytes motion_program_io_device,
597                motion_program_bytes;
598             IF RawBytesLen(motion_program_bytes)=0 THEN
599                 RETURN FALSE;
600             ENDIF
601         ENDIF
602         RETURN TRUE;
603     ERROR
604         IF ERRNO=ERR_RANYBIN_EOF THEN
605             SkipWarn;
606             TRYNEXT;
607         ENDIF
608     ENDFUNC

```

```

606
607 PROC motion_program_clear_bytes()
608     ClearRawBytes motion_program_bytes;
609 ENDPROC
610
611 FUNC bool try_motion_program_read_num(INOUT num val)
612     IF NOT try_motion_program_fill_bytes() THEN
613         val:=0;
614         RETURN FALSE;
615     ENDIF
616     UnpackRawBytes motion_program_bytes,
        motion_program_bytes_offset, val, \Float4;
617     motion_program_bytes_offset:=motion_program_bytes_offset+4;
618     RETURN TRUE;
619 ENDFUNC
620
621 FUNC bool try_motion_program_read_string(INOUT string val)
622     VAR num str_len;
623     VAR string str1;
624     VAR num str1_len;
625     VAR string str2;
626     VAR num str2_len;
627     IF NOT (
628         try_motion_program_fill_bytes()
629         AND try_motion_program_read_num(str_len)
630         AND try_motion_program_fill_bytes()
631     )
632     THEN
633         val:="";
634         RETURN FALSE;
635     ENDIF
636     IF RawBytesLen(motion_program_bytes)>=(
        motion_program_bytes_offset+31) THEN
637         UnpackRawBytes motion_program_bytes,
            motion_program_bytes_offset, val, \ASCII:=str_len;
638         motion_program_bytes_offset:=motion_program_bytes_offset
            +32;
639         RETURN TRUE;
640     ELSE
641         str1_len:=(RawBytesLen(motion_program_bytes)+1)-
            motion_program_bytes_offset;
642         IF str1_len>=str_len THEN
643             UnpackRawBytes motion_program_bytes,
                motion_program_bytes_offset, val, \ASCII:=str_len;
644             motion_program_bytes_offset:=
                motion_program_bytes_offset+str1_len;
645             IF NOT try_motion_program_fill_bytes() THEN
646                 RETURN FALSE;
647             ENDIF
648             motion_program_bytes_offset:=
                motion_program_bytes_offset+(32-str1_len);

```

```

649         RETURN TRUE;
650     ELSE
651         UnpackRawBytes motion_program_bytes,
            motion_program_bytes_offset, str1, \ASCII:=
            str1_len;
652         motion_program_bytes_offset:=
            motion_program_bytes_offset+str1_len;
653         IF NOT try_motion_program_fill_bytes() THEN
654             RETURN FALSE;
655         ENDIF
656         str2_len:=str_len-str1_len;
657         UnpackRawBytes motion_program_bytes,
            motion_program_bytes_offset, str2, \ASCII:=
            str2_len;
658         motion_program_bytes_offset:=
            motion_program_bytes_offset+(32-str1_len);
659         val:=str1+str2;
660         RETURN TRUE;
661     ENDIF
662 ENDIF
663
664 ENDFUNC
665
666 !!! All these FUNCs are for reading purposes
667
668 PROC motion_program_req_log_start()
669     VAR string msg{2};
670     msg{1}:=motion_program_state{task_ind}.program_timestamp;
671     msg{2}:=motion_program_state{task_ind}.program_timestamp;
672     IF MOTION_PROGRAM_DRIVER_MODE = 1 THEN
673         msg{2} := "motion_program---seqno-" + NumToStr(
            motion_program_state{task_ind}.program_seqno, 0);
674     ENDIF
675     RMQSendMessage logger_rmq, msg;
676 ENDPROC
677
678 PROC motion_program_req_log_end()
679     VAR string msg{2};
680     RMQSendMessage logger_rmq, msg;
681 ENDPROC
682
683 PROC motion_program_do_preempt()
684     VAR string filename;
685     IF motion_program_preempt>motion_program_state{task_ind}.
        preempt_current THEN
686         IF motion_max_cmd_ind=motion_program_preempt_cmd_num
            THEN
687             IF task_ind=1 THEN
688                 filename:=StrFormat("motion_program_p{1}"\Arg1:=
                    NumToStr(motion_program_preempt, 0));
689             ELSE

```

```

690         filename:=StrFormat("motion_program2_p{1}"\Arg1
        :=NumToStr(motion_program_preempt,0));
691     ENDIF
692     IF MOTION_PROGRAM_DRIVER_MODE = 1 THEN
693         filename:= filename + "---seqno-" + NumToStr(
        motion_program_state{task_ind}.program_seqno
        ,0);
694     ENDIF
695     filename:=filename + ".bin";
696     ErrWrite\I,"Preempting Motion Program","Preempting
        motion program with file "+filename;
697     IF task_ind=1 THEN
698         SetAO motion_program_preempt_current,
        motion_program_preempt;
699     ENDIF
700     motion_program_state{task_ind}.preempt_current:=
        motion_program_preempt;
701     close_motion_program_file;
702     open_motion_program_file filename, TRUE;
703     ELSEIF motion_max_cmd_ind>motion_program_preempt_cmd_num
        THEN
704         ErrWrite "Missed Preempt","Preempt command number
        missed";
705         RAISE ERR_MISSED_PREEMPT;
706     ENDIF
707     ENDIF
708 ENDPROC
709
710 TRAP motion_trigg_trap
711     VAR num cmd_ind;
712     VAR num local_cmd_ind;
713     VAR num cmd_num:=-1;
714     WHILE cmd_num=-1 AND (NOT motion_current_cmd_ind>
        motion_max_cmd_ind) DO
715         motion_current_cmd_ind:=motion_current_cmd_ind+1;
716         IF motion_current_cmd_ind>motion_max_cmd_ind THEN
717             cmd_ind:=motion_max_cmd_ind;
718         ELSE
719             cmd_ind:=motion_current_cmd_ind;
720         ENDIF
721         local_cmd_ind:=((cmd_ind-1) MOD 128)+1;
722         cmd_num:=motion_cmd_num_history{local_cmd_ind};
723     ENDWHILE
724
725     IF cmd_num<>-1 THEN
726         motion_program_state{task_ind}.current_cmd_num:=cmd_num;
727         IF task_ind=1 THEN
728             SetAO motion_program_current_cmd_num,cmd_num;
729         ENDIF
730     ENDIF
731 ENDTRAP

```

```

732
733 PROC try_motion_program_egm_init()
734     motion_program_egm_init;
735 ERROR
736     IF ERRNO=ERR_REFUNKPRC THEN
737         SkipWarn;
738         motion_program_have_egm:=FALSE;
739         TRYNEXT;
740     ENDIF
741 ENDPROC
742
743 PROC motion_program_main_driver_mode()
744     IDelete motion_program_driver_abort_into;
745     CONNECT motion_program_driver_abort_into WITH
746         motion_program_abort_driver_mode;
747     motion_program_driver_seqno:=-1;
748     motion_program_init;
749     motion_program_egm_start_stream;
750     WaitUntil motion_program_seqno_command >
751         motion_program_seqno_started AND
752         motion_program_driver_abort = 0;
753     SetAO motion_program_seqno_started,
754         motion_program_seqno_command;
755     ISignalDO motion_program_driver_abort, 1,
756         motion_program_driver_abort_into;
757     motion_program_driver_seqno:=motion_program_seqno_command;
758     ErrWrite\I,"Motion Program Driver Begin Program",StrFormat("
759         Motion Program Driver Begin Program seqno "\Arg1:=
760         NumToStr(motion_program_driver_seqno,0));
761     IF task_ind=1 THEN
762         motion_program_filename:="motion_program---seqno-" +
763             NumToStr(motion_program_driver_seqno,0) + ".bin";
764     ELSEIF task_ind=2 THEN
765         motion_program_filename:="motion_program2---seqno-" +
766             NumToStr(motion_program_driver_seqno,0) + ".bin";
767     ENDIF
768     run_motion_program_file(motion_program_filename);
769     !motion_program_reset_handler;
770     motion_program_fini_driver_mode;
771     ExitCycle;
772 ERROR
773     IF motion_program_driver_seqno >
774         motion_program_seqno_complete THEN
775         SetAO motion_program_seqno_complete,
776             motion_program_driver_seqno;
777         ErrWrite\I,"Motion Program Driver Program Complete","
778             Motion Program Complete";
779     ENDIF
780     RAISE;
781 ENDPROC

```

```
771     PROC motion_program_fini_driver_mode()
772         IF MOTION_PROGRAM_DRIVER_MODE = 1 THEN
773             IF motion_program_driver_seqno >
774                 motion_program_seqno_complete THEN
775                 SetA0 motion_program_seqno_complete,
776                     motion_program_driver_seqno;
777                 ErrWrite\I,"Motion Program Driver Program Complete
778                     ","Motion Program Complete";
779             ENDIF
780         ENDIF
781     ENDPROC
782
783     TRAP motion_program_abort_driver_mode
784         motion_program_fini_driver_mode;
785         ExitCycle;
786     ENDTRAP
787
788     PROC motion_program_stop_handler()
789
790     ENDPROC
791
792     ENDMODULE
```